SENTINELX: CYBER ATTACK INTELLIGENCE AND THREAT HUNTING PLATFORM

PROJECT REPORT (REVIEW 3: FINAL)

Submitted by

Shlok Gupta (24BCI0061)

Shreya Bhushan (24BCI0086)

Yashneil Purkayastha (24BDS0158)

Prepared For

DATABASE SYSTEMS (BCSE302P) - Lab PROJECT

Submitted To

Dr. Anand Bihari

School of Computer Science and Engineering



VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

Abstract

Most organisations already collect a lot of security data, such as firewall logs, intrusion alerts and lists of known bad IP addresses from threat feeds. The problem is that this data sits in different places, and news about a bad IP usually arrives days after that IP has already been probing the network. SentinelX is a small Security Operations Centre (SOC) built around one relational database, so that all of this is stored once, linked properly and searched with plain SQL.

The database has 16 tables in Third Normal Form (3NF) and an append-only audit trail, on MySQL 8.0.46. Review 1 covered the design and Review 2 showed it working. This final report brings both together and adds what Review 3 asks for: a security layer in which restricted users reach data only through views and procedures, an audit trail of every change to an incident, a demonstration of SQL injection and its fix, transactions under concurrency (lost update, row locking, deadlock and isolation levels), query optimisation with EXPLAIN and EXPLAIN ANALYZE, backup with point-in-time recovery, and 59 automatic tests that run as four different users. While testing our own design we found and fixed eight real defects, among them a privilege escalation through a stored procedure and an audit trail the administrator could forge; they are described where they occur. Every statement in this report was run on MySQL 8.0.46. The figures are images rendered from the captured mysql client output of that run, which is kept unchanged in the mysql-output folder of the repository.

We also narrowed the scope. The application layer (FastAPI, React) and the machine-learning parts described in Review 2 were never built; they are dropped from scope, and section 17 lists every change and the reason for it.

Abstract 2
1. Introduction 5
2. Objectives 5
3. Project Scope 6
4. Key Contacts and Stakeholders 6
5. Project Resource Requirements 6
5.1 Hardware Resource Requirements 6
5.2 Software Resource Requirements 6
6. System Architecture 8
7. Module-wise Functional Description 8
8. ER Diagram 10
9. Tables and Constraints 13
9.1 Normalisation, and the anomaly it prevents 13
9.2 The tables 14
10. Data Population (INSERT, UPDATE, DELETE) 20
11. SQL Implementation 23
11.1 A. Basic queries 23
11.2 B. Aggregate queries 26
11.3 C. Joins 28
11.4 D. Subqueries 31
11.5 E. Set operations 34
11.6 F. Advanced functions 35
11.7 G. Window functions 38
12. Advanced SQL Implementation 40
12.1 Complex Business Queries 40
12.2 Views 50
12.3 Stored Procedures 56
12.4 User-Defined Functions 58
12.5 Triggers 59
12.6 Cursors 64
12.7 SQL Security Implementation 66
13. Transactions and Concurrency 78
13.1 Concurrency 80
14. Indexing and Query Optimisation 86
14.1 When an index is not used 89

15. Backup and Point-in-Time Recovery 91
16. Testing 93
17. Scope Revision and Design Changes since Review 2 96
18. Review 3 Requirements Checklist 97
19. Limitations and Future Scope 98
20. Conclusion 99
21. Acknowledgement 99
22. Work Break Down 99
23. Review Evaluation 99

1. Introduction

Anyone who has seen how a small security team works knows the routine. An alert fires, someone opens the firewall console, then the IDS, then a spreadsheet of known bad IPs, and tries to match timestamps by hand. Each tool keeps its own copy of the facts, so the same attacker or server shows up in several places with slightly different details. Threat feeds also tend to be late. By the time an IP address appears on a blocklist it has often been scanning people for weeks, and nobody goes back to check last month's logs.

We picked this topic for our DBMS lab project because it fits a relational database very naturally. There are clear entities (servers, attackers, malware, vulnerabilities, incidents), clear relationships between them (an attack hits one server, an incident can group many attacks) and good reasons to use constraints, triggers, transactions and indexes. Checking old logs for a newly reported bad IP is still a manual job in most tools, so we made the database do that part by itself, and that became the main idea of the project.

Review 3 is about turning that working design into something we can defend: every rule enforced by the database, every user limited to what they need, every change to an incident traceable, every mistake recoverable, and every claim backed by a test that passes.

Terms used in this report:

Term	Meaning
SOC	Security operations centre: the team that watches for and handles attacks.
IOC	Indicator of compromise: a value known to be hostile, such as an attacker's IP address or a malware hash.
CVE, CVSS	A published software weakness (CVE-2024-3400) and its severity score from 0 to 10.
MITRE ATT&CK	A public catalogue of attack techniques; each attack type stores its technique ID (T1110).
Retro-hunt	Searching old logs for a newly reported bad IP, for the time it was hostile but not yet known.
DEFINER / INVOKER	Whose rights a view, procedure or trigger runs with: its creator's (DEFINER, the MySQL default) or the caller's (INVOKER).
Skip scan	A MySQL 8 index access that jumps through a composite index once per value of its first column.
Next-key lock	An InnoDB lock on an index record and the gap before it, used by locking reads.
Binary log	MySQL's ordered record of every committed change, used for recovery and replication.

2. Objectives

- Design a normalised relational schema in Third Normal Form (3NF) for a security operations centre that avoids update, insert and delete problems.
- Draw a clean ER diagram that keeps only the attributes we really use, and make sure it matches the tables we actually built.
- Keep every column only as large as it needs to be and back every rule with a constraint.

- Let the database do routine work by itself, like opening incidents for serious attacks and checking old logs when a new bad IP is added.
- Prove the design with real execution: queries, views, routines, triggers, transactions and security, each with a screenshot of its output.
- Secure the database itself: least privilege, row-level access, an audit trail that even the administrator cannot erase, protection against SQL injection, and recovery from mistakes.
- Back every claim with an automatic test that prints PASS or FAIL.

3. Project Scope

- Basic data about servers and devices (assets), CVEs, attack types mapped to MITRE ATT&CK, attacker groups and malware.
- Storing raw network logs and detected attacks, with indexes on time and source IP so searches over the large tables stay fast.
- Search and reporting views over attacks, assets and risk.
- Incident handling: automatic creation, assigning an analyst safely, escalation and closing.
- Retro-hunting and the time machine for indicators of compromise.
- User roles (admin, analyst, viewer) with least-privilege access: restricted users reach data only through views and procedures.
- An append-only audit trail of incident changes, and backup with point-in-time recovery.

Out of scope: real-time detection of attacks, a user interface and machine-learning models. Attacks arrive as data, as they would from a firewall or IDS, and people use the mysql client or MySQL Workbench.

4. Key Contacts and Stakeholders

Name	Registration Number	Role	Phone Number / Email
Shlok Gupta	24BCI0061	Team Member	
Shreya Bhushan	24BCI0086	Team Member	
Yashneil Purkayastha	24BDS0158	Team Member	
Dr. Anand Bihari		Faculty Guide	

5. Project Resource Requirements

5.1 Hardware Resource Requirements

- A laptop or desktop with an Intel i5 (8th gen) or Ryzen 5 and at least 8 GB RAM.
- About 3 GB of free disk space for MySQL, the tools and the database.

5.2 Software Resource Requirements

- Windows 11 (the machine all output was captured on).
- MySQL Community Server 8.0.46 with the mysql command-line client. INTERSECT and EXCEPT need 8.0.31 or later; CHECK constraints are enforced from 8.0.16.

- MySQL Workbench for browsing the schema.
- Python 3 for the scripts in tools/ that generate the test data and the step files, render the captured mysql output into the figures, draw two of the diagrams and lay out this report.
- Windows PowerShell for the one-click runner (runner/RUN_ME.bat).
- draw.io for planning the ER diagram, and Git with a shared GitHub repository.

6. System Architecture

The database is the whole system. The test data is loaded by plain INSERT scripts; new log batches go through sp_load_log_batch, which loads a batch as one transaction and rolls the whole batch back if any line fails. Everything that must never be skipped (opening incidents, retro-hunts, protecting indicator history, recording who changed an incident) is a constraint or trigger inside MySQL. People connect with their own MySQL accounts, and what they can do is decided by the role on their account: the administrator works with the data, the analyst and the viewer only through views and procedures. Backups use mysqldump and the binary log.

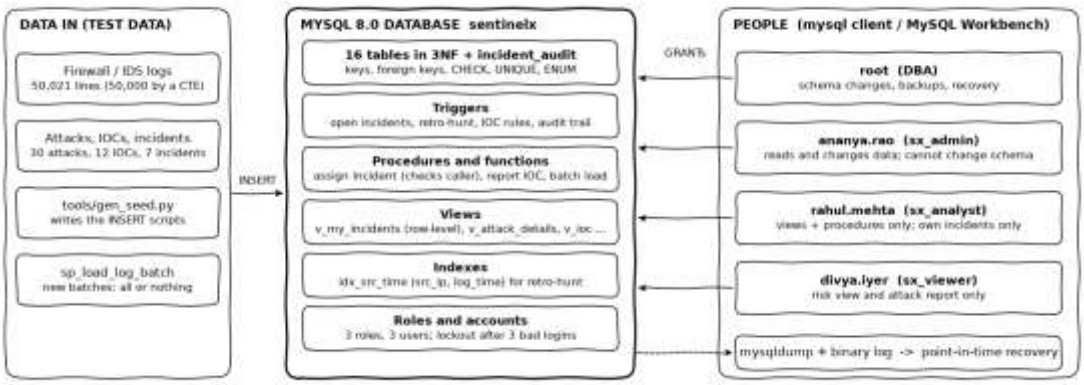


Fig. 1 System architecture

7. Module-wise Functional Description

Module	What it does	Database objects behind it
Search and reporting	Attacks per asset, open vulnerabilities, risk, and attack search by IP, country, type, malware, CVE, severity and date	v_attack_details, v_asset_risk, fn_asset_risk, queries in section 11
Incident handling	Opens an incident for any attack of severity 8 or more, groups repeat attacks within an hour, assigns analysts safely, escalates priority	trg_attack_auto_incident, sp_assign_incident, sp_escalate_incidents
Retro-hunt	When a new bad IP is added, searches old logs from the time it was already hostile but not yet known, and opens an incident if it finds anything	trg_ioc_retro_hunt, retro_hunt_run, retro_hit, v_retro_hunt_summary
Time machine	Every indicator keeps when it was hostile and when we learned about it, so results can be split into "known then" and "learned later"	ioc.valid_from/valid_to, recorded_from/recorded_to, trg_ioc_no_overlap, trg_ioc_no_delete, CQ-05
Reports	Attack summary by type for any date range	sp_attack_report, sp_asset_risk_report
Roles and security	Admins work with the data but cannot change the schema; analysts work only on their own incidents, through views and procedures; viewers only see reports. Accounts lock after three wrong passwords.	roles sx_admin, sx_analyst, sx_viewer; v_my_incidents, v_ioc; sp_assign_incident, sp_report_ioc

Module	What it does	Database objects behind it
Audit trail	Records who changed which incident, when, and the old and new status, priority and owner. Only its triggers write it; no role can change, forge or delete a row.	incident_audit, trg_incident_audit_update/delete, trg_audit_no_update/delete
Backup and recovery	Nightly dump plus the binary log restore the database to just before a mistake.	mysqldump, binary log, mysqlbinlog (section 15)

8. ER Diagram

The ER diagram on the next page is drawn in Chen notation and is unchanged from the revised Review 1. Rectangles are entities, diamonds are relationships and ovals are attributes, with the primary key underlined. MITIGATION is a weak entity (double rectangle) whose partial key step_no has a dashed underline, and its identifying relationship HAS_STEP is a double diamond. detected_at and patched_at belong to the EXPOSED_TO relationship itself. A double line means total participation, for example every attack must target an asset. There are 12 entities and 17 relationships: 12 one-to-many (one identifying), 4 many-to-many and 1 one-to-one.

Entity	What it represents
ASSET	A server or device we are protecting
VULNERABILITY	A published weakness (CVE) with its CVSS score
NETWORK_LOG	One raw line from the firewall or IDS
ATTACK_EVENT	One detected attack against an asset
ATTACK_TYPE	A kind of attack, mapped to a MITRE ATT&CK technique
MITIGATION (weak)	One response step for an attack type
THREAT_ACTOR	An attacker group
MALWARE	A piece of malicious software
IOC	An indicator of compromise: a bad IP, domain, file hash or URL
RETRO_HUNT_RUN	One automatic search of old logs for a newly added bad IP
INCIDENT	An investigation that groups related attacks
ANALYST	A person who uses the system

Relationship	Between	Type	Linked by
TARGETS	ASSET, ATTACK_EVENT	1 : N	attack_event.asset_id (NOT NULL)
CLASSIFIED_AS	ATTACK_TYPE, ATTACK_EVENT	1 : N	attack_event.type_id (NOT NULL)
ATTRIBUTED_TO	THREAT_ACTOR, ATTACK_EVENT	1 : N	attack_event.actor_id
DELIVERS	MALWARE, ATTACK_EVENT	1 : N	attack_event.malware_id
EXPLOITS	VULNERABILITY, ATTACK_EVENT	1 : N	attack_event.cve_id
EXPOSED_TO	ASSET, VULNERABILITY	M : N	link table asset_vulnerability
DESTINED_FOR	ASSET, NETWORK_LOG	1 : N	network_log.dst_asset_id (NOT NULL)
HAS_STEP	ATTACK_TYPE, MITIGATION	1 : N (identifying)	mitigation key (type_id, step_no)
DEVELOPS	THREAT_ACTOR, MALWARE	1 : N	malware.actor_id
LINKED_TO	THREAT_ACTOR, IOC	1 : N	ioc.actor_id
INDICATES	MALWARE, IOC	1 : N	ioc.malware_id
MATCHES	ATTACK_EVENT, IOC	M : N	link table event_ioc

Relationship	Between	Type	Linked by
TRIGGERS	IOC, RETRO_HUNT_RUN	1 : N	retro_hunt_run.ioc_version_id (NOT NULL)
HITS	RETRO_HUNT_RUN, NETWORK_LOG	M : N	link table retro_hit
OPENS	RETRO_HUNT_RUN, INCIDENT	1 : 1	retro_hunt_run.incident_id (UNIQUE)
GROUPS	INCIDENT, ATTACK_EVENT	M : N	link table incident_event
ASSIGNED_TO	ANALYST, INCIDENT	1 : N	incident.assigned_to

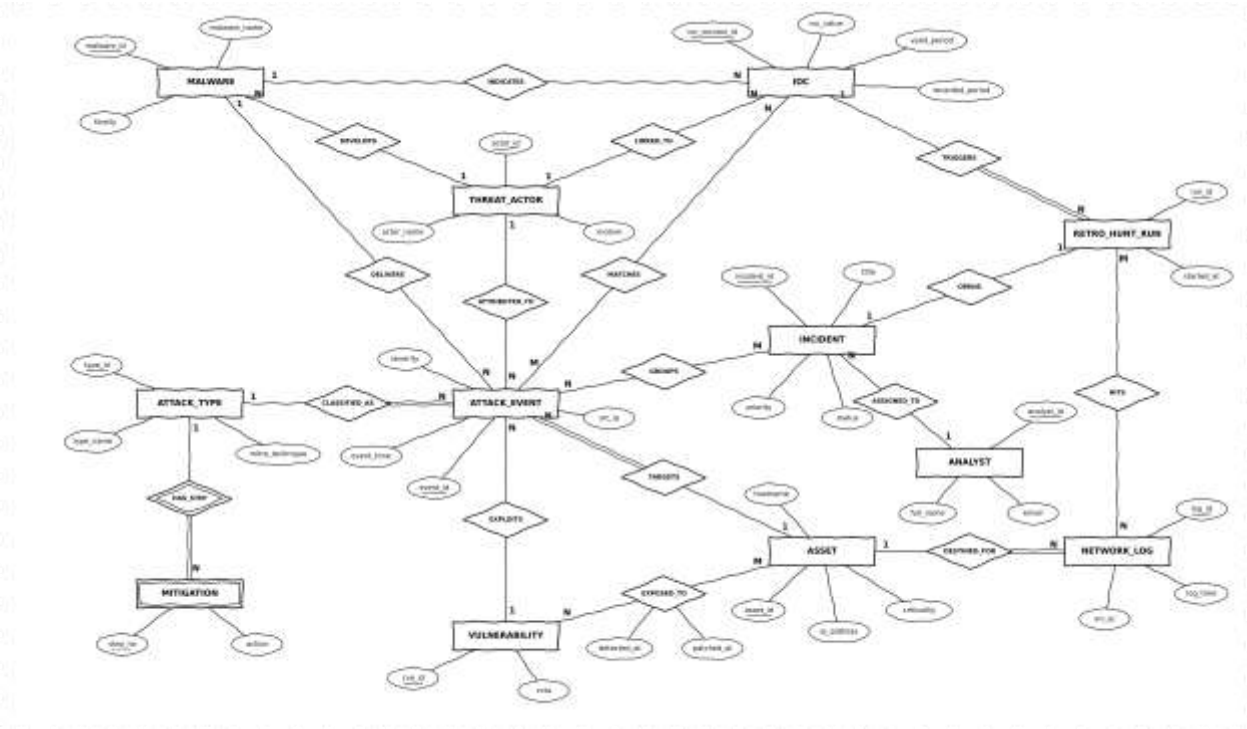


Fig. 2 ER diagram of SentinelX (Chen notation)

The relational schema below is drawn by a script (tools/draw_relational_schema.py) straight from sentinelx_schema.sql, so it always matches the tables MySQL actually creates. It replaces the hand-drawn Review 2 version, which still showed the old partition keys (event_time and log_time inside primary keys) that Review 2 had already removed. incident_audit appears as a separate table with no relationship line: it is an implementation table for accountability, not part of the conceptual model, and it deliberately has no foreign key so that its rows survive the deletion of the incident they describe.

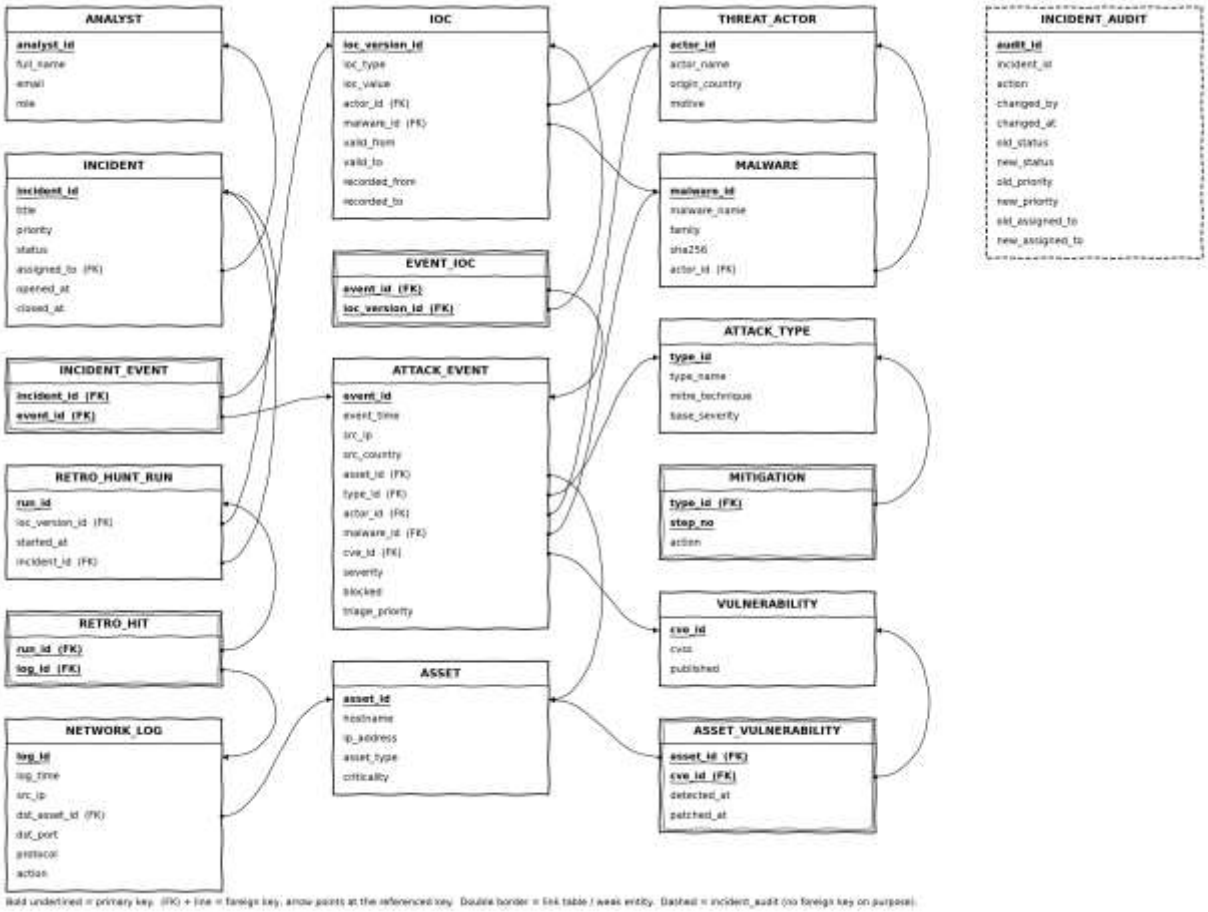
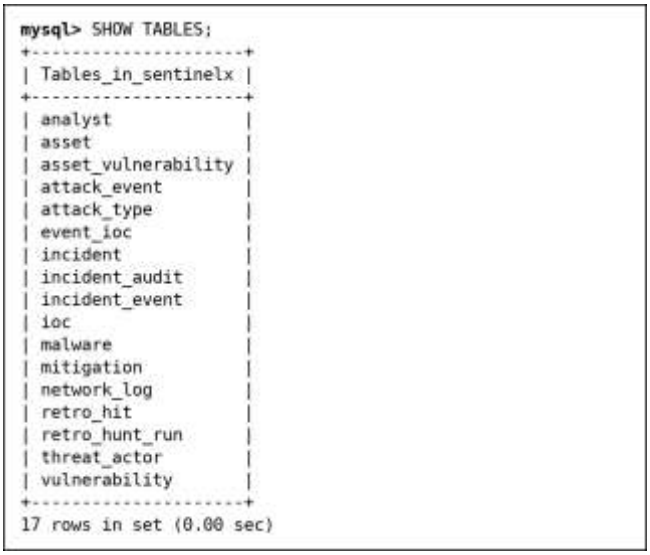


Fig. 3 Relational schema generated from the DDL

9. Tables and Constraints

All 16 tables, and the audit table added in Review 3, are in Third Normal Form: in every table each non-key column depends on the key, the whole key and nothing but the key. The normalisation steps are in section 9.1; the 23 attributes removed after Review 1 are listed in section 15 of the Review 2 report. For each table below we give a short description, the CREATE TABLE statement with every data type and constraint as MySQL ran it, and the normal form. MySQL echoes statements without comments, so the screenshots show the definitions without them.



```
mysql> SHOW TABLES;
+---------------------+
| Tables_in_sentinelx |
+---------------------+
| analyst             |
| asset               |
| asset_vulnerability |
| attack_event        |
| attack_type         |
| event_ioc           |
| incident            |
| incident_audit      |
| incident_event      |
| ioc                 |
| malware             |
| mitigation          |
| network_log         |
| retro_hit           |
| retro_hunt_run      |
| threat_actor        |
| vulnerability       |
+---------------------+
17 rows in set (0.00 sec)
```

Fig. 4 SHOW TABLES after running the 17 CREATE TABLE statements

9.1 Normalisation, and the anomaly it prevents

Step	What we did
Starting point	One flat row per attack with time, source IP, host details, attack name, attacker, malware, CVE and score, and a list of IOCs like "198.51.100.23, evil-1.example".
1NF	Split the IOC list so each row holds one indicator.
2NF	Attack details depend only on the event and indicator details only on the indicator, so they went into ATTACK_EVENT, IOC and the link table EVENT_IOC.
3NF	Host, attacker, malware, CVE, attack type and analyst details each moved to their own table; the two transitive dependencies in change 3 were removed.

The check below runs on the loaded data. The first query confirms a functional dependency (each MITRE technique belongs to one attack type, so the result is empty). Then the attacks are copied into one flat table together with their asset's criticality, as they would be stored without normalisation, and one row is "corrected": web-01 now has two criticalities at once, an update anomaly. In the real schema the criticality is stored once, in asset, so it cannot disagree with itself.

```
mysql> SELECT mitre_technique, COUNT(DISTINCT type_id) AS types
    -> FROM attack_type GROUP BY mitre_technique HAVING COUNT(DISTINCT type_id) > 1;
Empty set (0.00 sec)

mysql> CREATE TEMPORARY TABLE flat_attack AS
    -> SELECT e.event_id, a.hostname, a.criticality FROM attack_event e JOIN asset a ON a.asset_id = e.asset_id;
Query OK, 30 rows affected (0.00 sec)
Records: 30  Duplicates: 0  Warnings: 0

mysql> UPDATE flat_attack SET criticality = 1 WHERE event_id = 1;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT hostname, GROUP_CONCAT(DISTINCT criticality ORDER BY criticality) AS criticality_values
    -> FROM flat_attack WHERE hostname = 'web-01' GROUP BY hostname;
+----------+--------------------+
| hostname | criticality_values |
+----------+--------------------+
| web-01   | 1,5                |
+----------+--------------------+
1 row in set (0.00 sec)

mysql> DROP TEMPORARY TABLE flat_attack;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT hostname, criticality FROM asset WHERE hostname = 'web-01';
+----------+-------------+
| hostname | criticality |
+----------+-------------+
| web-01   |           5 |
+----------+-------------+
1 row in set (0.00 sec)
```

Fig. 5 Functional dependency check, and an update anomaly in a denormalised copy

9.2 The tables

Table: asset

Every server or device we protect. hostname and ip_address are unique too.

```
mysql> CREATE TABLE asset (
    ->   asset_id     INT AUTO_INCREMENT PRIMARY KEY,
    ->   hostname     VARCHAR(63) NOT NULL UNIQUE,
    ->   ip_address   VARCHAR(45) NOT NULL UNIQUE,
    ->   asset_type   ENUM('web','db','mail','dns','workstation','iot','vpn') NOT NULL,
    ->   criticality  TINYINT NOT NULL CHECK (criticality BETWEEN 1 AND 5)
    -> );
Query OK, 0 rows affected (0.04 sec)
```

Fig. 6 CREATE TABLE asset executed in MySQL

Normal form: Every column depends only on asset_id (hostname and ip_address are also candidate keys). 3NF.

Table: vulnerability

Published CVEs. cve_id must look like CVE-2024-3400. Severity is not stored; the view v_vulnerability works it out from cvss.

```
mysql> CREATE TABLE vulnerability (
    ->   cve_id     VARCHAR(16) PRIMARY KEY CHECK (cve_id REGEXP '^CVE-[0-9]{4}-[0-9]{4,7}$'),
    ->   cvss       DECIMAL(3,1) NOT NULL CHECK (cvss BETWEEN 0 AND 10),
    ->   published  DATE NOT NULL
    -> );
Query OK, 0 rows affected (0.05 sec)
```

Fig. 7 CREATE TABLE vulnerability executed in MySQL

Normal form: cvss and published depend only on cve_id. Storing severity would add the transitive dependency cve_id -> cvss -> severity. 3NF.

Table: asset_vulnerability

Link table for EXPOSED_TO (M:N between assets and CVEs), with when the CVE was found and patched.

```
mysql> CREATE TABLE asset_vulnerability (
    ->   asset_id     INT NOT NULL,
    ->   cve_id       VARCHAR(16) NOT NULL,
    ->   detected_at  DATE NOT NULL,
    ->   patched_at   DATE,
    ->   PRIMARY KEY (asset_id, cve_id),
    ->   FOREIGN KEY (asset_id) REFERENCES asset(asset_id) ON DELETE CASCADE,
    ->   FOREIGN KEY (cve_id) REFERENCES vulnerability(cve_id),
    ->   CHECK (patched_at IS NULL OR patched_at >= detected_at)
    -> );
Query OK, 0 rows affected (0.04 sec)
```

Fig. 8 CREATE TABLE asset_vulnerability executed in MySQL

Normal form: detected_at and patched_at depend on the whole key (asset_id, cve_id). 3NF.

Table: network_log

Raw log lines from the firewall and IDS. The index on (src_ip, log_time) serves the retro-hunt (section 14).

```
mysql> CREATE TABLE network_log (
    ->   log_id        BIGINT AUTO_INCREMENT PRIMARY KEY,
    ->   log_time      DATETIME NOT NULL,
    ->   src_ip        VARCHAR(45) NOT NULL,
    ->   dst_asset_id  INT NOT NULL,
    ->   dst_port      SMALLINT UNSIGNED NOT NULL,
    ->   protocol      ENUM('TCP','UDP','ICMP') NOT NULL,
    ->   action        ENUM('allowed','blocked') NOT NULL,
    ->   FOREIGN KEY (dst_asset_id) REFERENCES asset(asset_id),
    ->   INDEX idx_src_time (src_ip, log_time)
    -> );
Query OK, 0 rows affected (0.05 sec)
```

Fig. 9 CREATE TABLE network_log executed in MySQL

Normal form: Everything depends on log_id. 3NF.

Table: attack_type

Kinds of attack with their MITRE ATT&CK technique ID.

```
mysql> CREATE TABLE attack_type (
    ->   type_id          INT AUTO_INCREMENT PRIMARY KEY,
    ->   type_name        VARCHAR(40) NOT NULL UNIQUE,
    ->   mitre_technique  VARCHAR(9) NOT NULL UNIQUE CHECK (mitre_technique REGEXP '^T[0-9]{4}([.][0-9]{3})?$'),
    ->   base_severity    TINYINT NOT NULL CHECK (base_severity BETWEEN 1 AND 10)
    -> );
Query OK, 0 rows affected (0.04 sec)
```

Fig. 10 CREATE TABLE attack_type executed in MySQL

Normal form: Everything depends on type_id; type_name and mitre_technique are candidate keys. 3NF.

Table: mitigation

Weak entity: numbered response steps for each attack type, deleted together with the type.

```
mysql> CREATE TABLE mitigation (
    ->   type_id  INT NOT NULL,
    ->   step_no  TINYINT NOT NULL CHECK (step_no > 0),
    ->   action   VARCHAR(100) NOT NULL,
    ->   PRIMARY KEY (type_id, step_no),
    ->   FOREIGN KEY (type_id) REFERENCES attack_type(type_id) ON DELETE CASCADE
    -> );
Query OK, 0 rows affected (0.03 sec)
```

Fig. 11 CREATE TABLE mitigation executed in MySQL

Normal form: action depends on the full key (type_id, step_no). 3NF.

Table: threat_actor

Attacker groups with country and motive. The names are made up.

```
mysql> CREATE TABLE threat_actor (
    ->   actor_id        INT AUTO_INCREMENT PRIMARY KEY,
    ->   actor_name      VARCHAR(50) NOT NULL UNIQUE,
    ->   origin_country  CHAR(2) CHECK (origin_country REGEXP '^[A-Z]{2}$'),
    ->   motive          ENUM('financial','espionage','hacktivism','sabotage','unknown') NOT NULL
    -> );
Query OK, 0 rows affected (0.03 sec)
```

Fig. 12 CREATE TABLE threat_actor executed in MySQL

Normal form: Everything depends on actor_id. 3NF.

Table: malware

Malware samples. actor_id becomes NULL if the group is deleted.

```
mysql> CREATE TABLE malware (
    ->   malware_id    INT AUTO_INCREMENT PRIMARY KEY,
    ->   malware_name  VARCHAR(50) NOT NULL UNIQUE,
    ->   family        VARCHAR(30) NOT NULL,
    ->   sha256        CHAR(64) NOT NULL UNIQUE CHECK (sha256 REGEXP '^[0-9a-f]{64}$'),
    ->   actor_id      INT,
    ->   FOREIGN KEY (actor_id) REFERENCES threat_actor(actor_id) ON DELETE SET NULL
    -> );
Query OK, 0 rows affected (0.04 sec)
```

Fig. 13 CREATE TABLE malware executed in MySQL

Normal form: Everything depends on malware_id. category was removed because family decides it. 3NF.

Table: attack_event

One row per detected attack. Asset and type are required; attacker, malware and CVE are optional. triage_priority is set by an analyst and stays NULL until the attack has been triaged.

```
mysql> CREATE TABLE attack_event (
    ->   event_id            BIGINT AUTO_INCREMENT PRIMARY KEY,
    ->   event_time          DATETIME NOT NULL,
    ->   src_ip              VARCHAR(45) NOT NULL,
    ->   src_country         CHAR(2) NOT NULL CHECK (src_country REGEXP '^[A-Z]{2}$'),
    ->   asset_id            INT NOT NULL,
    ->   type_id             INT NOT NULL,
    ->   actor_id            INT,
    ->   malware_id          INT,
    ->   cve_id              VARCHAR(16),
    ->   severity            TINYINT NOT NULL CHECK (severity BETWEEN 1 AND 10),
    ->   blocked             BOOLEAN NOT NULL,
    ->   triage_priority     ENUM('LOW','MEDIUM','HIGH','CRITICAL'),
    ->   FOREIGN KEY (asset_id) REFERENCES asset(asset_id),
    ->   FOREIGN KEY (type_id) REFERENCES attack_type(type_id),
    ->   FOREIGN KEY (actor_id) REFERENCES threat_actor(actor_id),
    ->   FOREIGN KEY (malware_id) REFERENCES malware(malware_id),
    ->   FOREIGN KEY (cve_id) REFERENCES vulnerability(cve_id),
    ->   INDEX idx_time (event_time),
    ->   INDEX idx_src (src_ip)
    -> );
Query OK, 0 rows affected (0.07 sec)
```

Fig. 14 CREATE TABLE attack_event executed in MySQL

Normal form: Everything depends on event_id. triage_priority is an analyst decision, not a function of severity (events 5 and 6 both have severity 9 but are CRITICAL and HIGH), so there is no transitive dependency. src_country stays because an IP's location changes over time. actor_id records who carried out this attack, which is a different fact from who develops the malware (malware.actor_id), and it is needed when no malware is known. 3NF.

Table: analyst

People who use the system. email is unique and checked to look like an email address.

```
mysql> CREATE TABLE analyst (
    ->   analyst_id  INT AUTO_INCREMENT PRIMARY KEY,
    ->   full_name   VARCHAR(50) NOT NULL,
    ->   email       VARCHAR(50) NOT NULL UNIQUE CHECK (email REGEXP '^[^@ ]+@[^@ ]+[.][A-Za-z]{2,}$'),
    ->   role        ENUM('admin','analyst','viewer') NOT NULL
    -> );
Query OK, 0 rows affected (0.03 sec)
```

Fig. 15 CREATE TABLE analyst executed in MySQL

Normal form: Everything depends on analyst_id; email is also a candidate key. 3NF.

Table: incident

An investigation. closed_at is filled in only when the status is CLOSED.

```
mysql> CREATE TABLE incident (
    ->   incident_id  INT AUTO_INCREMENT PRIMARY KEY,
    ->   title        VARCHAR(100) NOT NULL,
    ->   priority     ENUM('LOW','MEDIUM','HIGH','CRITICAL') NOT NULL,
    ->   status       ENUM('OPEN','INVESTIGATING','CONTAINED','CLOSED') NOT NULL DEFAULT 'OPEN',
    ->   assigned_to  INT,
    ->   opened_at    DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    ->   closed_at    DATETIME,
    ->   FOREIGN KEY (assigned_to) REFERENCES analyst(analyst_id),
    ->   CHECK ((status = 'CLOSED') = (closed_at IS NOT NULL)),
    ->   CHECK (closed_at IS NULL OR closed_at >= opened_at)
    -> );
Query OK, 0 rows affected (0.03 sec)
```

Fig. 16 CREATE TABLE incident executed in MySQL

Normal form: Everything depends on incident_id. 3NF.

Table: incident_event

Link table for GROUPS (M:N between incidents and attacks).

```
mysql> CREATE TABLE incident_event (
    ->   incident_id  INT NOT NULL,
    ->   event_id     BIGINT NOT NULL,
    ->   PRIMARY KEY (incident_id, event_id),
    ->   FOREIGN KEY (incident_id) REFERENCES incident(incident_id) ON DELETE CASCADE,
    ->   FOREIGN KEY (event_id) REFERENCES attack_event(event_id) ON DELETE CASCADE
    -> );
Query OK, 0 rows affected (0.03 sec)
```

Fig. 17 CREATE TABLE incident_event executed in MySQL

Normal form: All-key table. 3NF.

Table: ioc

Indicators of compromise. valid_from/valid_to say when it was hostile and recorded_from/recorded_to say when we knew. Triggers keep the history append-only.

```
mysql> CREATE TABLE ioc (
    ->   ioc_version_id  BIGINT AUTO_INCREMENT PRIMARY KEY,
    ->   ioc_type        ENUM('ip','domain','hash','url') NOT NULL,
    ->   ioc_value       VARCHAR(255) NOT NULL,
    ->   actor_id        INT,
    ->   malware_id      INT,
    ->   valid_from      DATETIME NOT NULL,
    ->   valid_to        DATETIME,
    ->   recorded_from   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    ->   recorded_to     DATETIME,
    ->   FOREIGN KEY (actor_id) REFERENCES threat_actor(actor_id),
    ->   FOREIGN KEY (malware_id) REFERENCES malware(malware_id),
    ->   CHECK (ioc_type <> 'ip' OR ioc_value REGEXP '^[0-9]{1,3}([.][0-9]{1,3}){3}$'),
    ->   CHECK (valid_to IS NULL OR valid_to > valid_from),
    ->   CHECK (recorded_to IS NULL OR recorded_to > recorded_from),
    ->   INDEX idx_value (ioc_value)
    -> );
Query OK, 0 rows affected (0.04 sec)
```

Fig. 18 CREATE TABLE ioc executed in MySQL

Normal form: Everything depends on ioc_version_id. 3NF.

Table: event_ioc

Link table for MATCHES (M:N between attacks and indicators).

```
mysql> CREATE TABLE event_ioc (
    ->   event_id        BIGINT NOT NULL,
    ->   ioc_version_id  BIGINT NOT NULL,
    ->   PRIMARY KEY (event_id, ioc_version_id),
    ->   FOREIGN KEY (event_id) REFERENCES attack_event(event_id) ON DELETE CASCADE,
    ->   FOREIGN KEY (ioc_version_id) REFERENCES ioc(ioc_version_id)
    -> );
Query OK, 0 rows affected (0.04 sec)
```

Fig. 19 CREATE TABLE event_ioc executed in MySQL

Normal form: All-key table. 3NF.

Table: retro_hunt_run

One row per automatic search of old logs. incident_id is UNIQUE (1:1 OPENS).

```
mysql> CREATE TABLE retro_hunt_run (
    ->   run_id          INT AUTO_INCREMENT PRIMARY KEY,
    ->   ioc_version_id  BIGINT NOT NULL,
    ->   started_at      DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    ->   incident_id     INT UNIQUE,
    ->   FOREIGN KEY (ioc_version_id) REFERENCES ioc(ioc_version_id),
    ->   FOREIGN KEY (incident_id) REFERENCES incident(incident_id)
    -> );
Query OK, 0 rows affected (0.04 sec)
```

Fig. 20 CREATE TABLE retro_hunt_run executed in MySQL

Normal form: Everything depends on run_id. started_at is when the search actually ran (SYSDATE() in the trigger), not a copy of the indicator's recorded_from, and the counts come from v_retro_hunt_summary instead of being stored. 3NF.

Table: retro_hit

Link table for HITS: the old log lines a retro-hunt found.

```
mysql> CREATE TABLE retro_hit (
    ->   run_id  INT NOT NULL,
    ->   log_id  BIGINT NOT NULL,
    ->   PRIMARY KEY (run_id, log_id),
    ->   FOREIGN KEY (run_id) REFERENCES retro_hunt_run(run_id) ON DELETE CASCADE,
    ->   FOREIGN KEY (log_id) REFERENCES network_log(log_id)
    -> );
Query OK, 0 rows affected (0.04 sec)
```

Fig. 21 CREATE TABLE retro_hit executed in MySQL

Normal form: All-key table. 3NF.

Table: incident_audit

Added in Review 3. One row per change to an incident, written by triggers: who made it (USER()), when, and the old and new status, priority and owner. It has no foreign key on purpose, so the trail outlives a deleted incident. Two more triggers make it append-only, and no role has INSERT, UPDATE or DELETE on it.

```
mysql> CREATE TABLE incident_audit (
    ->   audit_id         BIGINT AUTO_INCREMENT PRIMARY KEY,
    ->   incident_id      INT NOT NULL,
    ->   action           ENUM('UPDATE','DELETE') NOT NULL,
    ->   changed_by       VARCHAR(288) NOT NULL,
    ->   changed_at       DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
    ->   old_status       ENUM('OPEN','INVESTIGATING','CONTAINED','CLOSED'),
    ->   new_status       ENUM('OPEN','INVESTIGATING','CONTAINED','CLOSED'),
    ->   old_priority     ENUM('LOW','MEDIUM','HIGH','CRITICAL'),
    ->   new_priority     ENUM('LOW','MEDIUM','HIGH','CRITICAL'),
    ->   old_assigned_to  INT,
    ->   new_assigned_to  INT,
    ->   INDEX idx_incident (incident_id, changed_at)
    -> );
Query OK, 0 rows affected (0.04 sec)
```

Fig. 22 CREATE TABLE incident_audit executed in MySQL

Normal form: Everything depends on audit_id; the old and new values describe one change. 3NF.

10. Data Population (INSERT, UPDATE, DELETE)

The data is a hand-made test set, small enough to check every result by hand, plus 50,000 generated background log lines so the index comparison in section 14 is meaningful. Attacker IPs come from the address ranges reserved for documentation (192.0.2.0/24, 198.51.100.0/24, 203.0.113.0/24), so no real address is marked as bad. The CVE IDs are real. The full script is 02_dml.sql; one INSERT per table type is shown here.

INSERT

```
mysql> INSERT INTO asset (asset_id, hostname, ip_address, asset_type, criticality) VALUES
    ->   (1, 'web-01', '10.0.1.10', 'web', 5),
    ->   (2, 'web-02', '10.0.1.11', 'web', 4),
    ->   (3, 'db-01', '10.0.2.20', 'db', 5),
    ->   (4, 'mail-01', '10.0.3.30', 'mail', 3),
    ->   (5, 'dns-01', '10.0.4.40', 'dns', 4),
    ->   (6, 'ws-hr-01', '10.0.5.51', 'workstation', 2),
    ->   (7, 'ws-fin-01', '10.0.5.52', 'workstation', 3),
    ->   (8, 'cam-01', '10.0.6.60', 'iot', 1),
    ->   (9, 'vpn-01', '10.0.7.70', 'vpn', 4),
    ->   (10, 'ws-dev-01', '10.0.5.53', 'workstation', 2);
Query OK, 10 rows affected (0.01 sec)
Records: 10  Duplicates: 0  Warnings: 0
```

Fig. 23 INSERT of the 10 assets

The other 15 tables are filled the same way, with explicit keys so the links between tables are predictable. The background log lines are generated with a recursive CTE in one statement:

```
mysql> SET SESSION cte_max_recursion_depth = 60000;
Query OK, 0 rows affected (0.00 sec)

mysql> INSERT INTO network_log (log_time, src_ip, dst_asset_id, dst_port, protocol, action)
    -> WITH RECURSIVE seq (n) AS (SELECT 1 UNION ALL SELECT n + 1 FROM seq WHERE n < 50000)
    -> SELECT TIMESTAMP('2026-09-01') + INTERVAL n * 100 SECOND,
    ->        CONCAT('172.16.', n MOD 40, '.', 1 + n MOD 250),
    ->        1 + n MOD 9,
    ->        ELT(1 + n MOD 5, 443, 80, 53, 25, 22),
    ->        IF(n MOD 5 = 2, 'UDP', 'TCP'),
    ->        IF(n MOD 47 = 0, 'blocked', 'allowed')
    -> FROM seq;
Query OK, 50000 rows affected (0.56 sec)
Records: 50000  Duplicates: 0  Warnings: 0
```

Fig. 24 Generating 50,000 background log lines with a recursive CTE

UPDATE

Attacks 25 and 26 have not been triaged yet. An analyst sets their triage priority: attack 25 has severity 7 but was blocked, so it becomes MEDIUM; attack 26 (severity 4, blocked) becomes LOW.

```
mysql> SELECT event_id, severity, blocked, triage_priority FROM attack_event WHERE event_id IN (25, 26);
+----------+----------+---------+-----------------+
| event_id | severity | blocked | triage_priority |
+----------+----------+---------+-----------------+
|       25 |        7 |       1 | NULL            |
|       26 |        4 |       1 | NULL            |
+----------+----------+---------+-----------------+
2 rows in set (0.00 sec)

mysql> UPDATE attack_event
    ->    SET triage_priority = CASE event_id WHEN 25 THEN 'MEDIUM'
    ->                                        WHEN 26 THEN 'LOW' END
    ->  WHERE event_id IN (25, 26);
Query OK, 2 rows affected (0.01 sec)
Rows matched: 2  Changed: 2  Warnings: 0

mysql> SELECT event_id, severity, blocked, triage_priority FROM attack_event WHERE event_id IN (25, 26);
+----------+----------+---------+-----------------+
| event_id | severity | blocked | triage_priority |
+----------+----------+---------+-----------------+
|       25 |        7 |       1 | MEDIUM          |
|       26 |        4 |       1 | LOW             |
+----------+----------+---------+-----------------+
2 rows in set (0.00 sec)
```

Fig. 25 UPDATE of triage_priority, before and after

DELETE

```
mysql> INSERT INTO asset (hostname, ip_address, asset_type, criticality) VALUES ('tmp-scan-01', '10.0.9.99', 'workstation', 1);
Query OK, 1 row affected (0.01 sec)

mysql> SELECT asset_id, hostname FROM asset WHERE hostname = 'tmp-scan-01';
+----------+-------------+
| asset_id | hostname    |
+----------+-------------+
|       11 | tmp-scan-01 |
+----------+-------------+
1 row in set (0.00 sec)

mysql> DELETE FROM asset WHERE hostname = 'tmp-scan-01';
Query OK, 1 row affected (0.00 sec)

mysql> SELECT COUNT(*) AS assets_left FROM asset;
+-------------+
| assets_left |
+-------------+
|          10 |
+-------------+
1 row in set (0.00 sec)
```

Fig. 26 INSERT and DELETE of a temporary asset

Constraints in action

Each statement below breaks one rule on purpose, and MySQL rejects it: a CHECK (criticality 9), a UNIQUE (duplicate hostname), a FOREIGN KEY on insert (type 99 does not exist), a FOREIGN KEY on delete (web-01 still has attacks), and the table-level CHECK that only closed incidents have a closed_at.

```
mysql> INSERT INTO asset (hostname, ip_address, asset_type, criticality) VALUES ('web-03', '10.0.1.12', 'web', 9);
ERROR 3819 (HY000): Check constraint 'asset_chk_1' is violated.

mysql> INSERT INTO asset (hostname, ip_address, asset_type, criticality) VALUES ('web-01', '10.0.1.99', 'web', 3);
ERROR 1062 (23000): Duplicate entry 'web-01' for key 'asset.hostname'

mysql> INSERT INTO attack_event (event_time, src_ip, src_country, asset_id, type_id, severity, blocked)
    -> VALUES ('2026-10-27 10:00:00', '203.0.113.5', 'US', 1, 99, 5, 1);
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails
  (`sentinelx`.`attack_event`, CONSTRAINT `attack_event_ibfk_2` FOREIGN KEY (`type_id`) REFERENCES
  `attack_type` (`type_id`))

mysql> DELETE FROM asset WHERE asset_id = 1;
ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constraint fails
  (`sentinelx`.`attack_event`, CONSTRAINT `attack_event_ibfk_1` FOREIGN KEY (`asset_id`) REFERENCES `asset`
  (`asset_id`))

mysql> INSERT INTO incident (title, priority, status, closed_at) VALUES ('Test', 'LOW', 'OPEN', '2026-10-27 10:00:00');
ERROR 3819 (HY000): Check constraint 'incident_chk_1' is violated.
```

Fig. 27 Five rejected statements, one per constraint

Row counts

```
mysql> SELECT 'asset' AS table_name, COUNT(*) AS row_count FROM asset
    -> UNION ALL SELECT 'vulnerability', COUNT(*) FROM vulnerability
    -> UNION ALL SELECT 'asset_vulnerability', COUNT(*) FROM asset_vulnerability
    -> UNION ALL SELECT 'network_log', COUNT(*) FROM network_log
    -> UNION ALL SELECT 'attack_type', COUNT(*) FROM attack_type
    -> UNION ALL SELECT 'mitigation', COUNT(*) FROM mitigation
    -> UNION ALL SELECT 'threat_actor', COUNT(*) FROM threat_actor
    -> UNION ALL SELECT 'malware', COUNT(*) FROM malware
    -> UNION ALL SELECT 'attack_event', COUNT(*) FROM attack_event
    -> UNION ALL SELECT 'analyst', COUNT(*) FROM analyst
    -> UNION ALL SELECT 'incident', COUNT(*) FROM incident
    -> UNION ALL SELECT 'incident_event', COUNT(*) FROM incident_event
    -> UNION ALL SELECT 'ioc', COUNT(*) FROM ioc
    -> UNION ALL SELECT 'event_ioc', COUNT(*) FROM event_ioc
    -> UNION ALL SELECT 'retro_hunt_run', COUNT(*) FROM retro_hunt_run
    -> UNION ALL SELECT 'retro_hit', COUNT(*) FROM retro_hit
    -> UNION ALL SELECT 'incident_audit', COUNT(*) FROM incident_audit;
+---------------------+-----------+
| table_name          | row_count |
+---------------------+-----------+
| asset               |        10 |
| vulnerability       |         9 |
| asset_vulnerability |         9 |
| network_log         |     50021 |
| attack_type         |         8 |
| mitigation          |        18 |
| threat_actor        |         6 |
| malware             |         8 |
| attack_event        |        30 |
| analyst             |         6 |
| incident            |         7 |
| incident_event      |        18 |
| ioc                 |        12 |
| event_ioc           |        27 |
| retro_hunt_run      |         2 |
| retro_hit           |         8 |
| incident_audit      |         0 |
+---------------------+-----------+
17 rows in set (0.01 sec)
```

Fig. 28 Row count of every table after loading (incident_audit is still empty)

11. SQL Implementation

Every query below was run on the loaded database, and each screenshot shows the statement together with its real output. The table shows how the 41 queries cover what the review requires.

Category	Required coverage	Min	Done	Query IDs
A. Basic queries	SELECT, WHERE, ORDER BY, DISTINCT, BETWEEN, IN, LIKE, IS NULL	6	7	A1, A2, A3, A4, A5, A6, A7
B. Aggregate queries	COUNT, SUM, AVG, MIN, MAX, GROUP BY, HAVING (WHERE vs HAVING in B5)	5	6	B1, B2, B3, B4, B5, B6
C. Joins	INNER, LEFT, RIGHT, FULL OUTER (emulated with UNION), SELF, CROSS	6	6	C1, C2, C3, C4, C5, C6
D. Subqueries	Scalar, single-row, multi-row (IN, ANY, ALL), correlated, EXISTS, NOT EXISTS, NOT IN with the NULL pitfall	6	9	D1, D2, D3, D4, D5, D6, D7, D8, D9
E. Set operations	UNION, UNION ALL, INTERSECT, EXCEPT	4	4	E1, E2, E3, E4
F. Advanced functions	CASE, COALESCE, NULLIF, date/time, string and numeric functions	6	6	F1, F2, F3, F4, F5, F6
G. Window functions	RANK, DENSE_RANK, ROW_NUMBER, running total, LAG, LEAD	2	3	G1, G2, G3

11.1 A. Basic queries

A1. Critical assets (criticality 4 or 5), most critical first

```
mysql> SELECT hostname, ip_address, asset_type, criticality
    -> FROM asset
    -> WHERE criticality >= 4
    -> ORDER BY criticality DESC, hostname;
+----------+------------+------------+-------------+
| hostname | ip_address | asset_type | criticality |
+----------+------------+------------+-------------+
| db-01    | 10.0.2.20  | db         |           5 |
| web-01   | 10.0.1.10  | web        |           5 |
| dns-01   | 10.0.4.40  | dns        |           4 |
| vpn-01   | 10.0.7.70  | vpn        |           4 |
| web-02   | 10.0.1.11  | web        |           4 |
+----------+------------+------------+-------------+
5 rows in set (0.00 sec)
```

Fig. 29 A1: Critical assets (criticality 4 or 5), most critical first

A2. Countries that attacks came from (DISTINCT)

```
mysql> SELECT DISTINCT src_country
    -> FROM attack_event
    -> ORDER BY src_country;
+-------------+
| src_country |
+-------------+
| BR          |
| CN          |
| IN          |
| KP          |
| NL          |
| RU          |
| US          |
+-------------+
7 rows in set (0.00 sec)
```

Fig. 30 A2: Countries that attacks came from (DISTINCT)

A3. Attacks between 10 and 14 October (BETWEEN)

```
mysql> SELECT event_id, event_time, src_ip, severity
    -> FROM attack_event
    -> WHERE event_time BETWEEN '2026-10-10 00:00:00' AND '2026-10-14 23:59:59'
    -> ORDER BY event_time;
+----------+---------------------+---------------+----------+
| event_id | event_time          | src_ip        | severity |
+----------+---------------------+---------------+----------+
|       15 | 2026-10-10 08:15:00 | 203.0.113.11  |       10 |
|       16 | 2026-10-10 08:40:00 | 203.0.113.11  |        9 |
|       17 | 2026-10-11 12:00:00 | 198.51.100.23 |        5 |
|       18 | 2026-10-12 19:30:00 | 203.0.113.99  |        4 |
|       19 | 2026-10-13 07:05:00 | 198.51.100.88 |        6 |
|       20 | 2026-10-14 23:50:00 | 192.0.2.50    |        6 |
+----------+---------------------+---------------+----------+
6 rows in set (0.00 sec)
```

Fig. 31 A3: Attacks between 10 and 14 October (BETWEEN)

A4. Internet-facing servers: web, db and vpn assets (IN)

```
mysql> SELECT hostname, asset_type, criticality
    -> FROM asset
    -> WHERE asset_type IN ('web', 'db', 'vpn')
    -> ORDER BY asset_type, hostname;
+----------+------------+-------------+
| hostname | asset_type | criticality |
+----------+------------+-------------+
| web-01   | web        |           5 |
| web-02   | web        |           4 |
| db-01    | db         |           5 |
| vpn-01   | vpn        |           4 |
+----------+------------+-------------+
4 rows in set (0.00 sec)
```

Fig. 32 A4: Internet-facing servers: web, db and vpn assets (IN)

A5. CVEs published in 2023 (LIKE)

```
mysql> SELECT cve_id, cvss, published
    -> FROM vulnerability
    -> WHERE cve_id LIKE 'CVE-2023-%'
    -> ORDER BY cvss DESC;
+----------------+------+------------+
| cve_id         | cvss | published  |
+----------------+------+------------+
| CVE-2023-34362 |  9.8 | 2023-06-02 |
| CVE-2023-4966  |  9.4 | 2023-10-10 |
| CVE-2023-44487 |  7.5 | 2023-10-10 |
| CVE-2023-48795 |  5.9 | 2023-12-18 |
+----------------+------+------------+
4 rows in set (0.00 sec)
```

Fig. 33 A5: CVEs published in 2023 (LIKE)

A6. Incidents nobody owns yet (IS NULL)

```
mysql> SELECT incident_id, title, priority, status
    -> FROM incident
    -> WHERE assigned_to IS NULL;
+-------------+-----------------------------------------------+----------+--------+
| incident_id | title                                         | priority | status |
+-------------+-----------------------------------------------+----------+--------+
|           5 | Retro-hunt: 203.0.113.11 hit 2 assets 4 times | CRITICAL | OPEN   |
+-------------+-----------------------------------------------+----------+--------+
1 row in set (0.00 sec)
```

Fig. 34 A6: Incidents nobody owns yet (IS NULL)

A7. Attacks no analyst has triaged yet (IS NULL)

```
mysql> SELECT event_id, event_time, src_ip, severity
    -> FROM attack_event
    -> WHERE triage_priority IS NULL
    -> ORDER BY event_id;
+----------+---------------------+---------------+----------+
| event_id | event_time          | src_ip        | severity |
+----------+---------------------+---------------+----------+
|       27 | 2026-10-22 21:30:00 | 198.51.100.88 |        5 |
|       28 | 2026-10-24 03:15:00 | 203.0.113.11  |        7 |
|       29 | 2026-10-25 11:11:00 | 198.51.100.23 |        9 |
|       30 | 2026-10-26 15:40:00 | 192.0.2.50    |       10 |
+----------+---------------------+---------------+----------+
4 rows in set (0.00 sec)
```

Fig. 35 A7: Attacks no analyst has triaged yet (IS NULL)

11.2 B. Aggregate queries

B1. Number of attacks per attack type (COUNT, GROUP BY)

```
mysql> SELECT t.type_name, COUNT(*) AS attacks
    -> FROM attack_event e
    -> JOIN attack_type t ON t.type_id = e.type_id
    -> GROUP BY t.type_name
    -> ORDER BY attacks DESC, t.type_name;
+--------------------------+---------+
| type_name                | attacks |
+--------------------------+---------+
| Exploit Public-Facing App |       7 |
| Brute Force              |       6 |
| DDoS                     |       5 |
| Phishing                 |       4 |
| Port Scan                |       4 |
| Credential Dumping       |       2 |
| Ransomware               |       2 |
+--------------------------+---------+
7 rows in set (0.00 sec)
```

Fig. 36 B1: Number of attacks per attack type (COUNT, GROUP BY)

B2. Attacks, blocked attacks and average severity per asset (SUM, AVG)

```
mysql> SELECT a.hostname,
    ->        COUNT(*)                  AS attacks,
    ->        SUM(e.blocked)            AS blocked,
    ->        ROUND(AVG(e.severity), 2)  AS avg_severity
    -> FROM attack_event e
    -> JOIN asset a ON a.asset_id = e.asset_id
    -> GROUP BY a.hostname
    -> ORDER BY attacks DESC, a.hostname;
+-----------+---------+---------+--------------+
| hostname  | attacks | blocked | avg_severity |
+-----------+---------+---------+--------------+
| web-01    |       7 |       3 |         6.43 |
| web-02    |       5 |       3 |         7.80 |
| db-01     |       4 |       2 |         9.00 |
| vpn-01    |       4 |       2 |         7.75 |
| mail-01   |       3 |       1 |         5.67 |
| cam-01    |       2 |       1 |         4.00 |
| dns-01    |       2 |       1 |         4.50 |
| ws-fin-01 |       2 |       2 |         5.50 |
| ws-hr-01  |       1 |       1 |         5.00 |
+-----------+---------+---------+--------------+
9 rows in set (0.00 sec)
```

Fig. 37 B2: Attacks, blocked attacks and average severity per asset (SUM, AVG)

B3. First and last attack and worst severity per source country (MIN, MAX)

```
mysql> SELECT src_country,
    ->        MIN(event_time) AS first_seen,
    ->        MAX(event_time) AS last_seen,
    ->        MAX(severity)   AS worst_severity
    -> FROM attack_event
    -> GROUP BY src_country
    -> ORDER BY first_seen;
+-------------+---------------------+---------------------+----------------+
| src_country | first_seen          | last_seen           | worst_severity |
+-------------+---------------------+---------------------+----------------+
| RU          | 2026-10-01 02:14:00 | 2026-10-25 11:11:00 |             10 |
| US          | 2026-10-02 11:05:00 | 2026-10-16 13:13:00 |              3 |
| CN          | 2026-10-03 09:30:00 | 2026-10-24 03:15:00 |             10 |
| KP          | 2026-10-05 01:00:00 | 2026-10-26 15:40:00 |             10 |
| BR          | 2026-10-06 14:20:00 | 2026-10-22 21:30:00 |              7 |
| NL          | 2026-10-09 16:45:00 | 2026-10-20 18:00:00 |              8 |
| IN          | 2026-10-12 19:30:00 | 2026-10-21 09:09:00 |              4 |
+-------------+---------------------+---------------------+----------------+
7 rows in set (0.00 sec)
```

Fig. 38 B3: First and last attack and worst severity per source country (MIN, MAX)

B4. Source IPs with more than 3 attacks (HAVING)

```
mysql> SELECT src_ip, COUNT(*) AS attacks
    -> FROM attack_event
    -> GROUP BY src_ip
    -> HAVING COUNT(*) > 3
    -> ORDER BY attacks DESC, src_ip;
+---------------+---------+
| src_ip        | attacks |
+---------------+---------+
| 192.0.2.50    |       4 |
| 198.51.100.23 |       4 |
| 198.51.100.45 |       4 |
| 198.51.100.88 |       4 |
| 203.0.113.200 |       4 |
+---------------+---------+
5 rows in set (0.00 sec)
```

Fig. 39 B4: Source IPs with more than 3 attacks (HAVING)

B5. WHERE vs HAVING: assets with 2 or more serious attacks

WHERE removes attacks below severity 7 before any grouping happens; HAVING then removes whole groups (assets) that end up with fewer than 2 rows. Moving the severity filter into HAVING would not work, because severity is not a group-level value.

```
mysql> SELECT a.hostname, COUNT(*) AS serious_attacks
    -> FROM attack_event e
    -> JOIN asset a ON a.asset_id = e.asset_id
    -> WHERE e.severity >= 7
    -> GROUP BY a.hostname
    -> HAVING COUNT(*) >= 2
    -> ORDER BY serious_attacks DESC, a.hostname;
+----------+-----------------+
| hostname | serious_attacks |
+----------+-----------------+
| web-02   |               5 |
| db-01    |               4 |
| web-01   |               4 |
| vpn-01   |               3 |
+----------+-----------------+
4 rows in set (0.00 sec)
```

Fig. 40 B5: WHERE vs HAVING: assets with 2 or more serious attacks

B6. CVSS statistics by year of publication

```
mysql> SELECT YEAR(published)       AS year_published,
    ->        COUNT(*)              AS cves,
    ->        ROUND(AVG(cvss), 2)   AS avg_cvss,
    ->        MIN(cvss)             AS min_cvss,
    ->        MAX(cvss)             AS max_cvss
    -> FROM vulnerability
    -> GROUP BY YEAR(published)
    -> ORDER BY year_published;
+----------------+------+----------+----------+----------+
| year_published | cves | avg_cvss | min_cvss | max_cvss |
+----------------+------+----------+----------+----------+
|           2019 |    1 |     9.80 |      9.8 |      9.8 |
|           2021 |    1 |    10.00 |     10.0 |     10.0 |
|           2022 |    1 |     9.80 |      9.8 |      9.8 |
|           2023 |    4 |     8.15 |      5.9 |      9.8 |
|           2024 |    2 |     9.05 |      8.1 |     10.0 |
+----------------+------+----------+----------+----------+
5 rows in set (0.00 sec)
```

Fig. 41 B6: CVSS statistics by year of publication

11.3 C. Joins

C1. INNER JOIN: attacks of severity 9 or more with asset and attack type

```
mysql> SELECT e.event_id, e.event_time, a.hostname, t.type_name, e.severity
    -> FROM attack_event e
    -> INNER JOIN asset a       ON a.asset_id = e.asset_id
    -> INNER JOIN attack_type t ON t.type_id  = e.type_id
    -> WHERE e.severity >= 9
    -> ORDER BY e.event_time;
+----------+---------------------+----------+--------------------------+----------+
| event_id | event_time          | hostname | type_name                | severity |
+----------+---------------------+----------+--------------------------+----------+
|        5 | 2026-10-04 22:10:00 | mail-01  | Exploit Public-Facing App |        9 |
|        6 | 2026-10-05 01:00:00 | db-01    | Exploit Public-Facing App |        9 |
|        7 | 2026-10-05 01:45:00 | db-01    | Credential Dumping        |        9 |
|       11 | 2026-10-07 04:10:00 | web-02   | Ransomware                |       10 |
|       15 | 2026-10-10 08:15:00 | vpn-01   | Exploit Public-Facing App |       10 |
|       16 | 2026-10-10 08:40:00 | vpn-01   | Credential Dumping        |        9 |
|       29 | 2026-10-25 11:11:00 | vpn-01   | Exploit Public-Facing App |        9 |
|       30 | 2026-10-26 15:40:00 | db-01    | Ransomware                |       10 |
+----------+---------------------+----------+--------------------------+----------+
8 rows in set (0.00 sec)
```

Fig. 42 C1: INNER JOIN: attacks of severity 9 or more with asset and attack type

C2. LEFT JOIN: every asset with its number of attacks, including assets never attacked

```
mysql> SELECT a.hostname, COUNT(e.event_id) AS attacks
    -> FROM asset a
    -> LEFT JOIN attack_event e ON e.asset_id = a.asset_id
    -> GROUP BY a.hostname
    -> ORDER BY attacks, a.hostname;
+-----------+---------+
| hostname  | attacks |
+-----------+---------+
| ws-dev-01 |       0 |
| ws-hr-01  |       1 |
| cam-01    |       2 |
| dns-01    |       2 |
| ws-fin-01 |       2 |
| mail-01   |       3 |
| db-01     |       4 |
| vpn-01    |       4 |
| web-02    |       5 |
| web-01    |       7 |
+-----------+---------+
10 rows in set (0.00 sec)
```

Fig. 43 C2: LEFT JOIN: every asset with its number of attacks, including assets never attacked

C3. RIGHT JOIN: every analyst with the incidents they own, including analysts with none

```
mysql> SELECT an.full_name, an.role, i.incident_id, i.status
    -> FROM incident i
    -> RIGHT JOIN analyst an ON an.analyst_id = i.assigned_to
    -> ORDER BY an.analyst_id, i.incident_id;
+-------------+---------+-------------+---------------+
| full_name   | role    | incident_id | status        |
+-------------+---------+-------------+---------------+
| Ananya Rao  | admin   |        NULL | NULL          |
| Rahul Mehta | analyst |           1 | CLOSED        |
| Rahul Mehta | analyst |           3 | CONTAINED     |
| Rahul Mehta | analyst |           7 | INVESTIGATING |
| Priya Nair  | analyst |           2 | CLOSED        |
| Priya Nair  | analyst |           6 | OPEN          |
| Karan Shah  | analyst |           4 | INVESTIGATING |
| Divya Iyer  | viewer  |        NULL | NULL          |
| Arjun Das   | analyst |        NULL | NULL          |
+-------------+---------+-------------+---------------+
9 rows in set (0.00 sec)
```

Fig. 44 C3: RIGHT JOIN: every analyst with the incidents they own, including analysts with none

C4. FULL OUTER JOIN emulation: threat actors and malware, keeping both unmatched sides

MySQL has no FULL OUTER JOIN, so it is emulated as LEFT JOIN UNION RIGHT JOIN. The result keeps Byte Bandits (no malware) and OrphanMiner (no known actor).

```
mysql> SELECT t.actor_name, m.malware_name
    -> FROM threat_actor t LEFT JOIN malware m ON m.actor_id = t.actor_id
    -> UNION
    -> SELECT t.actor_name, m.malware_name
    -> FROM threat_actor t RIGHT JOIN malware m ON m.actor_id = t.actor_id
    -> ORDER BY actor_name, malware_name;
+---------------+---------------+
| actor_name    | malware_name  |
+---------------+---------------+
| NULL          | OrphanMiner   |
| Byte Bandits  | NULL          |
| Crimson Lynx  | LynxLoader    |
| Crimson Lynx  | LynxLocker    |
| Ghost Parrot  | ParrotBot     |
| Iron Kestrel  | KestrelWiper  |
| Quiet Willow  | WillowRAT     |
| Silent Monsoon | MonsoonRAT   |
| Silent Monsoon | MonsoonStealer |
+---------------+---------------+
9 rows in set (0.00 sec)
```

Fig. 45 C4: FULL OUTER JOIN emulation: threat actors and malware, keeping both unmatched sides

C5. SELF JOIN: pairs of malware used by the same threat actor

```
mysql> SELECT t.actor_name, m1.malware_name AS malware_1, m2.malware_name AS malware_2
    -> FROM malware m1
    -> JOIN malware m2     ON m2.actor_id = m1.actor_id
    ->                    AND m2.malware_id > m1.malware_id
    -> JOIN threat_actor t ON t.actor_id = m1.actor_id
    -> ORDER BY t.actor_name, malware_1;
+---------------+------------+---------------+
| actor_name    | malware_1  | malware_2     |
+---------------+------------+---------------+
| Crimson Lynx  | LynxLocker | LynxLoader    |
| Silent Monsoon | MonsoonRAT | MonsoonStealer |
+---------------+------------+---------------+
2 rows in set (0.00 sec)
```

Fig. 46 C5: SELF JOIN: pairs of malware used by the same threat actor

C6. CROSS JOIN: hardening checklist of every criticality-5 asset against every severe attack type

A meaningful CROSS JOIN: every criticality-5 asset is paired with every attack type of base severity 8 or more, to produce a hardening checklist. Zero means the combination has not been seen yet.

```
mysql> SELECT a.hostname, t.type_name,
    ->        (SELECT COUNT(*) FROM attack_event e
    ->          WHERE e.asset_id = a.asset_id AND e.type_id = t.type_id) AS times_seen
    -> FROM asset a
    -> CROSS JOIN attack_type t
    -> WHERE a.criticality = 5 AND t.base_severity >= 8
    -> ORDER BY a.hostname, t.type_name;
+----------+--------------------------+-------------+
| hostname | type_name                | times_seen |
+----------+--------------------------+-------------+
| db-01    | Credential Dumping       |           1 |
| db-01    | Exploit Public-Facing App |           2 |
| db-01    | Ransomware               |           1 |
| web-01   | Credential Dumping       |           0 |
| web-01   | Exploit Public-Facing App |           0 |
| web-01   | Ransomware               |           0 |
+----------+--------------------------+-------------+
6 rows in set (0.00 sec)
```

Fig. 47 C6: CROSS JOIN: hardening checklist of every criticality-5 asset against every severe attack type

11.4 D. Subqueries

D1. Scalar subquery: CVEs scoring above the average CVSS

```
mysql> SELECT cve_id, cvss,
    ->        (SELECT ROUND(AVG(cvss), 2) FROM vulnerability) AS avg_cvss
    -> FROM vulnerability
    -> WHERE cvss > (SELECT AVG(cvss) FROM vulnerability)
    -> ORDER BY cvss DESC, cve_id;
+----------------+------+----------+
| cve_id         | cvss | avg_cvss |
+----------------+------+----------+
| CVE-2021-44228 | 10.0 |     8.92 |
| CVE-2024-3400  | 10.0 |     8.92 |
| CVE-2019-0708  |  9.8 |     8.92 |
| CVE-2022-22965 |  9.8 |     8.92 |
| CVE-2023-34362 |  9.8 |     8.92 |
| CVE-2023-4966  |  9.4 |     8.92 |
+----------------+------+----------+
6 rows in set (0.00 sec)
```

Fig. 48 D1: Scalar subquery: CVEs scoring above the average CVSS

D2. Single-row subquery: the analyst who owns the most recently opened incident

```
mysql> SELECT full_name, email
    -> FROM analyst
    -> WHERE analyst_id = (SELECT assigned_to
    ->                     FROM incident
    ->                     WHERE assigned_to IS NOT NULL
    ->                     ORDER BY opened_at DESC
    ->                     LIMIT 1);
+------------+-------------------------+
| full_name  | email                   |
+------------+-------------------------+
| Priya Nair | priya.nair@sentinelx.in |
+------------+-------------------------+
1 row in set (0.00 sec)
```

Fig. 49 D2: Single-row subquery: the analyst who owns the most recently opened incident

D3. Multi-row subquery with IN: assets that have been hit by ransomware

```
mysql> SELECT hostname, asset_type
    -> FROM asset
    -> WHERE asset_id IN (SELECT e.asset_id
    ->                    FROM attack_event e
    ->                    JOIN attack_type t ON t.type_id = e.type_id
    ->                    WHERE t.type_name = 'Ransomware')
    -> ORDER BY hostname;
+----------+------------+
| hostname | asset_type |
+----------+------------+
| db-01    | db         |
| web-02   | web        |
+----------+------------+
2 rows in set (0.00 sec)
```

Fig. 50 D3: Multi-row subquery with IN: assets that have been hit by ransomware

D4. Multi-row subquery with ALL: CVEs scoring higher than every CVE on web-02 and mail-01

```
mysql> SELECT cve_id, cvss
    -> FROM vulnerability
    -> WHERE cvss > ALL (SELECT v.cvss
    ->                   FROM asset_vulnerability av
    ->                   JOIN vulnerability v ON v.cve_id = av.cve_id
    ->                   WHERE av.asset_id IN (2, 4))
    -> ORDER BY cve_id;
+----------------+------+
| cve_id         | cvss |
+----------------+------+
| CVE-2021-44228 | 10.0 |
| CVE-2024-3400  | 10.0 |
+----------------+------+
2 rows in set (0.00 sec)
```

Fig. 51 D4: Multi-row subquery with ALL: CVEs scoring higher than every CVE on web-02 and mail-01

D5. Multi-row subquery with ANY: CVEs scoring higher than at least one CVE used by Russian sources

```
mysql> SELECT cve_id, cvss
    -> FROM vulnerability
    -> WHERE cvss > ANY (SELECT v.cvss
    ->                   FROM attack_event e
    ->                   JOIN vulnerability v ON v.cve_id = e.cve_id
    ->                   WHERE e.src_country = 'RU')
    -> ORDER BY cvss DESC, cve_id;
+----------------+------+
| cve_id         | cvss |
+----------------+------+
| CVE-2021-44228 | 10.0 |
| CVE-2024-3400  | 10.0 |
| CVE-2019-0708  |  9.8 |
| CVE-2022-22965 |  9.8 |
| CVE-2023-34362 |  9.8 |
| CVE-2023-4966  |  9.4 |
+----------------+------+
6 rows in set (0.00 sec)
```

Fig. 52 D5: Multi-row subquery with ANY: CVEs scoring higher than at least one CVE used by Russian sources

D6. Correlated subquery: the worst attack(s) on each asset

```
mysql> SELECT a.hostname, e.event_id, e.severity
    -> FROM attack_event e
    -> JOIN asset a ON a.asset_id = e.asset_id
    -> WHERE e.severity = (SELECT MAX(e2.severity)
    ->                     FROM attack_event e2
    ->                     WHERE e2.asset_id = e.asset_id)
    -> ORDER BY a.hostname, e.event_id;
+-----------+----------+----------+
| hostname  | event_id | severity |
+-----------+----------+----------+
| cam-01    |       18 |        4 |
| cam-01    |       26 |        4 |
| db-01     |       30 |       10 |
| dns-01    |       28 |        7 |
| mail-01   |        5 |        9 |
| vpn-01    |       15 |       10 |
| web-01    |       14 |        8 |
| web-01    |       24 |        8 |
| web-02    |       11 |       10 |
| ws-fin-01 |       20 |        6 |
| ws-hr-01  |       17 |        5 |
+-----------+----------+----------+
11 rows in set (0.00 sec)
```

Fig. 53 D6: Correlated subquery: the worst attack(s) on each asset

D7. EXISTS: threat actors with at least one attack that was not blocked

```
mysql> SELECT t.actor_name, t.motive
    -> FROM threat_actor t
    -> WHERE EXISTS (SELECT 1 FROM attack_event e
    ->               WHERE e.actor_id = t.actor_id AND e.blocked = 0)
    -> ORDER BY t.actor_name;
+---------------+------------+
| actor_name    | motive     |
+---------------+------------+
| Byte Bandits  | hacktivism |
| Crimson Lynx  | financial  |
| Iron Kestrel  | sabotage   |
| Silent Monsoon | espionage |
+---------------+------------+
4 rows in set (0.00 sec)
```

Fig. 54 D7: EXISTS: threat actors with at least one attack that was not blocked

D8. NOT EXISTS: assets that have never been attacked

```
mysql> SELECT hostname, asset_type, criticality
    -> FROM asset a
    -> WHERE NOT EXISTS (SELECT 1 FROM attack_event e
    ->                   WHERE e.asset_id = a.asset_id);
+-----------+------------+-------------+
| hostname  | asset_type | criticality |
+-----------+------------+-------------+
| ws-dev-01 | workstation |           2 |
+-----------+------------+-------------+
1 row in set (0.00 sec)
```

Fig. 55 D8: NOT EXISTS: assets that have never been attacked

D9. NOT IN and the NULL pitfall: threat actors with no malware

The NULL pitfall: malware.actor_id contains a NULL (OrphanMiner). x NOT IN (..., NULL) is never TRUE, so the first query returns no rows at all. Filtering out NULLs in the subquery (or using NOT EXISTS) gives the correct answer, Byte Bandits.

```
mysql> SELECT actor_name FROM threat_actor
    -> WHERE actor_id NOT IN (SELECT actor_id FROM malware);
Empty set (0.00 sec)

mysql> SELECT actor_name FROM threat_actor
    -> WHERE actor_id NOT IN (SELECT actor_id FROM malware WHERE actor_id IS NOT NULL);
+--------------+
| actor_name   |
+--------------+
| Byte Bandits |
+--------------+
1 row in set (0.00 sec)
```

Fig. 56 D9: NOT IN and the NULL pitfall: threat actors with no malware

11.5 E. Set operations

E1. UNION: every country we know about, as an attack source or as an attacker's origin

```
mysql> SELECT src_country AS country FROM attack_event
    -> UNION
    -> SELECT origin_country FROM threat_actor WHERE origin_country IS NOT NULL
    -> ORDER BY country;
+---------+
| country |
+---------+
| BR      |
| CN      |
| IN      |
| IR      |
| KP      |
| NL      |
| RU      |
| US      |
+---------+
8 rows in set (0.00 sec)
```

Fig. 57 E1: UNION: every country we know about, as an attack source or as an attacker's origin

E2. UNION ALL: full activity timeline of 198.51.100.45 across attacks and raw logs

```
mysql> SELECT event_time AS seen_at, 'attack' AS source, asset_id, severity AS detail
    -> FROM attack_event WHERE src_ip = '198.51.100.45'
    -> UNION ALL
    -> SELECT log_time, 'log', dst_asset_id, dst_port
    -> FROM network_log WHERE src_ip = '198.51.100.45'
    -> ORDER BY seen_at;
+---------------------+--------+----------+--------+
| seen_at             | source | asset_id | detail |
+---------------------+--------+----------+--------+
| 2026-10-03 03:00:00 | log    |        2 |   8080 |
| 2026-10-05 03:10:00 | log    |        2 |   8080 |
| 2026-10-06 23:00:00 | log    |        1 |    443 |
| 2026-10-07 03:30:00 | log    |        2 |   8080 |
| 2026-10-07 03:33:00 | attack |        2 |      8 |
| 2026-10-07 04:10:00 | attack |        2 |     10 |
| 2026-10-18 02:01:00 | log    |        1 |     22 |
| 2026-10-18 02:02:00 | attack |        1 |      7 |
| 2026-10-18 02:03:00 | log    |        1 |     22 |
| 2026-10-18 02:05:00 | attack |        1 |      8 |
+---------------------+--------+----------+--------+
10 rows in set (0.00 sec)
```

Fig. 58 E2: UNION ALL: full activity timeline of 198.51.100.45 across attacks and raw logs

E3. INTERSECT: attack sources that are also known bad IPs

INTERSECT and EXCEPT are native in MySQL 8.0.31 and later; we ran them on 8.0.46.

```
mysql> SELECT src_ip FROM attack_event
    -> INTERSECT
    -> SELECT ioc_value FROM ioc WHERE ioc_type = 'ip'
    -> ORDER BY src_ip;
+---------------+
| src_ip        |
+---------------+
| 192.0.2.50    |
| 192.0.2.77    |
| 198.51.100.23 |
| 198.51.100.45 |
| 198.51.100.88 |
| 203.0.113.10  |
| 203.0.113.11  |
+---------------+
7 rows in set (0.00 sec)
```

Fig. 59 E3: INTERSECT: attack sources that are also known bad IPs

E4. EXCEPT: attack sources that are not on the indicator list yet

```
mysql> SELECT src_ip FROM attack_event
    -> EXCEPT
    -> SELECT ioc_value FROM ioc WHERE ioc_type = 'ip'
    -> ORDER BY src_ip;
+---------------+
| src_ip        |
+---------------+
| 203.0.113.200 |
| 203.0.113.99  |
+---------------+
2 rows in set (0.00 sec)
```

Fig. 60 E4: EXCEPT: attack sources that are not on the indicator list yet

11.6 F. Advanced functions

F1. CASE: severity label from CVSS score

```
mysql> SELECT cve_id, cvss,
    ->        CASE WHEN cvss >= 9 THEN 'CRITICAL'
    ->             WHEN cvss >= 7 THEN 'HIGH'
    ->             WHEN cvss >= 4 THEN 'MEDIUM'
    ->             ELSE 'LOW' END AS severity
    -> FROM vulnerability
    -> ORDER BY cvss DESC, cve_id;
+----------------+------+----------+
| cve_id         | cvss | severity |
+----------------+------+----------+
| CVE-2021-44228 | 10.0 | CRITICAL |
| CVE-2024-3400  | 10.0 | CRITICAL |
| CVE-2019-0708  |  9.8 | CRITICAL |
| CVE-2022-22965 |  9.8 | CRITICAL |
| CVE-2023-34362 |  9.8 | CRITICAL |
| CVE-2023-4966  |  9.4 | CRITICAL |
| CVE-2024-6387  |  8.1 | HIGH     |
| CVE-2023-44487 |  7.5 | HIGH     |
| CVE-2023-48795 |  5.9 | MEDIUM   |
+----------------+------+----------+
9 rows in set (0.00 sec)
```

Fig. 61 F1: CASE: severity label from CVSS score

F2. COALESCE: recent attacks with 'Unattributed' instead of a missing actor

```
mysql> SELECT e.event_id, e.src_ip,
    ->        COALESCE(t.actor_name, 'Unattributed') AS actor,
    ->        COALESCE(m.malware_name, '-')          AS malware
    -> FROM attack_event e
    -> LEFT JOIN threat_actor t ON t.actor_id   = e.actor_id
    -> LEFT JOIN malware m      ON m.malware_id = e.malware_id
    -> WHERE e.event_time >= '2026-10-20'
    -> ORDER BY e.event_id;
+----------+--------------+---------------+---------------+
| event_id | src_ip       | actor         | malware       |
+----------+--------------+---------------+---------------+
|       25 | 192.0.2.77   | Byte Bandits  | -             |
|       26 | 203.0.113.99 | Unattributed  | OrphanMiner   |
|       27 | 198.51.100.88| Ghost Parrot  | ParrotBot     |
|       28 | 203.0.113.11 | Silent Monsoon| MonsoonStealer|
|       29 | 198.51.100.23| Crimson Lynx  | LynxLoader    |
|       30 | 192.0.2.50   | Iron Kestrel  | KestrelWiper  |
+----------+--------------+---------------+---------------+
6 rows in set (0.00 sec)
```

Fig. 62 F2: COALESCE: recent attacks with 'Unattributed' instead of a missing actor

F3. NULLIF: share of blocked attacks per asset without dividing by zero

NULLIF turns a zero count into NULL, so ws-dev-01 (never attacked) shows NULL instead of a division-by-zero.

```
mysql> SELECT a.hostname,
    ->        COUNT(e.event_id) AS attacks,
    ->        ROUND(100 * SUM(e.blocked) / NULLIF(COUNT(e.event_id), 0), 1) AS blocked_pct
    -> FROM asset a
    -> LEFT JOIN attack_event e ON e.asset_id = a.asset_id
    -> GROUP BY a.hostname
    -> ORDER BY blocked_pct DESC, a.hostname;
+-----------+---------+-------------+
| hostname  | attacks | blocked_pct |
+-----------+---------+-------------+
| ws-fin-01 |       2 |       100.0 |
| ws-hr-01  |       1 |       100.0 |
| web-02    |       5 |        60.0 |
| cam-01    |       2 |        50.0 |
| db-01     |       4 |        50.0 |
| dns-01    |       2 |        50.0 |
| vpn-01    |       4 |        50.0 |
| web-01    |       7 |        42.9 |
| mail-01   |       3 |        33.3 |
| ws-dev-01 |       0 |        NULL |
+-----------+---------+-------------+
10 rows in set (0.00 sec)
```

Fig. 63 F3: NULLIF: share of blocked attacks per asset without dividing by zero

F4. Date and time functions: how long each incident has been or was open

```
mysql> SELECT incident_id,
    ->        DATE_FORMAT(opened_at, '%d %b %Y %H:%i') AS opened,
    ->        DAYNAME(opened_at) AS opened_on,
    ->        TIMESTAMPDIFF(HOUR, opened_at, COALESCE(closed_at, '2026-10-28 12:00:00')) AS hours_open,
    ->        DATEDIFF('2026-10-28', DATE(opened_at)) AS age_days
    -> FROM incident
    -> ORDER BY incident_id;
+-------------+------------------+-----------+------------+----------+
| incident_id | opened           | opened_on | hours_open | age_days |
+-------------+------------------+-----------+------------+----------+
|           1 | 01 Oct 2026 02:25 | Thursday  |          7 |       27 |
|           2 | 04 Oct 2026 22:20 | Sunday    |         43 |       24 |
|           3 | 05 Oct 2026 01:10 | Monday    |        562 |       23 |
|           4 | 12 Oct 2026 08:00 | Monday    |        388 |       16 |
|           5 | 20 Oct 2026 09:00 | Tuesday   |        195 |        8 |
|           6 | 20 Oct 2026 18:10 | Tuesday   |        185 |        8 |
|           7 | 18 Oct 2026 02:10 | Sunday    |        249 |       10 |
+-------------+------------------+-----------+------------+----------+
7 rows in set (0.00 sec)
```

Fig. 64 F4: Date and time functions: how long each incident has been or was open

F5. String functions: analyst user names and short names

```
mysql> SELECT LPAD(analyst_id, 4, '0')                    AS staff_no,
    ->        UPPER(full_name)                             AS name_upper,
    ->        CONCAT(LEFT(full_name, 1), '. ',
    ->               SUBSTRING_INDEX(full_name, ' ', -1))  AS short_name,
    ->        SUBSTRING_INDEX(email, '@', 1)               AS username,
    ->        LENGTH(email)                                AS email_length
    -> FROM analyst
    -> ORDER BY analyst_id;
+----------+-------------+------------+-------------+--------------+
| staff_no | name_upper  | short_name | username    | email_length |
+----------+-------------+------------+-------------+--------------+
| 0001     | ANANYA RAO  | A. Rao     | ananya.rao  |           23 |
| 0002     | RAHUL MEHTA | R. Mehta   | rahul.mehta |           24 |
| 0003     | PRIYA NAIR  | P. Nair    | priya.nair  |           23 |
| 0004     | KARAN SHAH  | K. Shah    | karan.shah  |           23 |
| 0005     | DIVYA IYER  | D. Iyer    | divya.iyer  |           23 |
| 0006     | ARJUN DAS   | A. Das     | arjun.das   |           22 |
+----------+-------------+------------+-------------+--------------+
6 rows in set (0.00 sec)
```

Fig. 65 F5: String functions: analyst user names and short names

F6. Numeric functions on CVSS scores

```
mysql> SELECT cve_id, cvss,
    ->        ROUND(cvss)               AS rounded,
    ->        CEIL(cvss)                AS ceiling,
    ->        FLOOR(cvss)               AS floor_value,
    ->        TRUNCATE(cvss * 10, 0)    AS pct_of_max,
    ->        MOD(FLOOR(cvss * 10), 7)  AS bucket
    -> FROM vulnerability
    -> ORDER BY cvss DESC, cve_id;
+----------------+------+---------+---------+-------------+------------+--------+
| cve_id         | cvss | rounded | ceiling | floor_value | pct_of_max | bucket |
+----------------+------+---------+---------+-------------+------------+--------+
| CVE-2021-44228 | 10.0 |      10 |      10 |          10 |        100 |      2 |
| CVE-2024-3400  | 10.0 |      10 |      10 |          10 |        100 |      2 |
| CVE-2019-0708  |  9.8 |      10 |      10 |           9 |         98 |      0 |
| CVE-2022-22965 |  9.8 |      10 |      10 |           9 |         98 |      0 |
| CVE-2023-34362 |  9.8 |      10 |      10 |           9 |         98 |      0 |
| CVE-2023-4966  |  9.4 |       9 |      10 |           9 |         94 |      3 |
| CVE-2024-6387  |  8.1 |       8 |       9 |           8 |         81 |      4 |
| CVE-2023-44487 |  7.5 |       8 |       8 |           7 |         75 |      5 |
| CVE-2023-48795 |  5.9 |       6 |       6 |           5 |         59 |      3 |
+----------------+------+---------+---------+-------------+------------+--------+
9 rows in set (0.00 sec)
```

Fig. 66 F6: Numeric functions on CVSS scores

11.7 G. Window functions

G1. RANK, DENSE_RANK and ROW_NUMBER: assets ranked by number of attacks

RANK leaves gaps after ties (3, 3, 5), DENSE_RANK does not (3, 3, 4), and ROW_NUMBER is always unique.

```
mysql> SELECT a.hostname,
    ->        COUNT(e.event_id) AS attacks,
    ->        RANK()       OVER (ORDER BY COUNT(e.event_id) DESC) AS rnk,
    ->        DENSE_RANK() OVER (ORDER BY COUNT(e.event_id) DESC) AS dense_rnk,
    ->        ROW_NUMBER() OVER (ORDER BY COUNT(e.event_id) DESC, a.hostname) AS row_no
    -> FROM asset a
    -> LEFT JOIN attack_event e ON e.asset_id = a.asset_id
    -> GROUP BY a.hostname
    -> ORDER BY row_no;
+-----------+---------+-----+-----------+--------+
| hostname  | attacks | rnk | dense_rnk | row_no |
+-----------+---------+-----+-----------+--------+
| web-01    |       7 |   1 |         1 |      1 |
| web-02    |       5 |   2 |         2 |      2 |
| db-01     |       4 |   3 |         3 |      3 |
| vpn-01    |       4 |   3 |         3 |      4 |
| mail-01   |       3 |   5 |         4 |      5 |
| cam-01    |       2 |   6 |         5 |      6 |
| dns-01    |       2 |   6 |         5 |      7 |
| ws-fin-01 |       2 |   6 |         5 |      8 |
| ws-hr-01  |       1 |   9 |         6 |      9 |
| ws-dev-01 |       0 |  10 |         7 |     10 |
+-----------+---------+-----+-----------+--------+
10 rows in set (0.00 sec)
```

Fig. 67 G1: RANK, DENSE_RANK and ROW_NUMBER: assets ranked by number of attacks

G2. Running total of attacks per day for the first ten days of October

```
mysql> SELECT DATE(event_time) AS day,
    ->        COUNT(*) AS attacks,
    ->        SUM(COUNT(*)) OVER (ORDER BY DATE(event_time)) AS running_total
    -> FROM attack_event
    -> WHERE event_time < '2026-10-11'
    -> GROUP BY DATE(event_time)
    -> ORDER BY day;
+------------+---------+---------------+
| day        | attacks | running_total |
+------------+---------+---------------+
| 2026-10-01 |       2 |             2 |
| 2026-10-02 |       1 |             3 |
| 2026-10-03 |       1 |             4 |
| 2026-10-04 |       1 |             5 |
| 2026-10-05 |       2 |             7 |
| 2026-10-06 |       2 |             9 |
| 2026-10-07 |       2 |            11 |
| 2026-10-08 |       2 |            13 |
| 2026-10-09 |       1 |            14 |
| 2026-10-10 |       2 |            16 |
+------------+---------+---------------+
10 rows in set (0.00 sec)
```

Fig. 68 G2: Running total of attacks per day for the first ten days of October

G3. LAG and LEAD: minutes between attacks from the same Russian source IP

```
mysql> SELECT src_ip, event_id, event_time,
    ->        TIMESTAMPDIFF(MINUTE,
    ->                      LAG(event_time) OVER w, event_time) AS mins_since_prev,
    ->        LEAD(event_id) OVER w AS next_event
    -> FROM attack_event
    -> WHERE src_country = 'RU'
    -> WINDOW w AS (PARTITION BY src_ip ORDER BY event_time)
    -> ORDER BY src_ip, event_time;
+---------------+----------+---------------------+-----------------+------------+
| src_ip        | event_id | event_time          | mins_since_prev | next_event |
+---------------+----------+---------------------+-----------------+------------+
| 198.51.100.23 |        1 | 2026-10-01 02:14:00 |            NULL |          2 |
| 198.51.100.23 |        2 | 2026-10-01 02:20:00 |               6 |         17 |
| 198.51.100.23 |       17 | 2026-10-11 12:00:00 |           14980 |         29 |
| 198.51.100.23 |       29 | 2026-10-25 11:11:00 |           20111 |       NULL |
| 198.51.100.45 |       10 | 2026-10-07 03:33:00 |            NULL |         11 |
| 198.51.100.45 |       11 | 2026-10-07 04:10:00 |              37 |         23 |
| 198.51.100.45 |       23 | 2026-10-18 02:02:00 |           15712 |         24 |
| 198.51.100.45 |       24 | 2026-10-18 02:05:00 |               3 |       NULL |
+---------------+----------+---------------------+-----------------+------------+
8 rows in set (0.00 sec)
```

Fig. 69 G3: LAG and LEAD: minutes between attacks from the same Russian source IP

12. Advanced SQL Implementation

12.1 Complex Business Queries

Each query uses at least three tables or a nested or analytic construct. The expected output was worked out by hand from the test data before running; the actual output is the screenshot. All 11 matched.

CQ-01

Field	Content
ID	CQ-01
Business question	Which threat actor is behind the most serious attacks (severity 8+) on our important assets (criticality 4+)?

SQL	`SELECT t.actor_name, t.motive,`
	`  COUNT(*) AS serious_attacks,`
	`  COUNT(DISTINCT e.asset_id) AS assets_hit,`
	`  MAX(e.severity) AS worst`
	`FROM attack_event e`
	`JOIN threat_actor t ON t.actor_id = e.actor_id`
	`JOIN asset a ON a.asset_id = e.asset_id`
	`WHERE e.severity >= 8 AND a.criticality >= 4`
	`GROUP BY t.actor_id, t.actor_name, t.motive`
	`ORDER BY serious_attacks DESC, t.actor_name;`

Expected output	4 rows. Crimson Lynx first with 4 serious attacks on 3 assets (events 10, 11, 24, 29); Iron Kestrel 3 on db-01; Silent Monsoon 3 on 2 assets; Byte Bandits 1. Ghost Parrot is absent because its worst attack is severity 7.
Actual output screenshot	Below (Fig. 70)
Matches the prediction	Yes: 4 rows, as predicted, with the values listed above
Explanation	Three tables: attack_event joined to threat_actor (drops unattributed attacks) and to asset. WHERE keeps severity 8+ on criticality 4+ assets before grouping by actor; COUNT(DISTINCT asset_id) counts assets, not attacks.

```
mysql> SELECT t.actor_name, t.motive,
    ->        COUNT(*)                  AS serious_attacks,
    ->        COUNT(DISTINCT e.asset_id) AS assets_hit,
    ->        MAX(e.severity)           AS worst
    -> FROM attack_event e
    -> JOIN threat_actor t ON t.actor_id = e.actor_id
    -> JOIN asset a        ON a.asset_id = e.asset_id
    -> WHERE e.severity >= 8 AND a.criticality >= 4
    -> GROUP BY t.actor_id, t.actor_name, t.motive
    -> ORDER BY serious_attacks DESC, t.actor_name;
+---------------+------------+-----------------+------------+-------+
| actor_name    | motive     | serious_attacks | assets_hit | worst |
+---------------+------------+-----------------+------------+-------+
| Crimson Lynx  | financial  |               4 |          3 |    10 |
| Iron Kestrel  | sabotage   |               3 |          1 |    10 |
| Silent Monsoon| espionage  |               3 |          2 |    10 |
| Byte Bandits  | hacktivism |               1 |          1 |     8 |
+---------------+------------+-----------------+------------+-------+
4 rows in set (0.00 sec)
```

Fig. 70 CQ-01 actual output

CQ-02

Field	Content
ID	CQ-02
Business question	What does the board of incidents that are still not closed look like?

SQL	`SELECT i.incident_id, i.priority, i.status,`
	`COALESCE(an.full_name, 'Unassigned') AS owner,`
	`COUNT(*) AS attacks,`
	`MAX(e.severity) AS worst,`
	`GROUP_CONCAT(DISTINCT a.hostname ORDER BY a.hostname) AS assets`
	`FROM incident i`
	`JOIN incident_event ie ON ie.incident_id = i.incident_id`
	`JOIN attack_event e ON e.event_id = ie.event_id`
	`JOIN asset a ON a.asset_id = e.asset_id`
	`LEFT JOIN analyst an ON an.analyst_id = i.assigned_to`
	`WHERE i.status <> 'CLOSED'`
	`GROUP BY i.incident_id, i.priority, i.status, an.full_name`
	`ORDER BY FIELD(i.priority, 'CRITICAL', 'HIGH', 'MEDIUM', 'LOW'), i.incident_id;`

Expected output	5 incidents (all except the closed 1 and 2): 4 and 5 CRITICAL, 3 and 7 HIGH, 6 MEDIUM. Incident 5 has no owner; incident 6 covers web-01 and web-02 with 5 attacks.
Actual output screenshot	Below (Fig. 71)
Matches the prediction	Yes: 5 rows, as predicted, with the values listed above
Explanation	Five tables. incident to incident_event to attack_event to asset gives the attacks and hosts; LEFT JOIN analyst keeps the unassigned incident and COALESCE prints "Unassigned". FIELD() sorts by priority order instead of alphabetically.

```
mysql> SELECT i.incident_id, i.priority, i.status,
    ->        COALESCE(an.full_name, 'Unassigned')      AS owner,
    ->        COUNT(*)                                  AS attacks,
    ->        MAX(e.severity)                           AS worst,
    ->        GROUP_CONCAT(DISTINCT a.hostname ORDER BY a.hostname) AS assets
    -> FROM incident i
    -> JOIN incident_event ie ON ie.incident_id = i.incident_id
    -> JOIN attack_event e    ON e.event_id     = ie.event_id
    -> JOIN asset a           ON a.asset_id     = e.asset_id
    -> LEFT JOIN analyst an   ON an.analyst_id  = i.assigned_to
    -> WHERE i.status <> 'CLOSED'
    -> GROUP BY i.incident_id, i.priority, i.status, an.full_name
    -> ORDER BY FIELD(i.priority, 'CRITICAL', 'HIGH', 'MEDIUM', 'LOW'), i.incident_id;
+-------------+----------+---------------+-------------+---------+-------+---------------+
| incident_id | priority | status        | owner       | attacks | worst | assets        |
+-------------+----------+---------------+-------------+---------+-------+---------------+
|           4 | CRITICAL | INVESTIGATING | Karan Shah  |       2 |    10 | web-02        |
|           5 | CRITICAL | OPEN          | Unassigned  |       2 |    10 | vpn-01        |
|           3 | HIGH     | CONTAINED     | Rahul Mehta |       3 |    10 | db-01         |
|           7 | HIGH     | INVESTIGATING | Rahul Mehta |       2 |     8 | web-01        |
|           6 | MEDIUM   | OPEN          | Priya Nair  |       5 |     8 | web-01,web-02 |
+-------------+----------+---------------+-------------+---------+-------+---------------+
5 rows in set (0.00 sec)
```

Fig. 71 CQ-02 actual output

CQ-03

Field	Content
ID	CQ-03
Business question	Which assets were attacked through a CVE they still have not patched?

Field	Content

SQL	`SELECT a.hostname, e.event_id, e.cve_id, v.cvss, av.detected_at,`
	`  DATEDIFF('2026-10-28', av.detected_at) AS days_unpatched`
	`FROM attack_event e`
	`JOIN asset_vulnerability av ON av.asset_id = e.asset_id`
	`  AND av.cve_id = e.cve_id`
	`  AND av.patched_at IS NULL`
	`JOIN vulnerability v ON v.cve_id = e.cve_id`
	`JOIN asset a ON a.asset_id = e.asset_id`
	`ORDER BY v.cvss DESC, a.hostname;`

Expected output	5 rows: vpn-01 (CVE-2024-3400), web-02, mail-01, web-01, dns-01. db-01 (MOVEit) and vpn-01 (regreSSHion) must not appear because they were patched before the attack.
Actual output screenshot	Below (Fig. 72)
Matches the prediction	Yes: 5 rows, as predicted, with the values listed above
Explanation	The join to asset_vulnerability uses both asset_id and cve_id plus patched_at IS NULL in the ON clause, so an attack only matches if that exact asset still had that exact CVE open. DATEDIFF gives days since detection.

```
mysql> SELECT a.hostname, e.event_id, e.cve_id, v.cvss, av.detected_at,
    ->        DATEDIFF('2026-10-28', av.detected_at) AS days_unpatched
    -> FROM attack_event e
    -> JOIN asset_vulnerability av ON av.asset_id = e.asset_id
    ->                            AND av.cve_id   = e.cve_id
    ->                            AND av.patched_at IS NULL
    -> JOIN vulnerability v ON v.cve_id   = e.cve_id
    -> JOIN asset a         ON a.asset_id = e.asset_id
    -> ORDER BY v.cvss DESC, a.hostname;
+----------+----------+----------------+------+-------------+----------------+
| hostname | event_id | cve_id         | cvss | detected_at | days_unpatched |
+----------+----------+----------------+------+-------------+----------------+
| vpn-01   |       15 | CVE-2024-3400  | 10.0 | 2026-09-01  |             57 |
| web-02   |       10 | CVE-2022-22965 |  9.8 | 2026-09-05  |             53 |
| mail-01  |        5 | CVE-2023-4966  |  9.4 | 2026-09-12  |             46 |
| web-01   |       14 | CVE-2023-44487 |  7.5 | 2026-09-15  |             43 |
| dns-01   |       28 | CVE-2023-48795 |  5.9 | 2026-09-20  |             38 |
+----------+----------+----------------+------+-------------+----------------+
5 rows in set (0.00 sec)
```

Fig. 72 CQ-03 actual output

CQ-04

Field	Content
ID	CQ-04
Business question	Which malware families cause the most attacks, and who uses them?

SQL	`SELECT m.family,`
	`  COUNT(*) AS attacks,`
	`  GROUP_CONCAT(DISTINCT COALESCE(t.actor_name, 'Unknown')) AS used_by,`
	`  ROUND(100 * COUNT(*) / (SELECT COUNT(*) FROM attack_event`
	`  WHERE malware_id IS NOT NULL), 1) AS`
	`pct_of_malware_attacks`
	`FROM attack_event e`
	`JOIN malware m ON m.malware_id = e.malware_id`
	`LEFT JOIN threat_actor t ON t.actor_id = m.actor_id`
	`GROUP BY m.family`
	`ORDER BY attacks DESC, m.family;`

Field	Content
Expected output	7 families. Botnet (ParrotBot) leads with 4 of the 18 malware attacks, about 22%. Miner is "Unknown" because OrphanMiner has no actor.
Actual output screenshot	Below (Fig. 73)
Matches the prediction	Yes: 7 rows, as predicted, with the values listed above
Explanation	attack_event, malware and threat_actor (LEFT JOIN keeps the actor-less malware). The scalar subquery in the SELECT gives the denominator, the count of attacks that carried any malware.

```
mysql> SELECT m.family,
    ->        COUNT(*) AS attacks,
    ->        GROUP_CONCAT(DISTINCT COALESCE(t.actor_name, 'Unknown')) AS used_by,
    ->        ROUND(100 * COUNT(*) / (SELECT COUNT(*) FROM attack_event
    ->                                WHERE malware_id IS NOT NULL), 1) AS pct_of_malware_attacks
    -> FROM attack_event e
    -> JOIN malware m          ON m.malware_id = e.malware_id
    -> LEFT JOIN threat_actor t ON t.actor_id   = m.actor_id
    -> GROUP BY m.family
    -> ORDER BY attacks DESC, m.family;
+---------+---------+----------------+------------------------+
| family  | attacks | used_by        | pct_of_malware_attacks |
+---------+---------+----------------+------------------------+
| Botnet  |       4 | Ghost Parrot   |                   22.2 |
| Loader  |       3 | Crimson Lynx   |                   16.7 |
| RAT     |       3 | Silent Monsoon |                   16.7 |
| Stealer |       3 | Silent Monsoon |                   16.7 |
| Miner   |       2 | Unknown        |                   11.1 |
| Wiper   |       2 | Iron Kestrel   |                   11.1 |
| Locker  |       1 | Crimson Lynx   |                    5.6 |
+---------+---------+----------------+------------------------+
7 rows in set (0.00 sec)
```

Fig. 73 CQ-04 actual output

CQ-05

Field	Content
ID	CQ-05
Business question	Time machine: how many attacks matched an indicator we already knew about at the time, and how late were we for the rest?

SQL	`SELECT CASE WHEN i.recorded_from <= e.event_time THEN 'known at attack time'`
	`ELSE 'recognised later' END AS knowledge,`
	`COUNT(*) AS matches,`
	`COUNT(DISTINCT e.event_id) AS attacks,`
	`ROUND(AVG(CASE WHEN i.recorded_from > e.event_time`
	`THEN TIMESTAMPDIFF(HOUR, e.event_time, i.recorded_from) / 24`
	`END), 1) AS avg_days_late`
	`FROM attack_event e`
	`JOIN event_ioc ei ON ei.event_id = e.event_id`
	`JOIN ioc i ON i.ioc_version_id = ei.ioc_version_id`
	`GROUP BY knowledge`
	`ORDER BY knowledge;`

Expected output	27 attack-indicator matches: 18 known at attack time (17 attacks) and 9 recognised later (8 attacks), on average about 5 days late.
Actual output screenshot	Below (Fig. 74)

Field	Content
Matches the prediction	Yes: 2 rows, as predicted, with the values listed above
Explanation	Joins attack_event to event_ioc to ioc and compares recorded_from (when we learned) with event_time. The CASE in AVG returns NULL for "known" rows so only late rows are averaged. Hours / 24 keeps fractions of a day.

```
mysql> SELECT CASE WHEN i.recorded_from <= e.event_time THEN 'known at attack time'
    ->            ELSE 'recognised later' END                AS knowledge,
    ->       COUNT(*)                                         AS matches,
    ->       COUNT(DISTINCT e.event_id)                       AS attacks,
    ->       ROUND(AVG(CASE WHEN i.recorded_from > e.event_time
    ->                      THEN TIMESTAMPDIFF(HOUR, e.event_time, i.recorded_from) / 24
    ->                 END), 1)                              AS avg_days_late
    -> FROM attack_event e
    -> JOIN event_ioc ei ON ei.event_id      = e.event_id
    -> JOIN ioc i        ON i.ioc_version_id = ei.ioc_version_id
    -> GROUP BY knowledge
    -> ORDER BY knowledge;
+---------------------+---------+---------+---------------+
| knowledge           | matches | attacks | avg_days_late |
+---------------------+---------+---------+---------------+
| known at attack time |      18 |      17 |          NULL |
| recognised later     |       9 |       8 |           4.9 |
+---------------------+---------+---------+---------------+
2 rows in set (0.00 sec)
```

Fig. 74 CQ-05 actual output

CQ-06

Field	Content
ID	CQ-06
Business question	What did each retro-hunt find, and what happened to the incident it opened?

SQL	`SELECT r.run_id, i.ioc_value, r.started_at,`
	`COUNT(h.log_id) AS logs_matched,`
	`COUNT(DISTINCT l.dst_asset_id) AS assets_touched,`
	`MIN(l.log_time) AS first_contact,`
	`inc.status AS incident_status`
	`FROM retro_hunt_run r`
	`JOIN ioc i ON i.ioc_version_id = r.ioc_version_id`
	`LEFT JOIN retro_hit h ON h.run_id = r.run_id`
	`LEFT JOIN network_log l ON l.log_id = h.log_id`
	`LEFT JOIN incident inc ON inc.incident_id = r.incident_id`
	`GROUP BY r.run_id, i.ioc_value, r.started_at, inc.status`
	`ORDER BY r.run_id;`

Expected output	2 runs. Run 1 (198.51.100.45) found 4 old log lines on 2 assets, incident INVESTIGATING; run 2 (203.0.113.11) found 4 lines on 2 assets, incident OPEN.
Actual output screenshot	Below (Fig. 75)
Matches the prediction	Yes: 2 rows, as predicted, with the values listed above
Explanation	Five tables. LEFT JOINs to retro_hit and network_log keep runs with zero hits; COUNT(h.log_id) counts hits and COUNT(DISTINCT dst_asset_id) counts assets. This is the same logic as v_retro_hunt_summary.

```
mysql> SELECT r.run_id, i.ioc_value, r.started_at,
    ->        COUNT(h.log_id)               AS logs_matched,
    ->        COUNT(DISTINCT l.dst_asset_id) AS assets_touched,
    ->        MIN(l.log_time)               AS first_contact,
    ->        inc.status                    AS incident_status
    -> FROM retro_hunt_run r
    -> JOIN ioc i           ON i.ioc_version_id = r.ioc_version_id
    -> LEFT JOIN retro_hit h ON h.run_id  = r.run_id
    -> LEFT JOIN network_log l ON l.log_id = h.log_id
    -> LEFT JOIN incident inc  ON inc.incident_id = r.incident_id
    -> GROUP BY r.run_id, i.ioc_value, r.started_at, inc.status
    -> ORDER BY r.run_id;
+--------+--------------+---------------------+--------------+----------------+---------------------+-----------------+
| run_id | ioc_value    | started_at          | logs_matched | assets_touched | first_contact       | incident_status |
+--------+--------------+---------------------+--------------+----------------+---------------------+-----------------+
|      1 | 198.51.100.45 | 2026-10-12 08:00:00 |            4 |              2 | 2026-10-03 03:00:00 | INVESTIGATING   |
|      2 | 203.0.113.11  | 2026-10-20 09:00:00 |            4 |              2 | 2026-10-08 07:00:00 | OPEN            |
+--------+--------------+---------------------+--------------+----------------+---------------------+-----------------+
2 rows in set (0.00 sec)
```

Fig. 75 CQ-06 actual output

CQ-07

Field	Content
ID	CQ-07
Business question	What is the response playbook for the attack type that hits web-01 most often?

SQL	`SELECT t.type_name, m.step_no, m.action`
	`FROM mitigation m`
	`JOIN attack_type t ON t.type_id = m.type_id`
	`WHERE m.type_id = (SELECT e.type_id`
	`FROM attack_event e`
	`JOIN asset a ON a.asset_id = e.asset_id`
	`WHERE a.hostname = 'web-01'`
	`GROUP BY e.type_id`
	`ORDER BY COUNT(*) DESC`
	`LIMIT 1)`
	`ORDER BY m.step_no;`

Expected output	Brute Force (4 of web-01's 7 attacks), with its 3 steps: block the IP, lock the accounts, enforce MFA.
Actual output screenshot	Below (Fig. 76)
Matches the prediction	Yes: 3 rows, as predicted, with the values listed above
Explanation	Nested: the subquery groups web-01's attacks by type, orders by count and keeps the top type; the outer query joins mitigation to attack_type for that type.

```
mysql> SELECT t.type_name, m.step_no, m.action
    -> FROM mitigation m
    -> JOIN attack_type t ON t.type_id = m.type_id
    -> WHERE m.type_id = (SELECT e.type_id
    ->                    FROM attack_event e
    ->                    JOIN asset a ON a.asset_id = e.asset_id
    ->                    WHERE a.hostname = 'web-01'
    ->                    GROUP BY e.type_id
    ->                    ORDER BY COUNT(*) DESC
    ->                    LIMIT 1)
    -> ORDER BY m.step_no;
+-------------+---------+-----------------------------------+
| type_name   | step_no | action                            |
+-------------+---------+-----------------------------------+
| Brute Force |       1 | Block the source IP at the firewall |
| Brute Force |       2 | Lock the targeted accounts        |
| Brute Force |       3 | Enforce MFA on the login page     |
+-------------+---------+-----------------------------------+
3 rows in set (0.00 sec)
```

Fig. 76 CQ-07 actual output

CQ-08

Field	Content
ID	CQ-08
Business question	How is the open-incident workload spread across analysts?

SQL	`SELECT an.full_name,`
	`COUNT(i.incident_id) AS open_incidents,`
	`COALESCE(SUM(i.priority = 'CRITICAL'), 0) AS critical,`
	`RANK() OVER (ORDER BY COUNT(i.incident_id) DESC) AS workload_rank`
	`FROM analyst an`
	`LEFT JOIN incident i ON i.assigned_to = an.analyst_id`
	`AND i.status <> 'CLOSED'`
	`WHERE an.role = 'analyst'`
	`GROUP BY an.analyst_id, an.full_name`
	`ORDER BY workload_rank, an.full_name;`

Expected output	Rahul 2 open incidents (rank 1), Karan and Priya 1 each (rank 2, Karan's is critical), Arjun 0 (rank 4, because RANK skips after the tie).
Actual output screenshot	Below (Fig. 77)
Matches the prediction	Yes: 4 rows, as predicted, with the values listed above
Explanation	Analytic: LEFT JOIN with status <> CLOSED inside the ON clause keeps Arjun with zero. SUM(priority = 'CRITICAL') counts TRUE as 1. RANK() runs after GROUP BY.

```
mysql> SELECT an.full_name,
    ->        COUNT(i.incident_id)                          AS open_incidents,
    ->        COALESCE(SUM(i.priority = 'CRITICAL'), 0)     AS critical,
    ->        RANK() OVER (ORDER BY COUNT(i.incident_id) DESC) AS workload_rank
    -> FROM analyst an
    -> LEFT JOIN incident i ON i.assigned_to = an.analyst_id
    ->                     AND i.status <> 'CLOSED'
    -> WHERE an.role = 'analyst'
    -> GROUP BY an.analyst_id, an.full_name
    -> ORDER BY workload_rank, an.full_name;
+-------------+----------------+----------+---------------+
| full_name   | open_incidents | critical | workload_rank |
+-------------+----------------+----------+---------------+
| Rahul Mehta |              2 |        0 |             1 |
| Karan Shah  |              1 |        1 |             2 |
| Priya Nair  |              1 |        0 |             2 |
| Arjun Das   |              0 |        0 |             4 |
+-------------+----------------+----------+---------------+
4 rows in set (0.00 sec)
```

Fig. 77 CQ-08 actual output

CQ-09

Field	Content
ID	CQ-09
Business question	Which countries do attacks come from, what share does each have, and what is each country's most common attack type?

SQL	`SELECT c.src_country, c.attacks,`
	`  ROUND(100 * c.attacks / SUM(c.attacks) OVER (), 1) AS pct,`
	`  (SELECT t.type_name`
	`  FROM attack_event e2`
	`  JOIN attack_type t ON t.type_id = e2.type_id`
	`  WHERE e2.src_country = c.src_country`
	`  GROUP BY t.type_name`
	`  ORDER BY COUNT(*) DESC, t.type_name`
	`  LIMIT 1) AS top_type`
	`FROM (SELECT src_country, COUNT(*) AS attacks`
	`  FROM attack_event`
	`  GROUP BY src_country) AS c`
	`ORDER BY c.attacks DESC, c.src_country;`

Expected output	7 countries. RU 8 (26.7%), CN 6 (20.0%), BR/KP/US 4 each (13.3%), IN/NL 2 each (6.7%). RU's top type is Brute Force; KP has a 4-way tie, broken alphabetically to Credential Dumping.
Actual output screenshot	Below (Fig. 78)
Matches the prediction	Yes: 7 rows, as predicted, with the values listed above
Explanation	A derived table counts per country; SUM(...) OVER () gives the total for the percentage without a second query. A correlated subquery finds each country's most common type.

```
mysql> SELECT c.src_country, c.attacks,
    ->        ROUND(100 * c.attacks / SUM(c.attacks) OVER (), 1) AS pct,
    ->        (SELECT t.type_name
    ->           FROM attack_event e2
    ->           JOIN attack_type t ON t.type_id = e2.type_id
    ->          WHERE e2.src_country = c.src_country
    ->          GROUP BY t.type_name
    ->          ORDER BY COUNT(*) DESC, t.type_name
    ->          LIMIT 1) AS top_type
    -> FROM (SELECT src_country, COUNT(*) AS attacks
    ->       FROM attack_event
    ->       GROUP BY src_country) AS c
    -> ORDER BY c.attacks DESC, c.src_country;
+-------------+---------+------+---------------------------+
| src_country | attacks | pct  | top_type                  |
+-------------+---------+------+---------------------------+
| RU          |       8 | 26.7 | Brute Force               |
| CN          |       6 | 20.0 | Exploit Public-Facing App |
| BR          |       4 | 13.3 | DDoS                      |
| KP          |       4 | 13.3 | Credential Dumping        |
| US          |       4 | 13.3 | Port Scan                 |
| IN          |       2 |  6.7 | Brute Force               |
| NL          |       2 |  6.7 | DDoS                      |
+-------------+---------+------+---------------------------+
7 rows in set (0.00 sec)
```

Fig. 78 CQ-09 actual output

CQ-10

Field	Content
ID	CQ-10
Business question	Which assets with an unpatched critical CVE (CVSS 9+) have been attacked by an espionage group?

SQL	`SELECT DISTINCT a.hostname, av.cve_id, v.cvss, t.actor_name`
	`FROM asset_vulnerability av`
	`JOIN vulnerability v  ON v.cve_id   = av.cve_id`
	`JOIN asset a          ON a.asset_id = av.asset_id`
	`JOIN attack_event e   ON e.asset_id = av.asset_id`
	`JOIN threat_actor t   ON t.actor_id = e.actor_id`
	`WHERE av.patched_at IS NULL`
	`  AND v.cvss >= 9`
	`  AND t.motive = 'espionage'`
	`ORDER BY v.cvss DESC, a.hostname;`

Expected output	2 rows: vpn-01 with CVE-2024-3400 (10.0) and mail-01 with CVE-2023-4966 (9.4), both attacked by Silent Monsoon.
Actual output screenshot	Below (Fig. 79)
Matches the prediction	Yes: 2 rows, as predicted, with the values listed above
Explanation	Five tables. Starts from unpatched exposures with CVSS 9+, then joins attacks on the same asset by espionage groups. DISTINCT removes duplicates when a group attacked the asset more than once.

```
mysql> SELECT DISTINCT a.hostname, av.cve_id, v.cvss, t.actor_name
    -> FROM asset_vulnerability av
    -> JOIN vulnerability v  ON v.cve_id   = av.cve_id
    -> JOIN asset a          ON a.asset_id = av.asset_id
    -> JOIN attack_event e   ON e.asset_id = av.asset_id
    -> JOIN threat_actor t   ON t.actor_id = e.actor_id
    -> WHERE av.patched_at IS NULL
    ->   AND v.cvss >= 9
    ->   AND t.motive = 'espionage'
    -> ORDER BY v.cvss DESC, a.hostname;
+----------+---------------+------+----------------+
| hostname | cve_id        | cvss | actor_name     |
+----------+---------------+------+----------------+
| vpn-01   | CVE-2024-3400 | 10.0 | Silent Monsoon |
| mail-01  | CVE-2023-4966 |  9.4 | Silent Monsoon |
+----------+---------------+------+----------------+
2 rows in set (0.00 sec)
```

Fig. 79 CQ-10 actual output

CQ-11

Field	Content
ID	CQ-11
Business question	Which source IPs launched a follow-up attack within one hour (attack bursts)?

SQL	`SELECT src_ip, prev_event, event_id, gap_minutes, hostname`
	`FROM (SELECT e.src_ip, e.event_id, a.hostname,`
	`LAG(e.event_id) OVER w AS prev_event,`
	`TIMESTAMPDIFF(MINUTE, LAG(e.event_time) OVER w, e.event_time) AS gap_minutes`
	`FROM attack_event e`
	`JOIN asset a ON a.asset_id = e.asset_id`
	`WINDOW w AS (PARTITION BY e.src_ip ORDER BY e.event_time)) AS x`
	`WHERE gap_minutes <= 60`
	`ORDER BY gap_minutes, src_ip;`

Expected output	7 bursts. The fastest is 2 minutes (203.0.113.200 scanning two hosts), the slowest within the hour is 45 minutes (192.0.2.50 on db-01).
Actual output screenshot	Below (Fig. 80)
Matches the prediction	Yes: 7 rows, as predicted, with the values listed above
Explanation	LAG over a window partitioned by source IP gives each attack's previous attack from the same IP; the outer query keeps gaps of 60 minutes or less. The filter needs a derived table because window results cannot be used in WHERE.

```
mysql> SELECT src_ip, prev_event, event_id, gap_minutes, hostname
    -> FROM (SELECT e.src_ip, e.event_id, a.hostname,
    ->              LAG(e.event_id) OVER w AS prev_event,
    ->              TIMESTAMPDIFF(MINUTE, LAG(e.event_time) OVER w, e.event_time) AS gap_minutes
    ->       FROM attack_event e
    ->       JOIN asset a ON a.asset_id = e.asset_id
    ->       WINDOW w AS (PARTITION BY e.src_ip ORDER BY e.event_time)) AS x
    -> WHERE gap_minutes <= 60
    -> ORDER BY gap_minutes, src_ip;
+---------------+------------+----------+-------------+----------+
| src_ip        | prev_event | event_id | gap_minutes | hostname |
+---------------+------------+----------+-------------+----------+
| 203.0.113.200 |         12 |       13 |           2 | vpn-01   |
| 198.51.100.45 |         23 |       24 |           3 | web-01   |
| 198.51.100.88 |          8 |        9 |           5 | web-02   |
| 198.51.100.23 |          1 |        2 |           6 | web-01   |
| 203.0.113.11  |         15 |       16 |          25 | vpn-01   |
| 198.51.100.45 |         10 |       11 |          37 | web-02   |
| 192.0.2.50    |          6 |        7 |          45 | db-01    |
+---------------+------------+----------+-------------+----------+
7 rows in set (0.00 sec)
```

Fig. 80 CQ-11 actual output

12.2 Views

Six views: four written for the project and the two Review 1 views (v_vulnerability for CVE severity and v_retro_hunt_summary for retro-hunt counts) that replaced stored columns. v_ioc was added in Review 3 so that analysts no longer need any access to the ioc table. Whether each view is updatable is read from information_schema (Fig. 86).

v_attack_details (joins 5 tables)

Field	Content
Purpose	One flat row per attack with host, type, actor and malware names, so searches and reports need no joins.

CREATE VIEW statement	`CREATE VIEW v_attack_details AS`
	`SELECT e.event_id, e.event_time, e.src_ip, e.src_country,`
	`  a.hostname, t.type_name,`
	`  ta.actor_name, m.malware_name, e.cve_id,`
	`  e.severity, e.blocked`
	`FROM attack_event e`
	`JOIN asset a  ON a.asset_id = e.asset_id`
	`JOIN attack_type t  ON t.type_id = e.type_id`
	`LEFT JOIN threat_actor ta ON ta.actor_id = e.actor_id`
	`LEFT JOIN malware m  ON m.malware_id = e.malware_id;`

Query using the view	`SELECT event_id, event_time, src_ip, type_name, actor_name, severity`
	`FROM v_attack_details`
	`WHERE hostname = 'web-01'`
	`ORDER BY event_time;`

Screenshot	Fig. 81
Why useful	Abstraction and simplification: queries use one object.
Updatable?	No: it uses LEFT JOINs, so MySQL cannot map a change to one base row; the DELETE in the figure is refused (ERROR 1288). Only SELECT is granted.

```
mysql> CREATE VIEW v_attack_details AS
    -> SELECT e.event_id, e.event_time, e.src_ip, e.src_country,
    ->        a.hostname, t.type_name,
    ->        ta.actor_name, m.malware_name, e.cve_id,
    ->        e.severity, e.blocked
    -> FROM attack_event e
    -> JOIN asset a             ON a.asset_id   = e.asset_id
    -> JOIN attack_type t       ON t.type_id    = e.type_id
    -> LEFT JOIN threat_actor ta ON ta.actor_id  = e.actor_id
    -> LEFT JOIN malware m       ON m.malware_id = e.malware_id;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT event_id, event_time, src_ip, type_name, actor_name, severity
    -> FROM v_attack_details
    -> WHERE hostname = 'web-01'
    -> ORDER BY event_time;
+----------+---------------------+--------------+-------------+--------------+----------+
| event_id | event_time          | src_ip       | type_name   | actor_name   | severity |
+----------+---------------------+--------------+-------------+--------------+----------+
|        1 | 2026-10-01 02:14:00 | 198.51.100.23 | Brute Force | Crimson Lynx |        6 |
|        2 | 2026-10-01 02:20:00 | 198.51.100.23 | Brute Force | Crimson Lynx |        7 |
|        3 | 2026-10-02 11:05:00 | 203.0.113.200 | Port Scan   | NULL         |        3 |
|       14 | 2026-10-09 16:45:00 | 192.0.2.77    | DDoS        | Byte Bandits |        8 |
|       19 | 2026-10-13 07:05:00 | 198.51.100.88 | DDoS        | Ghost Parrot |        6 |
|       23 | 2026-10-18 02:02:00 | 198.51.100.45 | Brute Force | Crimson Lynx |        7 |
|       24 | 2026-10-18 02:05:00 | 198.51.100.45 | Brute Force | Crimson Lynx |        8 |
+----------+---------------------+--------------+-------------+--------------+----------+
7 rows in set (0.00 sec)

mysql> DELETE FROM v_attack_details WHERE event_id = 3;
ERROR 1288 (HY000): The target table v_attack_details of the DELETE is not updatable
```

Fig. 81 v_attack_details: definition, use and updatability

v_asset_risk (aggregated)

Field	Content
Purpose	Per asset: attacks, attacks not blocked, worst severity and open vulnerabilities, for risk reporting.

CREATE VIEW statement	`CREATE VIEW v_asset_risk AS`
	`SELECT a.asset_id, a.hostname, a.criticality,`
	`COUNT(e.event_id) AS attacks,`
	`COALESCE(SUM(e.blocked = 0), 0) AS not_blocked,`
	`MAX(e.severity) AS worst_severity,`
	`(SELECT COUNT(*) FROM asset_vulnerability av`
	`WHERE av.asset_id = a.asset_id AND av.patched_at IS NULL) AS open_vulns`
	`FROM asset a`
	`LEFT JOIN attack_event e ON e.asset_id = a.asset_id`
	`GROUP BY a.asset_id, a.hostname, a.criticality;`

Query using the view	`SELECT hostname, criticality, attacks, not_blocked, worst_severity, open_vulns`
	`FROM v_asset_risk`
	`ORDER BY not_blocked DESC, criticality DESC`
	`LIMIT 5;`

Screenshot	Fig. 82
Why useful	Simplification: the aggregation is written once.
Updatable?	No: GROUP BY, COUNT/SUM and a subquery; the UPDATE in the figure is refused (ERROR 1288).

```
mysql> CREATE VIEW v_asset_risk AS
    -> SELECT a.asset_id, a.hostname, a.criticality,
    ->        COUNT(e.event_id)              AS attacks,
    ->        COALESCE(SUM(e.blocked = 0), 0) AS not_blocked,
    ->        MAX(e.severity)                AS worst_severity,
    ->        (SELECT COUNT(*) FROM asset_vulnerability av
    ->          WHERE av.asset_id = a.asset_id AND av.patched_at IS NULL) AS open_vulns
    -> FROM asset a
    -> LEFT JOIN attack_event e ON e.asset_id = a.asset_id
    -> GROUP BY a.asset_id, a.hostname, a.criticality;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT hostname, criticality, attacks, not_blocked, worst_severity, open_vulns
    -> FROM v_asset_risk
    -> ORDER BY not_blocked DESC, criticality DESC
    -> LIMIT 5;
+----------+-------------+---------+-------------+----------------+------------+
| hostname | criticality | attacks | not_blocked | worst_severity | open_vulns |
+----------+-------------+---------+-------------+----------------+------------+
| web-01   |           5 |       7 |           4 |              8 |          1 |
| db-01    |           5 |       4 |           2 |             10 |          0 |
| web-02   |           4 |       5 |           2 |             10 |          1 |
| vpn-01   |           4 |       4 |           2 |             10 |          1 |
| mail-01  |           3 |       3 |           2 |              9 |          1 |
+----------+-------------+---------+-------------+----------------+------------+
5 rows in set (0.00 sec)

mysql> UPDATE v_asset_risk SET criticality = 5 WHERE asset_id = 2;
ERROR 1288 (HY000): The target table v_asset_risk of the UPDATE is not updatable
```

Fig. 82 v_asset_risk: definition, use and updatability

v_my_incidents (security, row-restricting)

Field	Content
Purpose	Shows each analyst only the incidents assigned to them, by matching the MySQL login (USER()) to the analyst's email.

CREATE VIEW statement	`CREATE VIEW v_my_incidents AS`
	`SELECT incident_id, title, priority, status, assigned_to, opened_at, closed_at`
	`FROM incident`
	`WHERE assigned_to = (SELECT analyst_id FROM analyst`
	`WHERE email = CONCAT(SUBSTRING_INDEX(USER(), '@', 1),`
	`'@sentinelx.in'))`
	`WITH CHECK OPTION;`

Query using the view	`SELECT USER() AS logged_in_as, COUNT(*) AS rows_visible FROM v_my_incidents;`
Screenshot	Fig. 83
Why useful	Security: analysts have no privilege on the incident table, only on this view. As root it shows 0 rows, because root is not an analyst; rahul's session shows his own incidents (Fig. 115).
Updatable?	Yes: one table, no aggregation or join. WITH CHECK OPTION blocks changes that would move a row out of the view, such as handing an incident to someone else (ERROR 1369).

```
mysql> CREATE VIEW v_my_incidents AS
    -> SELECT incident_id, title, priority, status, assigned_to, opened_at, closed_at
    -> FROM incident
    -> WHERE assigned_to = (SELECT analyst_id FROM analyst
    ->                      WHERE email = CONCAT(SUBSTRING_INDEX(USER(), '@', 1), '@sentinelx.in'))
    -> WITH CHECK OPTION;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT USER() AS logged_in_as, COUNT(*) AS rows_visible FROM v_my_incidents;
+----------------+--------------+
| logged_in_as   | rows_visible |
+----------------+--------------+
| root@localhost |            0 |
+----------------+--------------+
1 row in set (0.00 sec)

mysql> SELECT table_name, is_updatable
    -> FROM information_schema.views
    -> WHERE table_schema = 'sentinelx'
    -> ORDER BY table_name;
+------------------+--------------+
| TABLE_NAME       | IS_UPDATABLE |
+------------------+--------------+
| v_asset_risk     | NO           |
| v_attack_details | NO           |
| v_my_incidents   | YES          |
+------------------+--------------+
3 rows in set (0.00 sec)
```

Fig. 83 v_my_incidents: definition, use and updatability

v_ioc (security, Review 3)

Field	Content
Purpose	The indicator versions we currently believe (recorded_to IS NULL), so analysts can read indicators without any privilege on the ioc table.

CREATE VIEW statement	`CREATE VIEW v_ioc AS`
	`SELECT ioc_version_id, ioc_type, ioc_value, actor_id, malware_id,`
	`  valid_from, valid_to, recorded_from`
	`FROM ioc`
	`WHERE recorded_to IS NULL;`

Query using the view	`SELECT ioc_type, COUNT(*) AS indicators FROM v_ioc GROUP BY ioc_type ORDER BY ioc_type;`
Screenshot	Fig. 84
Why useful	Security and simplification: hides superseded versions and the base table.
Updatable?	Yes in MySQL's terms (one table, simple WHERE), but analysts only get SELECT; new indicators go through sp_report_ioc.

```
mysql> CREATE VIEW v_ioc AS
    -> SELECT ioc_version_id, ioc_type, ioc_value, actor_id, malware_id,
    ->        valid_from, valid_to, recorded_from
    -> FROM ioc
    -> WHERE recorded_to IS NULL;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT ioc_type, COUNT(*) AS indicators FROM v_ioc GROUP BY ioc_type ORDER BY ioc_type;
+----------+------------+
| ioc_type | indicators |
+----------+------------+
| ip       |          8 |
| domain   |          2 |
| hash     |          1 |
| url      |          1 |
+----------+------------+
4 rows in set (0.00 sec)
```

Fig. 84 v_ioc: definition and use

v_vulnerability (Review 1)

Field	Content
Purpose	Each CVE with a severity label worked out from its CVSS score, instead of storing severity.

CREATE VIEW statement	`CREATE VIEW v_vulnerability AS`
	`SELECT cve_id, cvss, published,`
	`  CASE WHEN cvss >= 9 THEN 'CRITICAL'`
	`  WHEN cvss >= 7 THEN 'HIGH'`
	`  WHEN cvss >= 4 THEN 'MEDIUM'`
	`  ELSE 'LOW' END AS severity`
	`FROM vulnerability;`

Query using the view	None separately: its definition and severity column are shown in the figure.
Screenshot	Fig. 85
Why useful	Keeps the vulnerability table in 3NF (no cvss -> severity dependency) while queries still see the label.
Updatable?	Yes for cve_id, cvss and published; the computed severity column itself cannot be updated.

```
mysql> CREATE VIEW v_vulnerability AS
    -> SELECT cve_id, cvss, published,
    ->        CASE WHEN cvss >= 9 THEN 'CRITICAL'
    ->             WHEN cvss >= 7 THEN 'HIGH'
    ->             WHEN cvss >= 4 THEN 'MEDIUM'
    ->             ELSE 'LOW' END AS severity
    -> FROM vulnerability;
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE VIEW v_retro_hunt_summary AS
    -> SELECT r.run_id, r.ioc_version_id, r.started_at, r.incident_id,
    ->        COUNT(h.log_id)                AS logs_matched,
    ->        COUNT(DISTINCT l.dst_asset_id) AS assets_touched
    -> FROM retro_hunt_run r
    -> LEFT JOIN retro_hit   h ON h.run_id = r.run_id
    -> LEFT JOIN network_log l ON l.log_id = h.log_id
    -> GROUP BY r.run_id, r.ioc_version_id, r.started_at, r.incident_id;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT * FROM v_retro_hunt_summary;
+--------+---------------+---------------------+-------------+--------------+----------------+
| run_id | ioc_version_id | started_at          | incident_id | logs_matched | assets_touched |
+--------+---------------+---------------------+-------------+--------------+----------------+
|      1 |             2 | 2026-10-12 08:00:00 |           4 |            4 |              2 |
|      2 |             4 | 2026-10-20 09:00:00 |           5 |            4 |              2 |
+--------+---------------+---------------------+-------------+--------------+----------------+
2 rows in set (0.00 sec)
```

Fig. 85 v_vulnerability and v_retro_hunt_summary

v_retro_hunt_summary (Review 1)

Field	Content
Purpose	Per retro-hunt run: lines matched and assets touched, counted from retro_hit.

CREATE VIEW statement	`CREATE VIEW v_retro_hunt_summary AS`
	`SELECT r.run_id, r.ioc_version_id, r.started_at, r.incident_id,`
	`COUNT(h.log_id) AS logs_matched,`
	`COUNT(DISTINCT l.dst_asset_id) AS assets_touched`
	`FROM retro_hunt_run r`
	`LEFT JOIN retro_hit h ON h.run_id = r.run_id`
	`LEFT JOIN network_log l ON l.log_id = h.log_id`
	`GROUP BY r.run_id, r.ioc_version_id, r.started_at, r.incident_id;`

Query using the view	`SELECT * FROM v_retro_hunt_summary;`
Screenshot	Fig. 85
Why useful	Replaces two stored counts that could go out of sync.
Updatable?	No: GROUP BY and COUNT.

```
mysql> SELECT table_name AS view_name, is_updatable
    -> FROM information_schema.views WHERE table_schema = 'sentinelx' ORDER BY table_name;
+---------------------+--------------+
| view_name           | IS_UPDATABLE |
+---------------------+--------------+
| v_asset_risk        | NO           |
| v_attack_details    | NO           |
| v_ioc               | YES          |
| v_my_incidents      | YES          |
| v_retro_hunt_summary | NO          |
| v_vulnerability     | YES          |
+---------------------+--------------+
6 rows in set (0.00 sec)
```

Fig. 86 Updatable or not, for all six views

12.3 Stored Procedures

sp_assign_incident(incident, analyst) assigns an incident inside a transaction. It locks the incident row with SELECT ... FOR UPDATE, so two analysts cannot claim it at the same moment, and rejects closed incidents and viewer accounts with SIGNAL. Any error rolls the transaction back and is passed to the caller (RESIGNAL). **Fixed in Review 3:** the procedure runs with its definer's (root's) rights, and in Review 2 it never checked who called it, so an analyst could use it to give his incident to a colleague or take a colleague's incident, which is exactly what WITH CHECK OPTION stops on v_my_incidents. It now maps USER() (the caller; inside a DEFINER routine CURRENT_USER() is the definer) to an analyst, and analysts may only claim unassigned incidents for themselves.

```
mysql> CREATE PROCEDURE sp_assign_incident(IN p_incident INT, IN p_analyst INT)
    -> BEGIN
    ->   DECLARE v_status VARCHAR(15);
    ->   DECLARE v_owner  INT;
    ->   DECLARE v_role   VARCHAR(10);
    ->   DECLARE v_caller_id   INT;
    ->   DECLARE v_caller_role VARCHAR(10);
    ->   DECLARE EXIT HANDLER FOR SQLEXCEPTION
    ->   BEGIN
    ->     ROLLBACK;
    ->     RESIGNAL;
    ->   END;
    ->
    ->
    ->
    ->   SELECT analyst_id, role INTO v_caller_id, v_caller_role FROM analyst
    ->    WHERE email = CONCAT(SUBSTRING_INDEX(USER(), '@', 1), '@sentinelx.in');
    ->   IF v_caller_role IS NULL AND SUBSTRING_INDEX(USER(), '@', 1) <> 'root' THEN
    ->     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Caller is not a registered analyst';
    ->   END IF;
    ->
    ->   START TRANSACTION;
    ->   SELECT status, assigned_to INTO v_status, v_owner FROM incident
    ->    WHERE incident_id = p_incident FOR UPDATE;
    ->   IF v_status IS NULL THEN
    ->     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'No such incident';
    ->   ELSEIF v_status = 'CLOSED' THEN
    ->     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Incident is already closed';
    ->   END IF;
    ->
    ->
    ->   IF v_caller_role = 'analyst'
    ->      AND (p_analyst <> v_caller_id OR (v_owner IS NOT NULL AND v_owner <> v_caller_id)) THEN
    ->     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Analysts can only claim unassigned incidents for themselves';
    ->   ELSEIF v_caller_role = 'viewer' THEN
    ->     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Viewers cannot assign incidents';
    ->   END IF;
    ->
    ->   SELECT role INTO v_role FROM analyst WHERE analyst_id = p_analyst;
    ->   IF v_role IS NULL OR v_role = 'viewer' THEN
    ->     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Only analysts or admins can own incidents';
    ->   END IF;
    ->
    ->   UPDATE incident
    ->      SET assigned_to = p_analyst,
    ->          status = IF(status = 'OPEN', 'INVESTIGATING', status)
    ->    WHERE incident_id = p_incident;
    ->   COMMIT;
    ->
    ->   SELECT incident_id, status, assigned_to FROM incident WHERE incident_id = p_incident;
    -> END;
Query OK, 0 rows affected (0.01 sec)
```

Fig. 87 CREATE PROCEDURE sp_assign_incident

```
mysql> CALL sp_assign_incident(5, 6);
+-------------+---------------+-------------+
| incident_id | status        | assigned_to |
+-------------+---------------+-------------+
|           5 | INVESTIGATING |           6 |
+-------------+---------------+-------------+
1 row in set (0.01 sec)

Query OK, 0 rows affected (0.01 sec)

mysql> CALL sp_assign_incident(1, 4);
ERROR 1644 (45000): Incident is already closed

mysql> CALL sp_assign_incident(6, 5);
ERROR 1644 (45000): Only analysts or admins can own incidents
```

Fig. 88 sp_assign_incident: a valid call, a closed incident and a viewer rejected

sp_attack_report(from, to) returns the attack summary by type for a date range. The viewer role gets EXECUTE on it instead of access to the attack table.

```
mysql> CREATE PROCEDURE sp_attack_report(IN p_from DATE, IN p_to DATE)
    -> READS SQL DATA
    -> BEGIN
    ->   SELECT t.type_name,
    ->          COUNT(*)        AS attacks,
    ->          SUM(e.blocked)  AS blocked,
    ->          MAX(e.severity) AS worst
    ->   FROM attack_event e
    ->   JOIN attack_type t ON t.type_id = e.type_id
    ->   WHERE e.event_time >= p_from
    ->     AND e.event_time <  p_to + INTERVAL 1 DAY
    ->   GROUP BY t.type_name
    ->   ORDER BY attacks DESC, t.type_name;
    -> END;
Query OK, 0 rows affected (0.03 sec)
```

Fig. 89 CREATE PROCEDURE sp_attack_report

```
mysql> CALL sp_attack_report('2026-10-01', '2026-10-15');
+--------------------------+---------+---------+-------+
| type_name                | attacks | blocked | worst |
+--------------------------+---------+---------+-------+
| Exploit Public-Facing App |       5 |       2 |    10 |
| DDoS                     |       4 |       3 |     8 |
| Brute Force              |       3 |       1 |     7 |
| Phishing                 |       3 |       2 |     6 |
| Port Scan                |       3 |       3 |     3 |
| Credential Dumping       |       2 |       0 |     9 |
| Ransomware               |       1 |       0 |    10 |
+--------------------------+---------+---------+-------+
7 rows in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)
```

Fig. 90 sp_attack_report called for 1 to 15 October

sp_report_ioc and sp_whoami (Review 3). sp_report_ioc is the only way an analyst can add an indicator. It sets recorded_from with SYSDATE(), the real clock: NOW() would follow a session's SET timestamp, which any user may change, so an analyst could have backdated when we learned about an indicator (fixed in Review 3, test S12). sp_whoami returns USER() and CURRENT_USER() side by side, which makes the difference between the caller and the definer visible.

```
mysql> CREATE PROCEDURE sp_report_ioc(IN p_type VARCHAR(6), IN p_value VARCHAR(255),
    ->                                IN p_valid_from DATETIME, IN p_actor INT, IN p_malware INT)
    -> BEGIN
    ->   INSERT INTO ioc (ioc_type, ioc_value, actor_id, malware_id, valid_from, recorded_from)
    ->   VALUES (p_type, p_value, p_actor, p_malware, p_valid_from, SYSDATE());
    ->   SELECT ioc_version_id, ioc_type, ioc_value, valid_from, recorded_from
    ->   FROM ioc WHERE ioc_version_id = LAST_INSERT_ID();
    -> END;
Query OK, 0 rows affected (0.01 sec)
```

Fig. 91 CREATE PROCEDURE sp_report_ioc

```
mysql> CREATE PROCEDURE sp_whoami()
    -> BEGIN
    ->   SELECT USER() AS called_by, CURRENT_USER() AS running_with_rights_of;
    -> END;
Query OK, 0 rows affected (0.01 sec)

mysql> CALL sp_whoami();
+----------------+-----------------------+
| called_by      | running_with_rights_of |
+----------------+-----------------------+
| root@localhost | root@localhost         |
+----------------+-----------------------+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)
```

Fig. 92 sp_whoami, and a call by root (both functions return root)

sp_load_log_batch, the batch loader with an exit handler, is shown with the transactions in section 13.

12.4 User-Defined Functions

fn_severity_label(cvss) is DETERMINISTIC and turns a CVSS score into LOW, MEDIUM, HIGH or CRITICAL. fn_asset_risk(asset_id) reads three tables and returns criticality x 10 + open vulnerabilities x 5 + attacks not blocked x 2.

```
mysql> CREATE FUNCTION fn_severity_label(p_cvss DECIMAL(3,1))
    -> RETURNS VARCHAR(8)
    -> DETERMINISTIC NO SQL
    -> BEGIN
    ->   RETURN CASE WHEN p_cvss >= 9 THEN 'CRITICAL'
    ->               WHEN p_cvss >= 7 THEN 'HIGH'
    ->               WHEN p_cvss >= 4 THEN 'MEDIUM'
    ->               ELSE 'LOW' END;
    -> END;
Query OK, 0 rows affected (0.01 sec)
```

Fig. 93 fn_severity_label

```
mysql> SELECT cve_id, cvss, fn_severity_label(cvss) AS label
    -> FROM vulnerability
    -> ORDER BY cvss DESC, cve_id;
+----------------+------+----------+
| cve_id         | cvss | label    |
+----------------+------+----------+
| CVE-2021-44228 | 10.0 | CRITICAL |
| CVE-2024-3400  | 10.0 | CRITICAL |
| CVE-2019-0708  |  9.8 | CRITICAL |
| CVE-2022-22965 |  9.8 | CRITICAL |
| CVE-2023-34362 |  9.8 | CRITICAL |
| CVE-2023-4966  |  9.4 | CRITICAL |
| CVE-2024-6387  |  8.1 | HIGH     |
| CVE-2023-44487 |  7.5 | HIGH     |
| CVE-2023-48795 |  5.9 | MEDIUM   |
+----------------+------+----------+
9 rows in set (0.00 sec)
```

Fig. 94 fn_severity_label used in a SELECT

```
mysql> CREATE FUNCTION fn_asset_risk(p_asset INT)
    -> RETURNS INT
    -> READS SQL DATA
    -> BEGIN
    ->   DECLARE v_crit, v_open, v_hits INT DEFAULT 0;
    ->   SELECT criticality INTO v_crit FROM asset WHERE asset_id = p_asset;
    ->   SELECT COUNT(*) INTO v_open FROM asset_vulnerability
    ->    WHERE asset_id = p_asset AND patched_at IS NULL;
    ->   SELECT COUNT(*) INTO v_hits FROM attack_event
    ->    WHERE asset_id = p_asset AND blocked = 0;
    ->   RETURN v_crit * 10 + v_open * 5 + v_hits * 2;
    -> END;
Query OK, 0 rows affected (0.01 sec)
```

Fig. 95 fn_asset_risk

```
mysql> SELECT hostname, criticality, fn_asset_risk(asset_id) AS risk_score
    -> FROM asset
    -> ORDER BY risk_score DESC, hostname
    -> LIMIT 5;
+----------+-------------+------------+
| hostname | criticality | risk_score |
+----------+-------------+------------+
| web-01   |           5 |         63 |
| db-01    |           5 |         54 |
| vpn-01   |           4 |         49 |
| web-02   |           4 |         49 |
| dns-01   |           4 |         47 |
+----------+-------------+------------+
5 rows in set (0.00 sec)
```

Fig. 96 fn_asset_risk used in a SELECT

12.5 Triggers

trg_attack_auto_incident (AFTER INSERT on attack_event) opens an incident for any attack of severity 8 or more. If the same asset already has an open incident for the same attack type from the last hour, the attack joins it instead. In the demo three attacks are inserted: the first (severity 9) opens incident 8, the second (severity 8, twenty minutes later) is added to it, and the third (severity 3) is ignored.

```
mysql> CREATE TRIGGER trg_attack_auto_incident
    -> AFTER INSERT ON attack_event
    -> FOR EACH ROW
    -> BEGIN
    ->   DECLARE v_inc INT;
    ->   IF NEW.severity >= 8 THEN
    ->
    ->     SET v_inc = (SELECT ie.incident_id
    ->                  FROM incident_event ie
    ->                  JOIN attack_event e ON e.event_id    = ie.event_id
    ->                  JOIN incident i     ON i.incident_id = ie.incident_id
    ->                  WHERE e.asset_id = NEW.asset_id
    ->                    AND e.type_id  = NEW.type_id
    ->                    AND i.status  <> 'CLOSED'
    ->                    AND e.event_time >= NEW.event_time - INTERVAL 1 HOUR
    ->                  ORDER BY i.opened_at DESC
    ->                  LIMIT 1);
    ->     IF v_inc IS NULL THEN
    ->       INSERT INTO incident (title, priority, status, opened_at)
    ->       VALUES (CONCAT((SELECT type_name FROM attack_type WHERE type_id = NEW.type_id),
    ->                      ' on ', (SELECT hostname FROM asset WHERE asset_id = NEW.asset_id),
    ->                      ' from ', NEW.src_ip),
    ->               IF(NEW.severity >= 9, 'CRITICAL', 'HIGH'), 'OPEN', NEW.event_time);
    ->       SET v_inc = LAST_INSERT_ID();
    ->     END IF;
    ->     INSERT INTO incident_event (incident_id, event_id) VALUES (v_inc, NEW.event_id);
    ->   END IF;
    -> END;
Query OK, 0 rows affected (0.01 sec)
```

Fig. 97 trg_attack_auto_incident

```
mysql> SELECT COUNT(*) AS incidents_before FROM incident;
+------------------+
| incidents_before |
+------------------+
|                7 |
+------------------+
1 row in set (0.00 sec)

mysql> INSERT INTO attack_event (event_time, src_ip, src_country, asset_id, type_id, severity, blocked)
    -> VALUES ('2026-10-27 22:00:00', '203.0.113.66', 'US', 3, 3, 9, 0);
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO attack_event (event_time, src_ip, src_country, asset_id, type_id, severity, blocked)
    -> VALUES ('2026-10-27 22:20:00', '203.0.113.66', 'US', 3, 3, 8, 0);
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO attack_event (event_time, src_ip, src_country, asset_id, type_id, severity, blocked)
    -> VALUES ('2026-10-27 22:30:00', '203.0.113.66', 'US', 3, 4, 3, 1);
Query OK, 1 row affected (0.00 sec)

mysql> SELECT i.incident_id, i.title, i.priority, i.status, COUNT(ie.event_id) AS attacks
    -> FROM incident i
    -> JOIN incident_event ie ON ie.incident_id = i.incident_id
    -> WHERE i.incident_id > 7
    -> GROUP BY i.incident_id, i.title, i.priority, i.status;
+-------------+-----------------------------------------------------+----------+--------+---------+
| incident_id | title                                               | priority | status | attacks |
+-------------+-----------------------------------------------------+----------+--------+---------+
|           8 | Exploit Public-Facing App on db-01 from 203.0.113.66 | CRITICAL | OPEN   |       2 |
+-------------+-----------------------------------------------------+----------+--------+---------+
1 row in set (0.00 sec)
```

Fig. 98 trg_attack_auto_incident firing on three new attacks

Four triggers on ioc: trg_ioc_no_overlap (BEFORE INSERT) rejects a new version whose valid period overlaps an open one, trg_ioc_no_delete (BEFORE DELETE) and trg_ioc_only_close (BEFORE UPDATE) keep the history append-only, and trg_ioc_retro_hunt (AFTER INSERT) runs the retro-hunt. Fixed in Review 3: in Review 2 only DELETE was blocked, so an UPDATE could rewrite an indicator or reopen a closed version; trg_ioc_only_close now allows exactly one change, closing an open version with

recorded_to (tests T32, T33). The overlap check also ignored closed versions wrongly the other way round: it still counted them, so a corrected version could never be added (T28). The retro-hunt records the time the search really ran (SYSDATE()).

```
mysql> CREATE TRIGGER trg_ioc_no_overlap
    -> BEFORE INSERT ON ioc
    -> FOR EACH ROW
    -> BEGIN
    ->   IF EXISTS (SELECT 1 FROM ioc
    ->              WHERE ioc_type = NEW.ioc_type
    ->                AND ioc_value = NEW.ioc_value
    ->                AND recorded_to IS NULL
    ->                AND (valid_to IS NULL OR valid_to > NEW.valid_from)
    ->                AND (NEW.valid_to IS NULL OR NEW.valid_to > valid_from)) THEN
    ->     SIGNAL SQLSTATE '45000'
    ->       SET MESSAGE_TEXT = 'An overlapping version of this indicator already exists';
    ->   END IF;
    -> END;
Query OK, 0 rows affected (0.01 sec)
```

Fig. 99 trg_ioc_no_overlap

```
mysql> CREATE TRIGGER trg_ioc_no_delete
    -> BEFORE DELETE ON ioc
    -> FOR EACH ROW
    -> BEGIN
    ->   SIGNAL SQLSTATE '45000'
    ->     SET MESSAGE_TEXT = 'Indicator history is append-only; close it with recorded_to instead';
    -> END;
Query OK, 0 rows affected (0.01 sec)
```

Fig. 100 trg_ioc_no_delete

```
mysql> CREATE TRIGGER trg_ioc_only_close
    -> BEFORE UPDATE ON ioc
    -> FOR EACH ROW
    -> BEGIN
    ->   IF NOT (NEW.ioc_version_id <=> OLD.ioc_version_id AND NEW.ioc_type <=> OLD.ioc_type
    ->           AND NEW.ioc_value <=> OLD.ioc_value AND NEW.actor_id <=> OLD.actor_id
    ->           AND NEW.malware_id <=> OLD.malware_id AND NEW.valid_from <=> OLD.valid_from
    ->           AND NEW.valid_to <=> OLD.valid_to AND NEW.recorded_from <=> OLD.recorded_from
    ->           AND OLD.recorded_to IS NULL AND NEW.recorded_to IS NOT NULL) THEN
    ->     SIGNAL SQLSTATE '45000'
    ->       SET MESSAGE_TEXT = 'Indicator versions cannot be changed; only an open version can be closed';
    ->   END IF;
    -> END;
Query OK, 0 rows affected (0.01 sec)
```

Fig. 101 trg_ioc_only_close

```
mysql> CREATE TRIGGER trg_ioc_retro_hunt
    -> AFTER INSERT ON ioc
    -> FOR EACH ROW
    -> BEGIN
    ->   DECLARE v_run, v_hits, v_assets INT;
    ->   IF NEW.ioc_type = 'ip' THEN
    ->     INSERT INTO retro_hunt_run (ioc_version_id, started_at)
    ->     VALUES (NEW.ioc_version_id, SYSDATE());
    ->     SET v_run = LAST_INSERT_ID();
    ->
    ->
    ->     INSERT INTO retro_hit (run_id, log_id)
    ->     SELECT v_run, log_id
    ->     FROM network_log
    ->     WHERE src_ip = NEW.ioc_value
    ->       AND log_time >= NEW.valid_from
    ->       AND log_time <  NEW.recorded_from
    ->       AND (NEW.valid_to IS NULL OR log_time < NEW.valid_to);
    ->     SET v_hits = ROW_COUNT();
    ->
    ->     IF v_hits > 0 THEN
    ->       SET v_assets = (SELECT COUNT(DISTINCT l.dst_asset_id)
    ->                       FROM retro_hit h JOIN network_log l ON l.log_id = h.log_id
    ->                       WHERE h.run_id = v_run);
    ->       INSERT INTO incident (title, priority, status, opened_at)
    ->       VALUES (CONCAT('Retro-hunt: ', NEW.ioc_value, ' hit ', v_assets,
    ->                      ' assets ', v_hits, ' times'),
    ->               IF(v_assets >= 3, 'CRITICAL', 'HIGH'), 'OPEN', NEW.recorded_from);
    ->       UPDATE retro_hunt_run SET incident_id = LAST_INSERT_ID() WHERE run_id = v_run;
    ->     END IF;
    ->   END IF;
    -> END;
Query OK, 0 rows affected (0.01 sec)
```

Fig. 102 trg_ioc_retro_hunt

In the demonstration, 198.51.100.77 is added as a bad IP that was hostile from 15 September. The trigger finds the 8 old log lines it sent to 3 assets before anyone knew, and opens a CRITICAL incident. The overlapping second version, the DELETE and an UPDATE that tries to change the IP are all refused.

```
mysql> INSERT INTO ioc (ioc_type, ioc_value, valid_from, recorded_from)
    -> VALUES ('ip', '198.51.100.77', '2026-09-15 00:00:00', '2026-10-28 10:00:00');
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM v_retro_hunt_summary WHERE ioc_version_id = LAST_INSERT_ID();
+--------+---------------+---------------------+-------------+--------------+----------------+
| run_id | ioc_version_id | started_at          | incident_id | logs_matched | assets_touched |
+--------+---------------+---------------------+-------------+--------------+----------------+
|      3 |             13 | 2026-10-29 22:31:23 |           9 |            8 |              3 |
+--------+---------------+---------------------+-------------+--------------+----------------+
1 row in set (0.00 sec)

mysql> SELECT incident_id, title, priority, status, opened_at
    -> FROM incident
    -> WHERE title LIKE 'Retro-hunt: 198.51.100.77%';
+-------------+-----------------------------------------------+----------+--------+---------------------+
| incident_id | title                                         | priority | status | opened_at           |
+-------------+-----------------------------------------------+----------+--------+---------------------+
|           9 | Retro-hunt: 198.51.100.77 hit 3 assets 8 times | CRITICAL | OPEN   | 2026-10-28 10:00:00 |
+-------------+-----------------------------------------------+----------+--------+---------------------+
1 row in set (0.00 sec)

mysql> INSERT INTO ioc (ioc_type, ioc_value, valid_from, recorded_from)
    -> VALUES ('ip', '198.51.100.77', '2026-10-01 00:00:00', '2026-10-28 11:00:00');
ERROR 1644 (45000): An overlapping version of this indicator already exists

mysql> DELETE FROM ioc WHERE ioc_value = '198.51.100.77';
ERROR 1644 (45000): Indicator history is append-only; close it with recorded_to instead

mysql> UPDATE ioc SET ioc_value = '198.51.100.78' WHERE ioc_value = '198.51.100.77';
ERROR 1644 (45000): Indicator versions cannot be changed; only an open version can be closed
```

Fig. 103 Retro-hunt demonstration, then the three refused changes

Audit triggers (Review 3). trg_incident_audit_update and trg_incident_audit_delete (AFTER UPDATE / AFTER DELETE on incident) write one row to incident_audit for every change, so it is recorded whether the change came through a view, a procedure or a direct statement. They record USER(), not CURRENT_USER(): inside a trigger CURRENT_USER() is the trigger's definer (root), which would make every row look the same. trg_audit_no_update and trg_audit_no_delete make the trail append-only even for root. No role has INSERT, UPDATE or DELETE on incident_audit, so nobody can add a forged row (in the first Review 3 build the admin still could: fixed, tests A06 and T34). TRUNCATE does not fire triggers, which is why no role except root has the DROP privilege that TRUNCATE needs. The time is SYSDATE(6), which a session cannot move. Gaps in audit_id are normal: an AUTO_INCREMENT value taken by a change that was later rolled back is not reused.

```
mysql> CREATE TRIGGER trg_incident_audit_update
    -> AFTER UPDATE ON incident
    -> FOR EACH ROW
    -> BEGIN
    ->   INSERT INTO incident_audit (incident_id, action, changed_by, changed_at,
    ->                               old_status, new_status, old_priority, new_priority,
    ->                               old_assigned_to, new_assigned_to)
    ->   VALUES (OLD.incident_id, 'UPDATE', USER(),
    ->           SYSDATE(6),
    ->           OLD.status, NEW.status,
    ->           OLD.priority, NEW.priority,
    ->           OLD.assigned_to, NEW.assigned_to);
    -> END;
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE TRIGGER trg_incident_audit_delete
    -> AFTER DELETE ON incident
    -> FOR EACH ROW
    -> BEGIN
    ->   INSERT INTO incident_audit (incident_id, action, changed_by, changed_at,
    ->                               old_status, old_priority, old_assigned_to)
    ->   VALUES (OLD.incident_id, 'DELETE', USER(), SYSDATE(6), OLD.status, OLD.priority, OLD.assigned_to);
    -> END;
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE TRIGGER trg_audit_no_update
    -> BEFORE UPDATE ON incident_audit
    -> FOR EACH ROW
    -> BEGIN
    ->   SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'The audit trail is append-only';
    -> END;
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE TRIGGER trg_audit_no_delete
    -> BEFORE DELETE ON incident_audit
    -> FOR EACH ROW
    -> BEGIN
    ->   SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'The audit trail is append-only';
    -> END;
Query OK, 0 rows affected (0.01 sec)
```

Fig. 104 The four audit triggers

```
mysql> UPDATE incident SET status = 'CONTAINED' WHERE incident_id = 4;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT audit_id, incident_id, action, changed_by, old_status, new_status FROM incident_audit;
+----------+-------------+--------+----------------+---------------+------------+
| audit_id | incident_id | action | changed_by     | old_status    | new_status |
+----------+-------------+--------+----------------+---------------+------------+
|        1 |           4 | UPDATE | root@localhost | INVESTIGATING | CONTAINED  |
+----------+-------------+--------+----------------+---------------+------------+
1 row in set (0.00 sec)

mysql> DELETE FROM incident_audit;
ERROR 1644 (45000): The audit trail is append-only
```

Fig. 105 An UPDATE is recorded; deleting audit rows is refused

12.6 Cursors

sp_escalate_incidents walks through every incident that is not closed with a cursor, and raises its priority when its worst attack demands it: to CRITICAL for severity 10, and from LOW or MEDIUM to HIGH for severity 8. It escalated incident 3 (HIGH to CRITICAL, because of attack 30) and incident 6 (MEDIUM to HIGH).

```
mysql> CREATE PROCEDURE sp_escalate_incidents()
    -> BEGIN
    ->   DECLARE done INT DEFAULT 0;
    ->   DECLARE v_id INT;
    ->   DECLARE v_pri VARCHAR(8);
    ->   DECLARE v_max TINYINT;
    ->   DECLARE cur CURSOR FOR
    ->     SELECT i.incident_id, i.priority, MAX(e.severity)
    ->     FROM incident i
    ->     JOIN incident_event ie ON ie.incident_id = i.incident_id
    ->     JOIN attack_event e    ON e.event_id     = ie.event_id
    ->     WHERE i.status <> 'CLOSED'
    ->     GROUP BY i.incident_id, i.priority
    ->     ORDER BY i.incident_id;
    ->   DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
    ->
    ->   DROP TEMPORARY TABLE IF EXISTS escalation_log;
    ->   CREATE TEMPORARY TABLE escalation_log (
    ->     incident_id INT, old_priority VARCHAR(8), new_priority VARCHAR(8), worst_severity TINYINT);
    ->
    ->   OPEN cur;
    ->   read_loop: LOOP
    ->     FETCH cur INTO v_id, v_pri, v_max;
    ->     IF done THEN LEAVE read_loop; END IF;
    ->     IF v_max >= 10 AND v_pri <> 'CRITICAL' THEN
    ->       UPDATE incident SET priority = 'CRITICAL' WHERE incident_id = v_id;
    ->       INSERT INTO escalation_log VALUES (v_id, v_pri, 'CRITICAL', v_max);
    ->     ELSEIF v_max >= 8 AND v_pri IN ('LOW', 'MEDIUM') THEN
    ->       UPDATE incident SET priority = 'HIGH' WHERE incident_id = v_id;
    ->       INSERT INTO escalation_log VALUES (v_id, v_pri, 'HIGH', v_max);
    ->     END IF;
    ->   END LOOP;
    ->   CLOSE cur;
    ->
    ->   SELECT * FROM escalation_log;
    -> END;
Query OK, 0 rows affected (0.01 sec)
```

Fig. 106 sp_escalate_incidents (cursor)

```
mysql> CALL sp_escalate_incidents();
+-------------+--------------+--------------+----------------+
| incident_id | old_priority | new_priority | worst_severity |
+-------------+--------------+--------------+----------------+
|           3 | HIGH         | CRITICAL     |             10 |
|           6 | MEDIUM       | HIGH         |              8 |
+-------------+--------------+--------------+----------------+
2 rows in set (0.01 sec)

Query OK, 0 rows affected (0.01 sec)
```

Fig. 107 sp_escalate_incidents called

sp_asset_risk_report loops over all assets with a cursor, calls fn_asset_risk for each, puts the score into a temporary table with a band (HIGH 60+, MEDIUM 30+, LOW) and returns it sorted.

```
mysql> CREATE PROCEDURE sp_asset_risk_report()
    -> BEGIN
    ->   DECLARE done INT DEFAULT 0;
    ->   DECLARE v_id INT;
    ->   DECLARE v_host VARCHAR(63);
    ->   DECLARE v_score INT;
    ->   DECLARE cur CURSOR FOR SELECT asset_id, hostname FROM asset ORDER BY asset_id;
    ->   DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
    ->
    ->   DROP TEMPORARY TABLE IF EXISTS risk_report;
    ->   CREATE TEMPORARY TABLE risk_report (hostname VARCHAR(63), risk_score INT, band VARCHAR(8));
    ->
    ->   OPEN cur;
    ->   read_loop: LOOP
    ->     FETCH cur INTO v_id, v_host;
    ->     IF done THEN LEAVE read_loop; END IF;
    ->     SET v_score = fn_asset_risk(v_id);
    ->     INSERT INTO risk_report VALUES (v_host, v_score,
    ->       CASE WHEN v_score >= 60 THEN 'HIGH' WHEN v_score >= 30 THEN 'MEDIUM' ELSE 'LOW' END);
    ->   END LOOP;
    ->   CLOSE cur;
    ->
    ->   SELECT * FROM risk_report ORDER BY risk_score DESC, hostname;
    -> END;
Query OK, 0 rows affected (0.01 sec)
```

Fig. 108 sp_asset_risk_report (cursor)

```
mysql> CALL sp_asset_risk_report();
+-----------+------------+--------+
| hostname  | risk_score | band   |
+-----------+------------+--------+
| web-01    |         63 | HIGH   |
| db-01     |         58 | MEDIUM |
| vpn-01    |         49 | MEDIUM |
| web-02    |         49 | MEDIUM |
| dns-01    |         47 | MEDIUM |
| mail-01   |         39 | MEDIUM |
| ws-fin-01 |         30 | MEDIUM |
| ws-hr-01  |         25 | LOW    |
| ws-dev-01 |         20 | LOW    |
| cam-01    |         12 | LOW    |
+-----------+------------+--------+
10 rows in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)
```

Fig. 109 sp_asset_risk_report called

A cursor works one row at a time. The same escalation rule can be written as one set-based UPDATE with a join to the worst severity per incident, which is faster on large tables; the cursor is kept because it also writes a per-row log of what it changed.

12.7 SQL Security Implementation

Security is layered. From the outside in: an account locks after three wrong passwords; each account gets one role; restricted roles reach data only through views and procedures; the view v_my_incidents limits analysts to their own rows; procedures that run with the definer's rights check who called them; every change to an incident is written to an audit trail that only its triggers can write and no role can change, forge or delete; and the binary log makes mistakes recoverable (section 15). Passwords are not stored in our tables at all: users log in as MySQL accounts, whose passwords MySQL keeps as salted SHA-256 hashes with 5,000 rounds (caching_sha2_password).

Object	sx_admin	sx_analyst	sx_viewer
16 data tables	SELECT, INSERT, UPDATE, DELETE	-	-
incident_audit	SELECT only (written by triggers)	-	-
Schema changes (CREATE, ALTER, DROP, TRIGGER, GRANT)	- (root, the DBA, only)	-	-
v_my_incidents	SELECT, UPDATE	SELECT, UPDATE (own rows only)	-
v_attack_details, v_vulnerability, v_retro_hunt_summary, v_ioc	SELECT	SELECT	-
v_asset_risk	SELECT	SELECT	SELECT
sp_assign_incident	EXECUTE	EXECUTE (claims for himself only)	-
sp_report_ioc, sp_whoami	EXECUTE	EXECUTE	-
sp_search_incidents	EXECUTE	EXECUTE (granted with the injection demonstration)	-
sp_attack_report	EXECUTE	EXECUTE	EXECUTE
Login	Locked for 1 day after 3 wrong passwords	Same	Same
User (default role)	ananya.rao	rahul.mehta	divya.iyer

Task	Evidence
Create 3 or more users/roles	CREATE ROLE and CREATE USER with default roles and lockout (Fig. 110)
GRANT privileges per role matrix	GRANT script (Fig. 110) and SHOW GRANTS output (Fig. 111, 112)
REVOKE demonstration	SHOW GRANTS before and after, and a new divya session each time: the SELECT works before the REVOKE and fails with ERROR 1142 after it (Fig. 113)
Least privilege proof	Viewer, analyst and admin log in and try forbidden operations: ERROR 1142, 1369 and 1644 (Fig. 114, 115, 116, 117); tests S01-S12, V01-V05, A01-A07 (section 16)

Task	Evidence
Secure exposure	Analyst and viewer have no privilege on any base table; they use views and procedures only. The analyst's procedures check the caller (Fig. 116)
Audit and recovery (beyond the template)	incident_audit (Fig. 117), SQL injection (Fig. 118-120), point-in-time recovery (section 15)

```
mysql> CREATE ROLE 'sx_admin', 'sx_analyst', 'sx_viewer';
Query OK, 0 rows affected (0.01 sec)

mysql> GRANT SELECT, EXECUTE, SHOW VIEW ON sentinelx.* TO 'sx_admin';
Query OK, 0 rows affected (0.01 sec)

mysql> GRANT INSERT, UPDATE, DELETE ON sentinelx.asset               TO 'sx_admin';
Query OK, 0 rows affected (0.00 sec)

mysql> GRANT INSERT, UPDATE, DELETE ON sentinelx.vulnerability       TO 'sx_admin';
Query OK, 0 rows affected (0.01 sec)

mysql> GRANT INSERT, UPDATE, DELETE ON sentinelx.asset_vulnerability  TO 'sx_admin';
Query OK, 0 rows affected (0.01 sec)

mysql> GRANT INSERT, UPDATE, DELETE ON sentinelx.network_log         TO 'sx_admin';
Query OK, 0 rows affected (0.00 sec)

mysql> GRANT INSERT, UPDATE, DELETE ON sentinelx.attack_type         TO 'sx_admin';
Query OK, 0 rows affected (0.01 sec)

mysql> GRANT INSERT, UPDATE, DELETE ON sentinelx.mitigation          TO 'sx_admin';
Query OK, 0 rows affected (0.01 sec)

mysql> GRANT INSERT, UPDATE, DELETE ON sentinelx.threat_actor        TO 'sx_admin';
Query OK, 0 rows affected (0.01 sec)

mysql> GRANT INSERT, UPDATE, DELETE ON sentinelx.malware             TO 'sx_admin';
Query OK, 0 rows affected (0.01 sec)

mysql> GRANT INSERT, UPDATE, DELETE ON sentinelx.attack_event        TO 'sx_admin';
Query OK, 0 rows affected (0.01 sec)

mysql> GRANT INSERT, UPDATE, DELETE ON sentinelx.analyst             TO 'sx_admin';
Query OK, 0 rows affected (0.01 sec)

mysql> GRANT INSERT, UPDATE, DELETE ON sentinelx.incident            TO 'sx_admin';
Query OK, 0 rows affected (0.01 sec)

mysql> GRANT INSERT, UPDATE, DELETE ON sentinelx.incident_event      TO 'sx_admin';
Query OK, 0 rows affected (0.01 sec)

mysql> GRANT INSERT, UPDATE, DELETE ON sentinelx.ioc                 TO 'sx_admin';
Query OK, 0 rows affected (0.01 sec)

mysql> GRANT INSERT, UPDATE, DELETE ON sentinelx.event_ioc           TO 'sx_admin';
Query OK, 0 rows affected (0.01 sec)

mysql> GRANT INSERT, UPDATE, DELETE ON sentinelx.retro_hunt_run      TO 'sx_admin';
Query OK, 0 rows affected (0.01 sec)

mysql> GRANT INSERT, UPDATE, DELETE ON sentinelx.retro_hit           TO 'sx_admin';
Query OK, 0 rows affected (0.01 sec)
```

Fig. 110 Roles, grants and users

```
mysql> SHOW GRANTS FOR 'sx_admin';
+-----------------------------------------------------------------------------------------+
| Grants for sx_admin@%                                                                   |
+-----------------------------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `sx_admin`@`%`                                                    |
| GRANT SELECT, EXECUTE, SHOW VIEW ON `sentinelx`.* TO `sx_admin`@`%`                     |
| GRANT INSERT, UPDATE, DELETE ON `sentinelx`.`analyst` TO `sx_admin`@`%`                 |
| GRANT INSERT, UPDATE, DELETE ON `sentinelx`.`asset_vulnerability` TO `sx_admin`@`%`     |
| GRANT INSERT, UPDATE, DELETE ON `sentinelx`.`asset` TO `sx_admin`@`%`                   |
| GRANT INSERT, UPDATE, DELETE ON `sentinelx`.`attack_event` TO `sx_admin`@`%`            |
| GRANT INSERT, UPDATE, DELETE ON `sentinelx`.`attack_type` TO `sx_admin`@`%`             |
| GRANT INSERT, UPDATE, DELETE ON `sentinelx`.`event_ioc` TO `sx_admin`@`%`               |
| GRANT INSERT, UPDATE, DELETE ON `sentinelx`.`incident_event` TO `sx_admin`@`%`          |
| GRANT INSERT, UPDATE, DELETE ON `sentinelx`.`incident` TO `sx_admin`@`%`                |
| GRANT INSERT, UPDATE, DELETE ON `sentinelx`.`ioc` TO `sx_admin`@`%`                     |
| GRANT INSERT, UPDATE, DELETE ON `sentinelx`.`malware` TO `sx_admin`@`%`                 |
| GRANT INSERT, UPDATE, DELETE ON `sentinelx`.`mitigation` TO `sx_admin`@`%`              |
| GRANT INSERT, UPDATE, DELETE ON `sentinelx`.`network_log` TO `sx_admin`@`%`             |
| GRANT INSERT, UPDATE, DELETE ON `sentinelx`.`retro_hit` TO `sx_admin`@`%`               |
| GRANT INSERT, UPDATE, DELETE ON `sentinelx`.`retro_hunt_run` TO `sx_admin`@`%`          |
| GRANT INSERT, UPDATE, DELETE ON `sentinelx`.`threat_actor` TO `sx_admin`@`%`            |
| GRANT INSERT, UPDATE, DELETE ON `sentinelx`.`vulnerability` TO `sx_admin`@`%`           |
+-----------------------------------------------------------------------------------------+
18 rows in set (0.00 sec)

mysql> SHOW GRANTS FOR 'sx_analyst';
+-----------------------------------------------------------------------------------------+
| Grants for sx_analyst@%                                                                 |
+-----------------------------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `sx_analyst`@`%`                                                  |
| GRANT SELECT ON `sentinelx`.`v_asset_risk` TO `sx_analyst`@`%`                          |
| GRANT SELECT ON `sentinelx`.`v_attack_details` TO `sx_analyst`@`%`                      |
| GRANT SELECT ON `sentinelx`.`v_ioc` TO `sx_analyst`@`%`                                 |
| GRANT SELECT, UPDATE ON `sentinelx`.`v_my_incidents` TO `sx_analyst`@`%`                |
| GRANT SELECT ON `sentinelx`.`v_retro_hunt_summary` TO `sx_analyst`@`%`                  |
| GRANT SELECT ON `sentinelx`.`v_vulnerability` TO `sx_analyst`@`%`                       |
| GRANT EXECUTE ON PROCEDURE `sentinelx`.`sp_assign_incident` TO `sx_analyst`@`%`         |
| GRANT EXECUTE ON PROCEDURE `sentinelx`.`sp_attack_report` TO `sx_analyst`@`%`           |
| GRANT EXECUTE ON PROCEDURE `sentinelx`.`sp_report_ioc` TO `sx_analyst`@`%`              |
| GRANT EXECUTE ON PROCEDURE `sentinelx`.`sp_whoami` TO `sx_analyst`@`%`                  |
+-----------------------------------------------------------------------------------------+
11 rows in set (0.00 sec)

mysql> SHOW GRANTS FOR 'sx_viewer';
+-----------------------------------------------------------------------------------------+
| Grants for sx_viewer@%                                                                  |
+-----------------------------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `sx_viewer`@`%`                                                   |
| GRANT SELECT ON `sentinelx`.`v_asset_risk` TO `sx_viewer`@`%`                           |
| GRANT EXECUTE ON PROCEDURE `sentinelx`.`sp_attack_report` TO `sx_viewer`@`%`            |
+-----------------------------------------------------------------------------------------+
3 rows in set (0.00 sec)
```

Fig. 111 SHOW GRANTS for the three roles

```
mysql> SHOW GRANTS FOR 'rahul.mehta'@'localhost';
+-----------------------------------------------------+
| Grants for rahul.mehta@localhost                    |
+-----------------------------------------------------+
| GRANT USAGE ON *.* TO `rahul.mehta`@`localhost`     |
| GRANT `sx_analyst`@`%` TO `rahul.mehta`@`localhost` |
+-----------------------------------------------------+
2 rows in set (0.00 sec)

mysql> SHOW GRANTS FOR 'rahul.mehta'@'localhost' USING 'sx_analyst';
+-----------------------------------------------------------------------------------------------+
| Grants for rahul.mehta@localhost                                                              |
+-----------------------------------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `rahul.mehta`@`localhost`                                               |
| GRANT SELECT ON `sentinelx`.`v_asset_risk` TO `rahul.mehta`@`localhost`                       |
| GRANT SELECT ON `sentinelx`.`v_attack_details` TO `rahul.mehta`@`localhost`                   |
| GRANT SELECT ON `sentinelx`.`v_ioc` TO `rahul.mehta`@`localhost`                              |
| GRANT SELECT, UPDATE ON `sentinelx`.`v_my_incidents` TO `rahul.mehta`@`localhost`             |
| GRANT SELECT ON `sentinelx`.`v_retro_hunt_summary` TO `rahul.mehta`@`localhost`               |
| GRANT SELECT ON `sentinelx`.`v_vulnerability` TO `rahul.mehta`@`localhost`                    |
| GRANT EXECUTE ON PROCEDURE `sentinelx`.`sp_assign_incident` TO `rahul.mehta`@`localhost`      |
| GRANT EXECUTE ON PROCEDURE `sentinelx`.`sp_attack_report` TO `rahul.mehta`@`localhost`        |
| GRANT EXECUTE ON PROCEDURE `sentinelx`.`sp_report_ioc` TO `rahul.mehta`@`localhost`           |
| GRANT EXECUTE ON PROCEDURE `sentinelx`.`sp_whoami` TO `rahul.mehta`@`localhost`               |
| GRANT `sx_analyst`@`%` TO `rahul.mehta`@`localhost`                                           |
+-----------------------------------------------------------------------------------------------+
12 rows in set (0.00 sec)

mysql> SHOW GRANTS FOR 'divya.iyer'@'localhost' USING 'sx_viewer';
+-----------------------------------------------------------------------------------------------+
| Grants for divya.iyer@localhost                                                               |
+-----------------------------------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `divya.iyer`@`localhost`                                                |
| GRANT SELECT ON `sentinelx`.`v_asset_risk` TO `divya.iyer`@`localhost`                        |
| GRANT EXECUTE ON PROCEDURE `sentinelx`.`sp_attack_report` TO `divya.iyer`@`localhost`         |
| GRANT `sx_viewer`@`%` TO `divya.iyer`@`localhost`                                             |
+-----------------------------------------------------------------------------------------------+
4 rows in set (0.00 sec)
```

Fig. 112 A user's grants show only the role; USING expands it

```
-- root
mysql> GRANT SELECT ON sentinelx.network_log TO 'sx_viewer';
Query OK, 0 rows affected (0.01 sec)

mysql> SHOW GRANTS FOR 'sx_viewer';
+-------------------------------------------------------------------------------+
| Grants for sx_viewer@%                                                        |
+-------------------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `sx_viewer`@`%`                                         |
| GRANT SELECT ON `sentinelx`.`network_log` TO `sx_viewer`@`%`                  |
| GRANT SELECT ON `sentinelx`.`v_asset_risk` TO `sx_viewer`@`%`                 |
| GRANT EXECUTE ON PROCEDURE `sentinelx`.`sp_attack_report` TO `sx_viewer`@`%` |
+-------------------------------------------------------------------------------+
4 rows in set (0.00 sec)

-- divya, new session
mysql> SELECT COUNT(*) AS log_lines FROM network_log;
+-----------+
| log_lines |
+-----------+
|     50021 |
+-----------+
1 row in set (0.01 sec)

-- root
mysql> REVOKE SELECT ON sentinelx.network_log FROM 'sx_viewer';
Query OK, 0 rows affected (0.00 sec)

mysql> SHOW GRANTS FOR 'sx_viewer';
+-------------------------------------------------------------------------------+
| Grants for sx_viewer@%                                                        |
+-------------------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `sx_viewer`@`%`                                         |
| GRANT SELECT ON `sentinelx`.`v_asset_risk` TO `sx_viewer`@`%`                 |
| GRANT EXECUTE ON PROCEDURE `sentinelx`.`sp_attack_report` TO `sx_viewer`@`%` |
+-------------------------------------------------------------------------------+
3 rows in set (0.00 sec)

-- divya, new session
mysql> SELECT COUNT(*) AS log_lines FROM network_log;
ERROR 1142 (42000): SELECT command denied to user 'divya.iyer'@'localhost' for table 'network_log'
```

Fig. 113 GRANT, then REVOKE: what the viewer can do before and after

Logged in as the viewer, divya.iyer: reports work, everything else is refused.

```
mysql> SELECT CURRENT_USER() AS logged_in_as, CURRENT_ROLE() AS active_role;
+----------------------+-----------------+
| logged_in_as         | active_role     |
+----------------------+-----------------+
| divya.iyer@localhost | `sx_viewer`@`%` |
+----------------------+-----------------+
1 row in set (0.00 sec)

mysql> SELECT hostname, criticality, attacks, not_blocked, open_vulns
    -> FROM v_asset_risk
    -> ORDER BY not_blocked DESC
    -> LIMIT 3;
+----------+-------------+---------+-------------+------------+
| hostname | criticality | attacks | not_blocked | open_vulns |
+----------+-------------+---------+-------------+------------+
| web-01   |           5 |       7 |           4 |          1 |
| db-01    |           5 |       7 |           4 |          0 |
| web-02   |           4 |       5 |           2 |          1 |
+----------+-------------+---------+-------------+------------+
3 rows in set (0.00 sec)

mysql> CALL sp_attack_report('2026-10-20', '2026-10-28');
+--------------------------+---------+---------+-------+
| type_name                | attacks | blocked | worst |
+--------------------------+---------+---------+-------+
| Exploit Public-Facing App |       4 |       1 |     9 |
| Brute Force              |       1 |       1 |     4 |
| DDoS                     |       1 |       1 |     7 |
| Phishing                 |       1 |       1 |     5 |
| Port Scan                |       1 |       1 |     3 |
| Ransomware               |       1 |       0 |    10 |
+--------------------------+---------+---------+-------+
6 rows in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

mysql> SELECT * FROM attack_event LIMIT 1;
ERROR 1142 (42000): SELECT command denied to user 'divya.iyer'@'localhost' for table 'attack_event'

mysql> DELETE FROM attack_event WHERE event_id = 1;
ERROR 1142 (42000): DELETE command denied to user 'divya.iyer'@'localhost' for table 'attack_event'

mysql> UPDATE v_asset_risk SET criticality = 1 WHERE asset_id = 1;
ERROR 1142 (42000): UPDATE command denied to user 'divya.iyer'@'localhost' for table 'v_asset_risk'
```

Fig. 114 Viewer: allowed and forbidden operations

Logged in as the analyst, rahul.mehta. He sees only his 3 incidents, can update their status through the view, cannot reassign one to another analyst (CHECK OPTION), and cannot read the incident table or drop tables:

```
mysql> SELECT CURRENT_USER() AS logged_in_as, CURRENT_ROLE() AS active_role;
+-----------------------+-----------------+
| logged_in_as          | active_role     |
+-----------------------+-----------------+
| rahul.mehta@localhost | `sx_analyst`@`%` |
+-----------------------+-----------------+
1 row in set (0.00 sec)

mysql> SELECT incident_id, title, priority, status FROM v_my_incidents;
+-------------+-----------------------------------------+----------+---------------+
| incident_id | title                                   | priority | status        |
+-------------+-----------------------------------------+----------+---------------+
|           1 | Brute force on web-01 from 198.51.100.23 | HIGH     | CLOSED        |
|           3 | Iron Kestrel intrusion on db-01          | CRITICAL | CONTAINED     |
|           7 | Brute force on web-01 from 198.51.100.45 | HIGH     | INVESTIGATING |
+-------------+-----------------------------------------+----------+---------------+
3 rows in set (0.00 sec)

mysql> UPDATE v_my_incidents SET status = 'CONTAINED' WHERE incident_id = 7;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT incident_id, status FROM v_my_incidents WHERE incident_id = 7;
+-------------+-----------+
| incident_id | status    |
+-------------+-----------+
|           7 | CONTAINED |
+-------------+-----------+
1 row in set (0.00 sec)

mysql> UPDATE v_my_incidents SET assigned_to = 3 WHERE incident_id = 7;
ERROR 1369 (HY000): CHECK OPTION failed 'sentinelx.v_my_incidents'

mysql> SELECT * FROM incident LIMIT 1;
ERROR 1142 (42000): SELECT command denied to user 'rahul.mehta'@'localhost' for table 'incident'

mysql> SELECT * FROM attack_event LIMIT 1;
ERROR 1142 (42000): SELECT command denied to user 'rahul.mehta'@'localhost' for table 'attack_event'

mysql> DROP TABLE asset;
ERROR 1142 (42000): DROP command denied to user 'rahul.mehta'@'localhost' for table 'asset'
```

Fig. 115 Analyst: own incidents only; tables and DDL refused

The privilege-escalation fix: sp_whoami shows that rahul calls it but it runs with root's rights, so sp_assign_incident must check the caller itself. Giving his incident away and taking a colleague's are refused; claiming an unassigned incident works. Writing to ioc directly is refused, but reporting a compromised internal host through sp_report_ioc works, and the retro-hunt trigger finds its 24 old log lines on 9 assets.

```
mysql> CALL sp_whoami();
+----------------------+------------------------+
| called_by            | running_with_rights_of |
+----------------------+------------------------+
| rahul.mehta@localhost | root@localhost         |
+----------------------+------------------------+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

mysql> CALL sp_assign_incident(7, 3);
ERROR 1644 (45000): Analysts can only claim unassigned incidents for themselves

mysql> CALL sp_assign_incident(6, 2);
ERROR 1644 (45000): Analysts can only claim unassigned incidents for themselves

mysql> CALL sp_assign_incident(8, 2);
+-------------+---------------+-------------+
| incident_id | status        | assigned_to |
+-------------+---------------+-------------+
|           8 | INVESTIGATING |           2 |
+-------------+---------------+-------------+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

mysql> INSERT INTO ioc (ioc_type, ioc_value, valid_from) VALUES ('ip', '172.16.7.8', '2026-10-01');
ERROR 1142 (42000): INSERT command denied to user 'rahul.mehta'@'localhost' for table 'ioc'

mysql> CALL sp_report_ioc('ip', '172.16.7.8', '2026-10-01', NULL, NULL);
+----------------+----------+------------+---------------------+---------------------+
| ioc_version_id | ioc_type | ioc_value  | valid_from          | recorded_from       |
+----------------+----------+------------+---------------------+---------------------+
|             15 | ip       | 172.16.7.8 | 2026-10-01 00:00:00 | 2026-10-29 22:31:26 |
+----------------+----------+------------+---------------------+---------------------+
1 row in set (0.01 sec)

Query OK, 0 rows affected (0.01 sec)

mysql> SELECT ioc_version_id, logs_matched, assets_touched, incident_id
    -> FROM v_retro_hunt_summary
    -> WHERE ioc_version_id = (SELECT MAX(ioc_version_id) FROM v_ioc);
+----------------+--------------+----------------+-------------+
| ioc_version_id | logs_matched | assets_touched | incident_id |
+----------------+--------------+----------------+-------------+
|             15 |           24 |              9 |          10 |
+----------------+--------------+----------------+-------------+
1 row in set (0.00 sec)
```

Fig. 116 Analyst: the procedure checks its caller; IOCs only through a procedure

Logged in as the admin, ananya.rao: the audit trail names rahul for the changes he made through the view and through the procedure, and the admin can neither delete it, add a forged row, truncate it nor drop its trigger: each attempt is refused for lack of privilege (ERROR 1142).

```
mysql> SELECT CURRENT_USER() AS logged_in_as, CURRENT_ROLE() AS active_role;
+----------------------+----------------+
| logged_in_as         | active_role    |
+----------------------+----------------+
| ananya.rao@localhost | `sx_admin`@`%` |
+----------------------+----------------+
1 row in set (0.00 sec)

mysql> SELECT COUNT(*) AS incidents FROM incident;
+-----------+
| incidents |
+-----------+
|        10 |
+-----------+
1 row in set (0.00 sec)

mysql> SELECT audit_id, incident_id, changed_by, old_status, new_status,
    ->        old_assigned_to, new_assigned_to
    -> FROM incident_audit
    -> ORDER BY audit_id;
+----------+-------------+----------------------+---------------+---------------+-----------------+-----------------+
| audit_id | incident_id | changed_by           | old_status    | new_status    | old_assigned_to | new_assigned_to |
+----------+-------------+----------------------+---------------+---------------+-----------------+-----------------+
|        1 |           4 | root@localhost       | INVESTIGATING | CONTAINED     |               4 |               4 |
|        2 |           3 | root@localhost       | CONTAINED     | CONTAINED     |               2 |               2 |
|        3 |           6 | root@localhost       | OPEN          | OPEN          |               3 |               3 |
|        5 |           7 | rahul.mehta@localhost | INVESTIGATING | CONTAINED     |               2 |               2 |
|        6 |           8 | rahul.mehta@localhost | OPEN          | INVESTIGATING |            NULL |               2 |
+----------+-------------+----------------------+---------------+---------------+-----------------+-----------------+
5 rows in set (0.00 sec)

mysql> DELETE FROM incident_audit WHERE audit_id = 1;
ERROR 1142 (42000): DELETE command denied to user 'ananya.rao'@'localhost' for table 'incident_audit'

mysql> INSERT INTO incident_audit (incident_id, action, changed_by) VALUES (7, 'UPDATE', 'rahul.mehta@localhost');
ERROR 1142 (42000): INSERT command denied to user 'ananya.rao'@'localhost' for table 'incident_audit'

mysql> TRUNCATE TABLE incident_audit;
ERROR 1142 (42000): DROP command denied to user 'ananya.rao'@'localhost' for table 'incident_audit'

mysql> DROP TRIGGER trg_audit_no_delete;
ERROR 1142 (42000): TRIGGER command denied to user 'ananya.rao'@'localhost' for table 'incident_audit'
```

Fig. 117 Admin: the audit trail, and attempts to erase it

SQL injection: why GRANT alone does not protect

Two procedures let an analyst search his own incidents by title. The unsafe one pastes the search text into the SQL with CONCAT; the safe one passes it as a parameter (?). Both run with the definer's rights.

```
mysql> CREATE PROCEDURE sp_search_incidents_unsafe(IN p_text VARCHAR(100))
    -> BEGIN
    ->   DECLARE v_me INT;
    ->   SELECT analyst_id INTO v_me FROM analyst
    ->    WHERE email = CONCAT(SUBSTRING_INDEX(USER(), '@', 1), '@sentinelx.in');
    ->   SET @sql = CONCAT('SELECT incident_id, title, priority, status, assigned_to FROM incident ',
    ->                     'WHERE assigned_to = ', COALESCE(v_me, -1),
    ->                     ' AND title LIKE ''%', p_text, '%''');
    ->   PREPARE st FROM @sql;
    ->   EXECUTE st;
    ->   DEALLOCATE PREPARE st;
    -> END;
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE PROCEDURE sp_search_incidents(IN p_text VARCHAR(100))
    -> BEGIN
    ->
    ->   SET @me = NULL;
    ->   SELECT analyst_id INTO @me FROM analyst
    ->    WHERE email = CONCAT(SUBSTRING_INDEX(USER(), '@', 1), '@sentinelx.in');
    ->   IF @me IS NULL THEN
    ->     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Caller is not a registered analyst';
    ->   END IF;
    ->   SET @text = p_text;
    ->   PREPARE st FROM 'SELECT incident_id, title, priority, status, assigned_to FROM incident
    ->                    WHERE assigned_to = ? AND title LIKE CONCAT(''%'', ?, ''%'')';
    ->   EXECUTE st USING @me, @text;
    ->   DEALLOCATE PREPARE st;
    -> END;
Query OK, 0 rows affected (0.01 sec)
```

Fig. 118 The unsafe (CONCAT) and the safe (parameterised) search procedure

Rahul only has EXECUTE on the procedure. The text %' OR 1=1 -- turns the WHERE clause into "(... AND title LIKE '%%') OR 1=1", so every incident comes back, read with root's rights; a UNION returns every analyst's email from a table he cannot read. Stacked statements are refused, because PREPARE accepts one statement only. The parameterised procedure treats the same text as data and returns nothing. It also takes the caller's identity only from the analyst table: it clears the session variable it uses first and refuses a caller it cannot match, so setting @me by hand does not help (fixed in Review 3, test T35).

```
mysql> CALL sp_search_incidents_unsafe('Brute');
+-------------+----------------------------------------+----------+-----------+-------------+
| incident_id | title                                  | priority | status    | assigned_to |
+-------------+----------------------------------------+----------+-----------+-------------+
|           1 | Brute force on web-01 from 198.51.100.23 | HIGH     | CLOSED    |           2 |
|           7 | Brute force on web-01 from 198.51.100.45 | HIGH     | CONTAINED |           2 |
+-------------+----------------------------------------+----------+-----------+-------------+
2 rows in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

mysql> CALL sp_search_incidents_unsafe('%'' OR 1=1 -- ');
+-------------+-----------------------------------------------------------+----------+--------------+-------------+
| incident_id | title                                                     | priority | status       | assigned_to |
+-------------+-----------------------------------------------------------+----------+--------------+-------------+
|           1 | Brute force on web-01 from 198.51.100.23                  | HIGH     | CLOSED       |           2 |
|           2 | Citrix Bleed exploit on mail-01                           | CRITICAL | CLOSED       |           3 |
|           3 | Iron Kestrel intrusion on db-01                           | CRITICAL | CONTAINED    |           2 |
|           4 | Retro-hunt: 198.51.100.45 hit 2 assets 4 times            | CRITICAL | CONTAINED    |           4 |
|           5 | Retro-hunt: 203.0.113.11 hit 2 assets 4 times             | CRITICAL | INVESTIGATING |           6 |
|           6 | DDoS waves on web servers                                 | HIGH     | OPEN         |           3 |
|           7 | Brute force on web-01 from 198.51.100.45                  | HIGH     | CONTAINED    |           2 |
|           8 | Exploit Public-Facing App on db-01 from 203.0.113.66      | CRITICAL | INVESTIGATING |           2 |
|           9 | Retro-hunt: 198.51.100.77 hit 3 assets 8 times            | CRITICAL | OPEN         |        NULL |
|          10 | Retro-hunt: 172.16.7.8 hit 9 assets 24 times              | CRITICAL | OPEN         |        NULL |
+-------------+-----------------------------------------------------------+----------+--------------+-------------+
10 rows in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

mysql> CALL sp_search_incidents_unsafe('%'' UNION SELECT analyst_id, full_name, email, role, NULL FROM analyst -- ');
+-------------+-----------------------------------------------------------+-------------------------+--------------+-------------+
| incident_id | title                                                     | priority                | status       | assigned_to |
+-------------+-----------------------------------------------------------+-------------------------+--------------+-------------+
|           1 | Brute force on web-01 from 198.51.100.23                  | HIGH                    | CLOSED       |           2 |
|           3 | Iron Kestrel intrusion on db-01                           | CRITICAL                | CONTAINED    |           2 |
|           7 | Brute force on web-01 from 198.51.100.45                  | HIGH                    | CONTAINED    |           2 |
|           8 | Exploit Public-Facing App on db-01 from 203.0.113.66      | CRITICAL                | INVESTIGATING |           2 |
|           1 | Ananya Rao                                                | ananya.rao@sentinelx.in | admin        |        NULL |
|           2 | Rahul Mehta                                               | rahul.mehta@sentinelx.in | analyst      |        NULL |
|           3 | Priya Nair                                                | priya.nair@sentinelx.in | analyst      |        NULL |
|           4 | Karan Shah                                                | karan.shah@sentinelx.in | analyst      |        NULL |
|           5 | Divya Iyer                                                | divya.iyer@sentinelx.in | viewer       |        NULL |
|           6 | Arjun Das                                                 | arjun.das@sentinelx.in  | analyst      |        NULL |
+-------------+-----------------------------------------------------------+-------------------------+--------------+-------------+
10 rows in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)
```

Fig. 119 Injection through the unsafe procedure

```
mysql> CALL sp_search_incidents_unsafe('x''; DROP TABLE asset; -- ');
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL
  server version for the right syntax to use near 'DROP TABLE asset; -- %''

mysql> CALL sp_search_incidents('Brute');
+-------------+----------------------------------------+----------+-----------+-------------+
| incident_id | title                                  | priority | status    | assigned_to |
+-------------+----------------------------------------+----------+-----------+-------------+
|           1 | Brute force on web-01 from 198.51.100.23 | HIGH     | CLOSED    |           2 |
|           7 | Brute force on web-01 from 198.51.100.45 | HIGH     | CONTAINED |           2 |
+-------------+----------------------------------------+----------+-----------+-------------+
2 rows in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

mysql> CALL sp_search_incidents('%'' OR 1=1 -- ');
Empty set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)
```

Fig. 120 Stacked statement refused; the safe procedure is not injectable

```
mysql> DROP PROCEDURE sp_search_incidents_unsafe;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT routine_name, security_type FROM information_schema.routines
    -> WHERE routine_schema = 'sentinelx' AND routine_name LIKE 'sp_search%';
+---------------------+---------------+
| ROUTINE_NAME        | SECURITY_TYPE |
+---------------------+---------------+
| sp_search_incidents | DEFINER       |
+---------------------+---------------+
1 row in set (0.00 sec)
```

Fig. 121 The vulnerable procedure is dropped after the demonstration

Defences, in order of importance: never build SQL from user text (use parameters or static SQL); give code only the rights it needs (an INVOKER procedure, or a low-privilege definer instead of root); validate input. GRANT cannot help, because the injected SQL runs with whatever rights the vulnerable code has. Here that was root, so the same hole could have read any table on the server, including the password hashes in mysql.user; a definer account limited to this database would have contained it (section 19).

13. Transactions and Concurrency

SAVEPOINT and COMMIT. A new indicator is inserted and a savepoint is set. A wrong priority change follows, is rolled back to the savepoint, and the transaction is committed. The indicator is kept and the priority change is not.

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> INSERT INTO ioc (ioc_type, ioc_value, actor_id, valid_from, recorded_from)
    -> VALUES ('domain', 'kestrel-drop.example', 4, '2026-10-20 00:00:00', '2026-10-27 12:00:00');
Query OK, 1 row affected (0.00 sec)

mysql> SAVEPOINT after_ioc;
Query OK, 0 rows affected (0.00 sec)

mysql> UPDATE incident SET priority = 'LOW' WHERE incident_id = 3;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT incident_id, priority FROM incident WHERE incident_id = 3;
+-------------+----------+
| incident_id | priority |
+-------------+----------+
|           3 | LOW      |
+-------------+----------+
1 row in set (0.00 sec)

mysql> ROLLBACK TO SAVEPOINT after_ioc;
Query OK, 0 rows affected (0.00 sec)

mysql> COMMIT;
Query OK, 0 rows affected (0.08 sec)

mysql> SELECT ioc_version_id, ioc_type, ioc_value FROM ioc WHERE ioc_value = 'kestrel-drop.example';
+----------------+----------+----------------------+
| ioc_version_id | ioc_type | ioc_value            |
+----------------+----------+----------------------+
|             14 | domain   | kestrel-drop.example |
+----------------+----------+----------------------+
1 row in set (0.00 sec)

mysql> SELECT incident_id, priority FROM incident WHERE incident_id = 3;
+-------------+----------+
| incident_id | priority |
+-------------+----------+
|           3 | CRITICAL |
+-------------+----------+
1 row in set (0.00 sec)
```

Fig. 122 SAVEPOINT, ROLLBACK TO SAVEPOINT and COMMIT

ROLLBACK. The second line of the batch points to an asset that does not exist. The script then calls ROLLBACK, so the log count is unchanged (50,021 before and after).

```
mysql> SELECT COUNT(*) AS logs_before FROM network_log;
+-------------+
| logs_before |
+-------------+
|       50021 |
+-------------+
1 row in set (0.00 sec)

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> INSERT INTO network_log (log_time, src_ip, dst_asset_id, dst_port, protocol, action)
    -> VALUES ('2026-10-28 09:00:00', '203.0.113.5', 1, 443, 'TCP', 'allowed');
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO network_log (log_time, src_ip, dst_asset_id, dst_port, protocol, action)
    -> VALUES ('2026-10-28 09:00:01', '203.0.113.5', 99, 443, 'TCP', 'allowed');
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails
  (`sentinelx`.`network_log`, CONSTRAINT `network_log_ibfk_1` FOREIGN KEY (`dst_asset_id`) REFERENCES
  `asset` (`asset_id`))

mysql> ROLLBACK;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT COUNT(*) AS logs_after FROM network_log;
+------------+
| logs_after |
+------------+
|      50021 |
+------------+
1 row in set (0.01 sec)
```

Fig. 123 A failed batch, rolled back explicitly

A failed statement does not roll back the transaction (found in Review 3). MySQL undoes only the statement that failed. If the script had said COMMIT instead of ROLLBACK, the good first line would have been kept and the batch would be half loaded. The Review 2 demonstration only worked because the script rolled back by hand.

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> INSERT INTO network_log (log_time, src_ip, dst_asset_id, dst_port, protocol, action)
    -> VALUES ('2026-10-28 09:05:00', '203.0.113.6', 1, 443, 'TCP', 'allowed');
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO network_log (log_time, src_ip, dst_asset_id, dst_port, protocol, action)
    -> VALUES ('2026-10-28 09:05:01', '203.0.113.6', 99, 443, 'TCP', 'allowed');
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails
  (`sentinelx`.`network_log`, CONSTRAINT `network_log_ibfk_1` FOREIGN KEY (`dst_asset_id`) REFERENCES
  `asset` (`asset_id`))

mysql> COMMIT;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT COUNT(*) AS kept_after_commit FROM network_log WHERE src_ip = '203.0.113.6';
+-------------------+
| kept_after_commit |
+-------------------+
|                 1 |
+-------------------+
1 row in set (0.00 sec)

mysql> DELETE FROM network_log WHERE src_ip = '203.0.113.6';
Query OK, 1 row affected (0.01 sec)
```

Fig. 124 One INSERT fails, COMMIT keeps the other: half a batch

The loader therefore handles it itself: sp_load_log_batch declares an EXIT HANDLER FOR SQLEXCEPTION that rolls back and re-raises the error. A bad batch leaves no rows; a good batch commits all three.

```
mysql> CREATE PROCEDURE sp_load_log_batch(IN p_last_asset INT)
    -> BEGIN
    ->   DECLARE EXIT HANDLER FOR SQLEXCEPTION
    ->   BEGIN
    ->     ROLLBACK;
    ->     RESIGNAL;
    ->   END;
    ->
    ->   START TRANSACTION;
    ->   INSERT INTO network_log (log_time, src_ip, dst_asset_id, dst_port, protocol, action)
    ->   VALUES ('2026-10-28 09:10:00', '203.0.113.7', 1, 22, 'TCP', 'blocked'),
    ->          ('2026-10-28 09:10:01', '203.0.113.7', 2, 22, 'TCP', 'blocked');
    ->   INSERT INTO network_log (log_time, src_ip, dst_asset_id, dst_port, protocol, action)
    ->   VALUES ('2026-10-28 09:10:02', '203.0.113.7', p_last_asset, 22, 'TCP', 'blocked');
    ->   COMMIT;
    -> END;
Query OK, 0 rows affected (0.01 sec)

mysql> CALL sp_load_log_batch(99);
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails
  (`sentinelx`.`network_log`, CONSTRAINT `network_log_ibfk_1` FOREIGN KEY (`dst_asset_id`) REFERENCES
  `asset` (`asset_id`))

mysql> SELECT COUNT(*) AS rows_after_failed_batch FROM network_log WHERE src_ip = '203.0.113.7';
+-------------------------+
| rows_after_failed_batch |
+-------------------------+
|                       0 |
+-------------------------+
1 row in set (0.00 sec)

mysql> CALL sp_load_log_batch(3);
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT COUNT(*) AS rows_after_good_batch FROM network_log WHERE src_ip = '203.0.113.7';
+------------------------+
| rows_after_good_batch |
+------------------------+
|                      3 |
+------------------------+
1 row in set (0.00 sec)

mysql> DELETE FROM network_log WHERE src_ip = '203.0.113.7';
Query OK, 3 rows affected (0.01 sec)
```

Fig. 125 sp_load_log_batch: the handler rolls back the whole batch

13.1 Concurrency

Each scenario uses two mysql sessions at the same time: terminal A (root) and terminal B (ananya.rao, the admin). In the rehearsal the sessions were timed with DO SLEEP() so that B's statements land between A's; in the live demonstration they are typed step by step (demo/06_concurrency.md).

Lost update. Both sessions read incident 6 as MEDIUM and each raises it "one level" to HIGH. Two escalations happened, but the result is HIGH, not CRITICAL: B's escalation was lost. Its UPDATE matched the row but changed nothing (Changed: 0), because A had already written HIGH, and the audit trail shows the same: old_priority HIGH, new_priority HIGH.

```
-- Terminal A (root)
mysql> START TRANSACTION;
Query OK, 0 rows affected

mysql> SELECT priority, SYSDATE(6) AS a_read_at FROM incident WHERE incident_id = 6;
+----------+----------------------------+
| priority | a_read_at                  |
+----------+----------------------------+
| MEDIUM   | 2026-10-29 22:33:20.538102 |
+----------+----------------------------+
1 row in set

mysql> DO SLEEP(2);
Query OK, 0 rows affected

mysql> UPDATE incident SET priority = 'HIGH' WHERE incident_id = 6;
Query OK, 1 row affected
Rows matched: 1  Changed: 1  Warnings: 0

mysql> COMMIT;
Query OK, 0 rows affected

-- Terminal B (ananya.rao)
mysql> START TRANSACTION;
Query OK, 0 rows affected

mysql> SELECT priority, SYSDATE(6) AS b_read_at FROM incident WHERE incident_id = 6;
+----------+----------------------------+
| priority | b_read_at                  |
+----------+----------------------------+
| MEDIUM   | 2026-10-29 22:33:21.640291 |
+----------+----------------------------+
1 row in set

mysql> DO SLEEP(2);
Query OK, 0 rows affected

mysql> UPDATE incident SET priority = 'HIGH' WHERE incident_id = 6;
Query OK, 0 rows affected
Rows matched: 1  Changed: 0  Warnings: 0

mysql> COMMIT;
Query OK, 0 rows affected

mysql> SELECT priority AS final_priority FROM incident WHERE incident_id = 6;
+----------------+
| final_priority |
+----------------+
| HIGH           |
+----------------+
1 row in set

mysql> SELECT SUBSTRING_INDEX(changed_by, '@', 1) AS who, old_priority, new_priority
    -> FROM incident_audit WHERE incident_id = 6 ORDER BY audit_id DESC LIMIT 2;
+------------+--------------+--------------+
| who        | old_priority | new_priority |
+------------+--------------+--------------+
| ananya.rao | HIGH         | HIGH         |
| root       | MEDIUM       | HIGH         |
+------------+--------------+--------------+
2 rows in set
```

Fig. 126 Lost update: both sessions write HIGH

Who	old_priority	new_priority
root	MEDIUM	HIGH
ananya.rao	HIGH	HIGH

The fix: lock the row when you read it. With SELECT ... FOR UPDATE, B waits until A commits, then reads HIGH and sets CRITICAL. Other fixes are optimistic versioning (a version column checked in the UPDATE's WHERE clause) or an atomic UPDATE that computes the new value from the current one.

```
-- Terminal A (root)
mysql> START TRANSACTION;
Query OK, 0 rows affected

mysql> SELECT priority, SYSDATE(6) AS a_locked_at FROM incident WHERE incident_id = 6 FOR UPDATE;
+----------+----------------------------+
| priority | a_locked_at                |
+----------+----------------------------+
| MEDIUM   | 2026-10-29 22:33:23.935361 |
+----------+----------------------------+
1 row in set

mysql> DO SLEEP(3);
Query OK, 0 rows affected

mysql> UPDATE incident SET priority = 'HIGH' WHERE incident_id = 6;
Query OK, 1 row affected
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT SYSDATE(6) AS a_commits_at;
+----------------------------+
| a_commits_at               |
+----------------------------+
| 2026-10-29 22:33:26.940922 |
+----------------------------+
1 row in set

mysql> COMMIT;
Query OK, 0 rows affected

-- Terminal B (ananya.rao)
mysql> SELECT SYSDATE(6) AS b_asks_at;
+----------------------------+
| b_asks_at                  |
+----------------------------+
| 2026-10-29 22:33:24.988212 |
+----------------------------+
1 row in set

mysql> START TRANSACTION;
Query OK, 0 rows affected

mysql> SELECT priority, SYSDATE(6) AS b_got_row_at FROM incident WHERE incident_id = 6 FOR UPDATE;
+----------+----------------------------+
| priority | b_got_row_at               |
+----------+----------------------------+
| HIGH     | 2026-10-29 22:33:26.946861 |
+----------+----------------------------+
1 row in set

mysql> UPDATE incident SET priority = 'CRITICAL' WHERE incident_id = 6;
Query OK, 1 row affected
Rows matched: 1  Changed: 1  Warnings: 0

mysql> COMMIT;
Query OK, 0 rows affected

mysql> SELECT priority AS final_priority FROM incident WHERE incident_id = 6;
+----------------+
| final_priority |
+----------------+
| CRITICAL       |
+----------------+
1 row in set
```

Fig. 127 FOR UPDATE: B waits, reads HIGH, writes CRITICAL

SYSDATE(6) is the clock at the moment each statement returned. B asked for the row at 22:33:24.988212, A committed at 22:33:26.940922, and B received the row at 22:33:26.946861: it waited for A's lock and then saw A's committed HIGH.

Deadlock. A locks incident 3 then wants 6; B locks 6 then wants 3. InnoDB detects the cycle, rolls back B with ERROR 1213 and lets A finish. Prevention: lock rows in one fixed order (lowest incident_id first, as sp_escalate_incidents does) and retry on 1213.

```
-- Terminal A (root)
Query OK, 0 rows affected

mysql> START TRANSACTION;
Query OK, 0 rows affected

mysql> UPDATE incident SET priority = 'HIGH' WHERE incident_id = 3;
Query OK, 1 row affected
Rows matched: 1  Changed: 1  Warnings: 0

mysql> DO SLEEP(2);
Query OK, 0 rows affected

mysql> UPDATE incident SET priority = 'HIGH' WHERE incident_id = 6;
Query OK, 1 row affected
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT 'A got both rows' AS a;
+-----------------+
| a               |
+-----------------+
| A got both rows |
+-----------------+
1 row in set

mysql> COMMIT;
Query OK, 0 rows affected

-- Terminal B (ananya.rao)
Query OK, 0 rows affected

mysql> START TRANSACTION;
Query OK, 0 rows affected

mysql> UPDATE incident SET priority = 'HIGH' WHERE incident_id = 6;
Query OK, 1 row affected
Rows matched: 1  Changed: 1  Warnings: 0

mysql> DO SLEEP(2);
Query OK, 0 rows affected

mysql> UPDATE incident SET priority = 'HIGH' WHERE incident_id = 3;
ERROR 1213 (40001) at line 5: Deadlock found when trying to get lock; try restarting transaction
```

Fig. 128 Deadlock detected; B is rolled back

SHOW ENGINE INNODB STATUS confirms it: "*** WE ROLL BACK TRANSACTION (2)", transaction 2 being B.

Isolation levels. In READ COMMITTED, A read incident 6 as OPEN, B changed it to INVESTIGATING, and A's second read inside the same transaction said INVESTIGATING (a non-repeatable read). In REPEATABLE READ, MySQL's default, the second read still said OPEN, because the transaction reads from its own snapshot.

```
-- Terminal A (root)
Query OK, 0 rows affected

mysql> START TRANSACTION;
Query OK, 0 rows affected

mysql> SELECT status AS first_read FROM incident WHERE incident_id = 6;
+------------+
| first_read |
+------------+
| OPEN       |
+------------+
1 row in set

mysql> DO SLEEP(2);
Query OK, 0 rows affected

mysql> SELECT status AS second_read FROM incident WHERE incident_id = 6;
+---------------+
| second_read   |
+---------------+
| INVESTIGATING |
+---------------+
1 row in set

mysql> COMMIT;
Query OK, 0 rows affected

-- Terminal B (ananya.rao)
mysql> UPDATE incident SET status = 'INVESTIGATING' WHERE incident_id = 6;
Query OK, 1 row affected
Rows matched: 1  Changed: 1  Warnings: 0
```

Fig. 129 READ COMMITTED: the second read sees B's committed change

```
-- Terminal A (root)
Query OK, 0 rows affected

mysql> START TRANSACTION;
Query OK, 0 rows affected

mysql> SELECT status AS first_read FROM incident WHERE incident_id = 6;
+------------+
| first_read |
+------------+
| OPEN       |
+------------+
1 row in set

mysql> DO SLEEP(2);
Query OK, 0 rows affected

mysql> SELECT status AS second_read FROM incident WHERE incident_id = 6;
+-------------+
| second_read |
+-------------+
| OPEN        |
+-------------+
1 row in set

mysql> COMMIT;
Query OK, 0 rows affected

-- Terminal B (ananya.rao)
mysql> UPDATE incident SET status = 'INVESTIGATING' WHERE incident_id = 6;
Query OK, 1 row affected
Rows matched: 1  Changed: 1  Warnings: 0
```

Fig. 130 REPEATABLE READ: the second read still sees the snapshot

Isolation level	Dirty read	Non-repeatable read	Phantom read
READ UNCOMMITTED	possible	possible	possible

Isolation level	Dirty read	Non-repeatable read	Phantom read
READ COMMITTED	prevented	possible (Fig. 129)	possible
REPEATABLE READ (default)	prevented	prevented (Fig. 130)	prevented for plain reads (snapshot); locking reads use next-key locks
SERIALIZABLE	prevented	prevented	prevented

Our own IOC trigger under concurrency. Two sessions add the same bad IP at once. A inserts and holds its transaction open; B starts at 22:33:36.168092 and its INSERT waits, because the SELECT inside a trigger fired by an INSERT is a locking read. A commits at 22:33:38.128131, and B is rejected by trg_ioc_no_overlap at 22:33:38.134808: only one version is stored. With unlucky timing InnoDB could instead report a deadlock (ERROR 1213) and roll one session back; either way the rule holds.

```
-- Terminal A (root)
mysql> START TRANSACTION;
Query OK, 0 rows affected

mysql> INSERT INTO ioc (ioc_type, ioc_value, valid_from, recorded_from) VALUES ('ip', '192.0.2.250', '2026-10-01', SYSDATE());
Query OK, 1 row affected

mysql> SELECT SYSDATE(6) AS a_inserted_at;
+----------------------------+
| a_inserted_at              |
+----------------------------+
| 2026-10-29 22:33:35.117892 |
+----------------------------+
1 row in set

mysql> DO SLEEP(3);
Query OK, 0 rows affected

mysql> SELECT SYSDATE(6) AS a_commits_at;
+----------------------------+
| a_commits_at               |
+----------------------------+
| 2026-10-29 22:33:38.128131 |
+----------------------------+
1 row in set

mysql> COMMIT;
Query OK, 0 rows affected

-- Terminal B (ananya.rao)
mysql> SELECT SYSDATE(6) AS b_starts_at;
+----------------------------+
| b_starts_at                |
+----------------------------+
| 2026-10-29 22:33:36.168092 |
+----------------------------+
1 row in set

mysql> INSERT INTO ioc (ioc_type, ioc_value, valid_from, recorded_from) VALUES ('ip', '192.0.2.250', '2026-10-05', SYSDATE());
ERROR 1644 (45000) at line 2: An overlapping version of this indicator already exists

mysql> SELECT SYSDATE(6) AS b_rejected_at;
+----------------------------+
| b_rejected_at              |
+----------------------------+
| 2026-10-29 22:33:38.134808 |
+----------------------------+
1 row in set
```

Fig. 131 The second INSERT waits for the first, then is rejected

14. Indexing and Query Optimisation

Besides the primary keys and the indexes MySQL creates for foreign keys, we added five. idx_src_time (src_ip, log_time) on network_log serves the retro-hunt and is measured below. idx_time and idx_src on attack_event serve searches by time and by source IP; on today's 30 attacks the optimiser may still prefer a full scan, and they are there for when the table grows. idx_value on ioc finds an indicator by value (used by the overlap trigger), and idx_incident (incident_id, changed_at) on incident_audit returns one incident's history in order.

```
mysql> SELECT table_name, index_name,
    ->        GROUP_CONCAT(column_name ORDER BY seq_in_index) AS columns_in_index,
    ->        IF(non_unique = 0, 'yes', 'no') AS is_unique
    -> FROM information_schema.statistics
    -> WHERE table_schema = 'sentinelx'
    ->   AND table_name IN ('network_log', 'attack_event')
    -> GROUP BY table_name, index_name, non_unique
    -> ORDER BY table_name, index_name;
+--------------+--------------+------------------+-----------+
| TABLE_NAME   | INDEX_NAME   | columns_in_index | is_unique |
+--------------+--------------+------------------+-----------+
| attack_event | actor_id     | actor_id         | no        |
| attack_event | asset_id     | asset_id         | no        |
| attack_event | cve_id       | cve_id           | no        |
| attack_event | idx_src      | src_ip           | no        |
| attack_event | idx_time     | event_time       | no        |
| attack_event | malware_id   | malware_id       | no        |
| attack_event | PRIMARY      | event_id         | yes       |
| attack_event | type_id      | type_id          | no        |
| network_log  | dst_asset_id | dst_asset_id     | no        |
| network_log  | idx_src_time | src_ip,log_time  | no        |
| network_log  | PRIMARY      | log_id           | yes       |
+--------------+--------------+------------------+-----------+
11 rows in set (0.00 sec)
```

Fig. 132 Indexes on network_log and attack_event

To measure idx_src_time we dropped it, ran EXPLAIN on the retro-hunt lookup, then created it again and repeated the EXPLAIN.

```
mysql> ALTER TABLE network_log DROP INDEX idx_src_time;
Query OK, 0 rows affected (0.02 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> ANALYZE TABLE network_log;
+----------------------+---------+----------+----------+
| Table                | Op      | Msg_type | Msg_text |
+----------------------+---------+----------+----------+
| sentinelx.network_log | analyze | status   | OK       |
+----------------------+---------+----------+----------+
1 row in set (0.01 sec)

mysql> EXPLAIN SELECT log_id, log_time, dst_asset_id
    -> FROM network_log
    -> WHERE src_ip = '198.51.100.77'
    ->   AND log_time >= '2026-09-15' AND log_time < '2026-10-28'\G
*************************** 1. row ***************************
           id: 1
  select_type: SIMPLE
        table: network_log
   partitions: NULL
         type: ALL
possible_keys: NULL
          key: NULL
      key_len: NULL
          ref: NULL
         rows: 50155
     filtered: 1.11
        Extra: Using where
1 row in set, 1 warning (0.00 sec)

mysql> SELECT COUNT(*) AS hits FROM network_log
    -> WHERE src_ip = '198.51.100.77'
    ->   AND log_time >= '2026-09-15' AND log_time < '2026-10-28';
+------+
| hits |
+------+
|    8 |
+------+
1 row in set (0.01 sec)
```

Fig. 133 EXPLAIN without idx_src_time: full table scan

```
mysql> CREATE INDEX idx_src_time ON network_log (src_ip, log_time);
Query OK, 0 rows affected (0.31 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> ANALYZE TABLE network_log;
+----------------------+---------+----------+----------+
| Table                | Op      | Msg_type | Msg_text |
+----------------------+---------+----------+----------+
| sentinelx.network_log | analyze | status   | OK       |
+----------------------+---------+----------+----------+
1 row in set (0.02 sec)

mysql> EXPLAIN SELECT log_id, log_time, dst_asset_id
    -> FROM network_log
    -> WHERE src_ip = '198.51.100.77'
    ->   AND log_time >= '2026-09-15' AND log_time < '2026-10-28'\G
*************************** 1. row ***************************
           id: 1
  select_type: SIMPLE
        table: network_log
   partitions: NULL
         type: range
possible_keys: idx_src_time
          key: idx_src_time
      key_len: 187
          ref: NULL
         rows: 8
     filtered: 100.00
        Extra: Using index condition
1 row in set, 1 warning (0.00 sec)

mysql> SELECT COUNT(*) AS hits FROM network_log
    -> WHERE src_ip = '198.51.100.77'
    ->   AND log_time >= '2026-09-15' AND log_time < '2026-10-28';
+------+
| hits |
+------+
|    8 |
+------+
1 row in set (0.00 sec)
```

Fig. 134 EXPLAIN with idx_src_time: range read of 8 rows

	Without idx_src_time	With idx_src_time
Access type	ALL (full table scan)	range
Index used	none	idx_src_time
Rows examined (estimate)	50,155	8
Rows actually matching	8	8

Row counts in EXPLAIN are estimates from the table statistics, so they move slightly after every ANALYZE TABLE (50,155 in this run, against 50,021 rows actually stored); the access type is what matters: ALL reads every row, range reads only the matching slice of the index.

The index lets MySQL jump straight to the 8 lines from that IP instead of reading all 50,021 rows. With 50,000 rows both runs still finish in about 0.01 seconds, but a full scan grows with the size of the table while the range scan only reads the matching rows, so the gap widens as logs accumulate.

Reading the EXPLAIN with the index, column by column:

Column	Value	Meaning
type	range (was ALL)	Read one slice of the index instead of every row
key	idx_src_time	The index the optimiser chose

Column	Value	Meaning
key_len	187	Both index columns are used: src_ip VARCHAR(45) in utf8mb4 takes 45 x 4 + 2 bytes, plus 5 bytes for the DATETIME log_time
rows	8 (was 50,155)	Estimated rows to read
Extra	Using index condition	The log_time range is checked inside the index before a row is fetched; dst_asset_id is not in the index, so each matching row is still read

14.1 When an index is not used

idx_src_time is (src_ip, log_time). A filter on log_time alone cannot seek it, because the index is sorted by src_ip first (the leftmost-prefix rule). With skip scan switched off MySQL reads the whole index; MySQL 8.0.13 and later can instead skip-scan it, doing one range read for each of the 1,004 distinct source IPs.

```
mysql> EXPLAIN SELECT /*+ NO_SKIP_SCAN(network_log) */ COUNT(*) FROM network_log
    -> WHERE log_time >= '2026-10-27' AND log_time < '2026-10-28'\G
*************************** 1. row ***************************
           id: 1
  select_type: SIMPLE
        table: network_log
   partitions: NULL
         type: index
possible_keys: NULL
          key: idx_src_time
      key_len: 187
          ref: NULL
         rows: 47293
     filtered: 11.11
        Extra: Using where; Using index
1 row in set, 1 warning (0.00 sec)

mysql> EXPLAIN SELECT COUNT(*) FROM network_log
    -> WHERE log_time >= '2026-10-27' AND log_time < '2026-10-28'\G
*************************** 1. row ***************************
           id: 1
  select_type: SIMPLE
        table: network_log
   partitions: NULL
         type: range
possible_keys: idx_src_time
          key: idx_src_time
      key_len: 187
          ref: NULL
         rows: 5253
     filtered: 100.00
        Extra: Using where; Using index for skip scan
1 row in set, 1 warning (0.00 sec)
```

Fig. 135 Leftmost prefix: full index scan, or skip scan

A function on the indexed column hides it from the optimiser: SUBSTRING_INDEX(src_ip, ...) reads the whole index, while the same filter written as a prefix LIKE becomes a range read of about 16 rows.

```
mysql> EXPLAIN SELECT COUNT(*) FROM network_log
    -> WHERE SUBSTRING_INDEX(src_ip, '.', 3) = '198.51.100'\G
*************************** 1. row ***************************
           id: 1
  select_type: SIMPLE
        table: network_log
   partitions: NULL
         type: index
possible_keys: NULL
          key: idx_src_time
      key_len: 187
          ref: NULL
         rows: 47293
     filtered: 100.00
        Extra: Using where; Using index
1 row in set, 1 warning (0.00 sec)

mysql> EXPLAIN SELECT COUNT(*) FROM network_log
    -> WHERE src_ip LIKE '198.51.100.%'\G
*************************** 1. row ***************************
           id: 1
  select_type: SIMPLE
        table: network_log
   partitions: NULL
         type: range
possible_keys: idx_src_time
          key: idx_src_time
      key_len: 182
          ref: NULL
         rows: 16
     filtered: 100.00
        Extra: Using where; Using index
1 row in set, 1 warning (0.00 sec)
```

Fig. 136 A function hides the index; a prefix LIKE uses it

EXPLAIN ANALYZE runs the query and reports real times and row counts next to the estimates: 8 rows, read in 0.036 ms through idx_src_time.

```
mysql> EXPLAIN ANALYZE SELECT log_id, log_time, dst_asset_id
    -> FROM network_log
    -> WHERE src_ip = '198.51.100.77'
    ->   AND log_time >= '2026-09-15' AND log_time < '2026-10-28';
+----------------------------------------------------------------------------------------------------------------------------
    ----------------------------------------------------------------------------------------------------------------------------
    ----------------------------------------------------------------------------------+
| EXPLAIN
    |
+----------------------------------------------------------------------------------------------------------------------------
    ----------------------------------------------------------------------------------------------------------------------------
    ----------------------------------------------------------------------------------+
| -> Index range scan on network_log using idx_src_time over (src_ip = '198.51.100.77' AND '2026-09-15
    00:00:00' <= log_time < '2026-10-28 00:00:00'), with index condition: ((network_log.src_ip = '198.51.100.77')
    and (network_log.log_time >= TIMESTAMP'2026-09-15 00:00:00') and (network_log.log_time < TIMESTAMP'2026-10-28
    00:00:00'))  (cost=3.86 rows=8) (actual time=0.0341..0.0362 rows=8 loops=1)
 |
+----------------------------------------------------------------------------------------------------------------------------
    ----------------------------------------------------------------------------------------------------------------------------
    ----------------------------------------------------------------------------------+
1 row in set (0.00 sec)
```

Fig. 137 EXPLAIN ANALYZE of the retro-hunt lookup

Indexes are not free: every INSERT into network_log must also update each index, and they take disk and memory. On a 10-row table such as asset an index brings nothing, because the whole table is one page.

15. Backup and Point-in-Time Recovery

A full backup is a snapshot of one moment. The binary log records every change after it, in order, each at a numbered position. To undo a mistake made at 3:15 PM we restore the snapshot and replay the binary log from the snapshot's position up to the position just before the bad statement. We stop at a position, not a time: --stop-datetime has one-second resolution, and the bad statement may share its second with good ones. This was rehearsed on a separate MySQL 8.0.46 server, not run live.

Step	Command or statement	Result
1. Nightly full backup	`mysqldump --single-transaction --source-data=2 --routines --triggers --events --databases sentinelx --result-file=sentinelx_full.sql`	Consistent snapshot; the file records binlog.000002 position 16453827
2. Work after the backup	`INSERT INTO attack_event ... '203.0.113.123' ...` `UPDATE incident SET status = 'INVESTIGATING' WHERE incident_id = 6;`	Must survive the recovery
3. The accident	`DELETE FROM asset_vulnerability;`	All 9 patch records gone
4. Find it	`SHOW BINLOG EVENTS IN 'binlog.000002' FROM 16453827;`	The bad transaction starts at position 16454920

5. Restore and replay	`mysql --init-command="SET sql_log_bin=0" -e "source sentinelx_full.sql"`
	`mysqlbinlog --read-from-remote-server --start-position=16453827 --stop-position=16454920 --disable-log-bin --result-file=replay.sql binlog.000002`
	`mysql -e "source replay.sql"`
		Snapshot back, then the two good transactions replayed

6. Check	`SELECT COUNT(*) FROM asset_vulnerability;`	9 rows back; the post-backup attack and the status change are present (Fig. 139)

```
mysql> SHOW BINLOG EVENTS IN 'binlog.000002' FROM 16453827 LIMIT 19;
+---------------+----------+----------------+-----------+-------------+-----------------------------------------------+
| Log_name      | Pos      | Event_type     | Server_id | End_log_pos | Info                                          |
+---------------+----------+----------------+-----------+-------------+-----------------------------------------------+
| binlog.000002 | 16453827 | Anonymous_Gtid |         1 |    16453906 | SET @@SESSION.GTID_NEXT= 'ANONYMOUS'          |
| binlog.000002 | 16453906 | Query          |         1 |    16453996 | BEGIN                                         |
| binlog.000002 | 16453996 | Table_map      |         1 |    16454085 | table_id: 551 (sentinelx.attack_event)        |
| binlog.000002 | 16454085 | Table_map      |         1 |    16454163 | table_id: 554 (sentinelx.incident)            |
| binlog.000002 | 16454163 | Table_map      |         1 |    16454229 | table_id: 556 (sentinelx.incident_event)      |
| binlog.000002 | 16454229 | Write_rows     |         1 |    16454307 | table_id: 551 flags: STMT_END_F               |
| binlog.000002 | 16454307 | Xid            |         1 |    16454338 | COMMIT /* xid=10886 */                        |
| binlog.000002 | 16454338 | Anonymous_Gtid |         1 |    16454417 | SET @@SESSION.GTID_NEXT= 'ANONYMOUS'          |
| binlog.000002 | 16454417 | Query          |         1 |    16454508 | BEGIN                                         |
| binlog.000002 | 16454508 | Table_map      |         1 |    16454586 | table_id: 554 (sentinelx.incident)            |
| binlog.000002 | 16454586 | Table_map      |         1 |    16454680 | table_id: 555 (sentinelx.incident_audit)      |
| binlog.000002 | 16454680 | Update_rows    |         1 |    16454802 | table_id: 554                                 |
| binlog.000002 | 16454802 | Write_rows     |         1 |    16454889 | table_id: 555 flags: STMT_END_F               |
| binlog.000002 | 16454889 | Xid            |         1 |    16454920 | COMMIT /* xid=10889 */                        |
| binlog.000002 | 16454920 | Anonymous_Gtid |         1 |    16454999 | SET @@SESSION.GTID_NEXT= 'ANONYMOUS'          |
| binlog.000002 | 16454999 | Query          |         1 |    16455079 | BEGIN                                         |
| binlog.000002 | 16455079 | Table_map      |         1 |    16455159 | table_id: 550 (sentinelx.asset_vulnerability) |
| binlog.000002 | 16455159 | Delete_rows    |         1 |    16455406 | table_id: 550 flags: STMT_END_F               |
| binlog.000002 | 16455406 | Xid            |         1 |    16455437 | COMMIT /* xid=10895 */                        |
+---------------+----------+----------------+-----------+-------------+-----------------------------------------------+
```

Fig. 138 The binary log after the backup: two good transactions, then the DELETE at 16454920

```
mysql> SELECT COUNT(*) AS patches FROM asset_vulnerability;
+---------+
| patches |
+---------+
|       9 |
+---------+

mysql> SELECT COUNT(*) AS post_backup_attack FROM attack_event WHERE src_ip = '203.0.113.123';
+--------------------+
| post_backup_attack |
+--------------------+
|                  1 |
+--------------------+

mysql> SELECT incident_id, status FROM incident WHERE incident_id = 6;
+-------------+---------------+
| incident_id | status        |
+-------------+---------------+
|           6 | INVESTIGATING |
+-------------+---------------+
```

Fig. 139 After restore and replay: the deleted rows are back and the later work is kept

Durability after a power failure is a different mechanism: InnoDB writes every change to its redo log and flushes it to disk at each COMMIT (innodb_flush_log_at_trx_commit = 1), the doublewrite buffer protects against half-written pages, and after a crash InnoDB replays the redo log. A committed DELETE is durable too; only a backup and the binary log can undo it. Users and grants live in the mysql schema and are re-created by 12_security.sql rather than by this dump.

16. Testing

13_tests.sql holds 59 automatic tests. Each one states a claim made in this report and checks it: either a value (sp_check) or the exact error number a statement must raise (sp_expect_error, which runs the statement as dynamic SQL and catches the error). Both helpers are SQL SECURITY INVOKER, so a test never runs with more rights than the user running it. The runner executes them as root and then logged in as the analyst, the viewer and the admin.

ID	Run as	What it checks	Expected	Actual	Result
T01	root	16 tables plus incident_audit	17	17	PASS
T02	root	network_log is fully loaded	50021	50021	PASS
T03	root	CHECK rejects criticality 9	3819	3819	PASS
T04	root	UNIQUE rejects a second web-01	1062	1062	PASS
T05	root	FK rejects an unknown attack type	1452	1452	PASS
T06	root	FK blocks deleting an attacked asset	1451	1451	PASS
T07	root	CHECK: CLOSED needs closed_at	3819	3819	PASS
T08	root	ENUM rejects priority URGENT (strict mode)	1265	1265	PASS
T09	root	ON DELETE CASCADE clears incident_event	0	0	PASS
T10	root	aggregated view is not updatable	1288	1288	PASS
T11	root	severity 9 attack opens an incident	11	11	PASS
T12	root	repeat within 1 hour joins that incident	11	11	PASS
T13	root	retro-hunt found 8 old logs for 198.51.100.77	8	8	PASS
T14	root	overlapping IOC version is rejected	1644	1644	PASS
T15	root	IOC history is append-only	1644	1644	PASS
T16	root	fn_severity_label(9.8), (6.5)	CRITICAL/MEDIUM	CRITICAL/MEDIUM	PASS
T17	root	cannot assign a closed incident	1644	1644	PASS
T18	root	cannot assign an incident to a viewer	1644	1644	PASS
T19	root	bad log batch raises the FK error	1452	1452	PASS
T20	root	... and leaves none of its rows behind	0	0	PASS
T21	root	savepoint kept the IOC, undid the priority change	1/CRITICAL	1/CRITICAL	PASS
T22	root	NOT IN over a NULL returns nothing	0	0	PASS
T23	root	audit trail names rahul for his claim of incident 8	rahul.mehta	rahul.mehta	PASS
T24	root	audit trail is append-only (DELETE)	1644	1644	PASS
T25	root	audit trail is append-only (UPDATE)	1644	1644	PASS
T26	root	injectable procedure was removed	0	0	PASS
T27	root	admin role has no CREATE/ALTER/DROP/TRIGGER	0	0	PASS
T28	root	a closed IOC version can be replaced by a new one	0	0	PASS
T29	root	all three accounts lock after 3 failed logins	3	3	PASS
T30	root	escalation cursor raised incident 6 from MEDIUM to HIGH	1	1	PASS
T31	root	fn_asset_risk matches its formula for every asset	0	0	PASS
T32	root	an IOC version cannot be rewritten	1644	1644	PASS

ID	Run as	What it checks	Expected	Actual	Result
T33	root	a closed IOC version cannot be reopened	1644	1644	PASS
T34	root	admin has no write privilege on incident_audit	0	0	PASS
T35	root	search procedure ignores a forged @me (root is no analyst)	1644	1644	PASS
S01	rahul (analyst)	analyst cannot read a base table	1142	1142	PASS
S02	rahul (analyst)	analyst can read through the view	0	0	PASS
S03	rahul (analyst)	analyst cannot run DDL	1142	1142	PASS
S04	rahul (analyst)	v_my_incidents shows only his own rows	0	0	PASS
S05	rahul (analyst)	WITH CHECK OPTION blocks handing an incident over	1369	1369	PASS
S06	rahul (analyst)	procedure blocks giving his incident away	1644	1644	PASS
S07	rahul (analyst)	procedure blocks taking a colleague's incident	1644	1644	PASS
S08	rahul (analyst)	procedure lets him claim an unassigned incident	0	0	PASS
S09	rahul (analyst)	analyst cannot write the ioc table directly	1142	1142	PASS
S10	rahul (analyst)	analyst reports an IOC through the procedure	0	0	PASS
S11	rahul (analyst)	analyst cannot read the audit trail	1142	1142	PASS
S12	rahul (analyst)	SET timestamp cannot backdate a reported IOC	0	0	PASS
V01	divya (viewer)	viewer can read the risk view	0	0	PASS
V02	divya (viewer)	viewer cannot read a base table	1142	1142	PASS
V03	divya (viewer)	viewer cannot delete	1142	1142	PASS
V04	divya (viewer)	viewer cannot run the assign procedure	1370	1370	PASS
V05	divya (viewer)	viewer cannot update the risk view	1142	1142	PASS
A01	ananya (admin)	admin can change incident data	0	0	PASS
A02	ananya (admin)	... and the audit trail names the admin	ananya.rao	ananya.rao	PASS
A03	ananya (admin)	admin cannot delete audit rows	1142	1142	PASS
A04	ananya (admin)	admin cannot TRUNCATE the audit trail	1142	1142	PASS
A05	ananya (admin)	admin cannot change the schema	1142	1142	PASS
A06	ananya (admin)	admin cannot forge an audit row	1142	1142	PASS
A07	ananya (admin)	admin cannot edit an audit row	1142	1142	PASS

Error numbers: 1062 duplicate key, 1142 privilege denied, 1265 ENUM value rejected, 1288 view not updatable, 1369 CHECK OPTION failed, 1370 EXECUTE denied, 1451/1452 foreign key, 1644 SIGNAL from our triggers and procedures, 3819 CHECK constraint; 0 means the statement had to succeed.

Run as	Passed	Failed
root	35	0

Run as	Passed	Failed
rahul (analyst)	12	0
divya (viewer)	5	0
ananya (admin)	7	0
Total	59	0

Each test step ends by printing its own passed/failed count; the totals above are read from those lines.

17. Scope Revision and Design Changes since Review 2

The Review 2 report described more than the project contains. Sections 5.2, 6 and 19 and Fig. 1 listed a FastAPI backend, a React dashboard and machine-learning jobs written with pandas and scikit-learn. These were planned but never built, and none of them are in the repository. Fig. 1 also showed a Python ingestion pipeline and a dataset of 600,000 logs and 115,000 events. The Python in the project only generates the test data and the step files and renders the captured output into figures, and the dataset is 50,021 log lines and 30 attacks. We should have marked these parts as planned work, and we are correcting that here.

For Review 3 we have narrowed the scope to what a DBMS project should be judged on: the database design, the SQL, and the security of the database itself. The database was always the core of SentinelX. Every rule that matters already runs inside MySQL as a constraint, trigger, procedure or privilege, so removing the application layer removes no behaviour. Users work through the mysql client and MySQL Workbench, and what each user can do is decided by their role, not by an application.

Removed	Where Review 2 claimed it	Review 3 status
FastAPI backend	5.2 and 6 (p. 5), 19 (p. 74)	Dropped from scope. Roles, views and procedures are the access layer (12.7).
React SOC dashboard	5.2 (p. 5), 7 (p. 6), 19 (p. 75), Fig. 1	Dropped from scope. The "Search and reporting" module is the views v_attack_details and v_asset_risk.
scikit-learn ML jobs	5.2 and 6 (p. 5), 9 (pp. 13-14), 10 (pp. 20-21), A7 (p. 25), C5 (p. 29)	Dropped from scope. The two columns they were meant to fill were changed (changes 12 and 13).
Python ingestion / ETL	Fig. 1 (p. 5)	Never built. Test data comes from tools/gen_seed.py and a recursive CTE.

Design changes since Review 2 (numbering continues from the Review 2 change log):

#	Change	Reason (details in the section concerned)
12	attack_event.predicted_severity renamed to triage_priority: a priority an analyst sets, NULL until triaged	No ML job exists to predict anything.
13	malware.cluster_id dropped	Only the ML job would have filled it.
14	01_ddl.sql selects the database again after re-creating it	Run on its own, the script dropped the database it was using and stopped with ERROR 1046 at the first CREATE TABLE.
15	sp_assign_incident checks who is calling it	Our own testing found a privilege escalation.
16	Analyst role reads only views and runs only procedures	Review 2 granted analysts SELECT on ten base tables, plus INSERT on ioc, so views were not the only access path.
17	Accounts lock after 3 wrong passwords (FAILED_LOGIN_ATTEMPTS 3 PASSWORD_LOCK_TIME 1)	Before this, any number of wrong passwords could be tried.
18	New procedure sp_load_log_batch with an exit handler that rolls back	MySQL undoes only the failed statement, not the transaction.
19	Four more EXPLAIN plans (section 14.1)	They show the leftmost-prefix rule (full index scan, and MySQL 8's skip scan as the alternative), a function on the column that hides the index, the same filter rewritten as a range read, and EXPLAIN ANALYZE with real timings.

#	Change	Reason (details in the section concerned)
20	13_tests.sql: 59 automatic tests	35 run as root, 12 as the analyst, 5 as the viewer and 7 as the admin.
21	New table incident_audit with triggers on incident	Every UPDATE or DELETE of an incident is recorded with the account that made it (USER()), the time, and the old and new status, priority and owner.
22	sx_admin lost CREATE, ALTER, DROP and TRIGGER	Review 2 gave the admin role ALL PRIVILEGES, which included dropping the audit triggers or truncating the audit table (TRUNCATE does not fire triggers).
23	SQL-injection demonstration (section 12.7)	A search procedure built with CONCAT lets the analyst read every incident and every analyst's email, although he only holds EXECUTE:
24	trg_ioc_no_overlap ignores versions closed with recorded_to	Found while building the demo.
25	Backup and point-in-time recovery runbook (demo/07_backup_recovery.md)	Rehearsed with mysqldump and the binary log, stopping at the binlog position of the bad DELETE.
26	Architecture figure redrawn (tools/draw_architecture.py)	The old Fig.
27	Relational schema diagram redrawn from the DDL (tools/draw_relational_schema.py)	The hand-drawn Review 2 version still showed event_time and log_time inside primary keys, removed in Review 2 (change 10), and the two ML columns.
28	sx_admin writes per data table; on incident_audit it only has SELECT	Found in our pre-submission review:
29	SYSDATE() instead of NOW() in sp_report_ioc, the audit triggers and the retro-hunt	Any session can run SET timestamp, and NOW() follows it, so an analyst could backdate an indicator or an audit row.
30	New trigger trg_ioc_only_close	Only DELETE was blocked, so an UPDATE could rewrite an indicator or reopen a closed version.
31	retro_hunt_run.started_at is the time the search really ran	It was always a copy of the indicator's recorded_from, which made it redundant.
32	sp_search_incidents refuses a caller it cannot match to an analyst	It read the caller's id into a session variable without clearing it, so an account with no analyst row could set the variable by hand and read another analyst's incidents.
33	REVOKE shown by behaviour; normalisation and update anomaly shown on the data; updatability listed for all six views	The Review 3 template asks for before/after evidence and anomalies demonstrated, not only statements.

18. Review 3 Requirements Checklist

Requirement	Evidence	Where	Status
Complete database with all tables populated	17 tables, 50,021 log lines, 30 attacks, row counts	9, 10	Done
Full scripts, re-runnable from scratch	01_ddl.sql to 13_tests.sql run cleanly on an empty server; one-click runner; fixed ERROR 1046	10, 17	Done
Advanced SQL	41 queries in 7 categories; 11 complex queries, each matching its hand prediction	11, 12.1	Done
Views (2-3 minimum)	6 views with the full template, updatable or not for each: 5-table join, aggregated, row-restricting, v_ioc, two Review 1 views	12.2	Done

Requirement	Evidence	Where	Status
Stored procedures (at least 2)	sp_assign_incident, sp_attack_report, sp_report_ioc, sp_whoami, sp_load_log_batch, search pair	12.3, 12.7, 13	Done
User-defined functions (at least 2)	fn_severity_label, fn_asset_risk	12.4	Done
Triggers (at least 2)	9 triggers: auto-incident, retro-hunt, three IOC rules, four for the audit trail	12.5	Done
Cursors (at least 2)	sp_escalate_incidents, sp_asset_risk_report	12.6	Done
Transactions and concurrency	SAVEPOINT, ROLLBACK, exit-handler rollback; lost update, FOR UPDATE, deadlock, two isolation levels and the IOC trigger, each run in two real sessions	13	Done
Indexing and query optimisation	EXPLAIN before/after (50,155 estimated rows to 8); leftmost prefix, skip scan, function on column, EXPLAIN ANALYZE	14	Done
Security implementation	3 roles, 3 users, GRANT/REVOKE shown by behaviour, least privilege, secure exposure, lockout, forge-proof audit trail, SQL injection	12.7	Done
Testing (15 or more cases)	59 tests run as four users, 59 pass	16	Done
Normalisation and anomalies	Normalisation steps, an FD check and an update anomaly shown on the data	9.1	Done
Backup and recovery	mysqldump plus binary log, rehearsed to the position before a DELETE	15	Done
Final report	This document	-	Done
Live demonstration and viva	Run sheet and rerunnable segment files (demo/README.md), viva map (docs/viva_map.md)	repository	Ready

19. Limitations and Future Scope

- There is no application or user interface. The project is the database: users connect with the mysql client.
- There is no automatic detection or prediction. Attacks and their severity are recorded as input, as they would come from a firewall or IDS, and analysts set the triage priority.
- The data is a hand-made test set, not real traffic. Attacker IPs come from documentation-only address ranges, and the attacker and malware names are made up. The CVE IDs are real.
- Retro-hunts only work for IP addresses. Domains and file hashes would need DNS and file logs.
- MySQL has no exclusion constraint, so the rule that indicator versions never overlap lives in a trigger.
- Triggers record only changes that succeed. Failed logins and denied privileges need the server's error log or an audit plugin, which MySQL Community does not include.
- Future: run the procedures under a dedicated low-privilege definer account instead of root, so that a mistake in one procedure cannot reach the whole database.

- Future: very high log volumes would need archive tables or partitioning for old logs, and a streaming loader.

20. Conclusion

Review 1 gave us a clean 3NF design and Review 2 showed it working. Review 3 made it defensible. Testing our own work found eight real defects: a script that only ran because our runner hid its mistake; a stored procedure that let an analyst bypass the row-level security we were proud of; a rollback demonstration that relied on a manual ROLLBACK; a trigger that made closed indicators permanent; an audit trail the administrator could add forged rows to; indicator history that an UPDATE could rewrite; times that a session could backdate with SET timestamp; and a search procedure that trusted a session variable. Each is fixed and has a test. The database now enforces its own rules, limits every user to what they need, records who changed each incident, behaves correctly in every concurrent scenario we ran, recovers from an accidental delete, and proves this with 59 tests that pass on a clean build. The biggest lesson was that a security feature is only as strong as the code that runs with more rights than its caller.

21. Acknowledgement

An AI assistant (Claude, by Anthropic) was used to review our SQL for defects, to help write parts of the Review 3 scripts and to help draft this report. Every script was run on MySQL 8.0.46, and every result in this report is the real output of that run.

22. Work Break Down

Registration Number	Name	Work Assigned (Reviews 1 and 2)	Review 3 responsibility (presents and defends)
24BCI0061	Shlok Gupta	Retro-hunt and IOC triggers, indexes and EXPLAIN, transactions, FastAPI backend (dropped from scope, section 17)	Transactions and concurrency (13), indexing and optimisation (14), backup and recovery (15), IOC trigger fixes. Demo: steps 1 and 4, introduction and close
24BCI0086	Shreya Bhushan	ER diagram, normalisation, DDL and constraints, views, stored procedures and cursors, SQL security	Security (12.7): role and privilege fixes, audit trail, SQL injection; views (12.2). Demo: step 2 and concurrency terminal A
24BDS0158	Yashneil Purkayastha	Test data and DML, SQL queries A-G, complex business queries, functions, React dashboard (dropped from scope, section 17), the Review 2 report	Testing (16, 59 tests), complex-query verification (12.1), normalisation evidence (9.1), this report. Demo: SQL injection, steps 3 and 5, concurrency terminal B

23. Review Evaluation

COMPONENT	MARKS	SHLOK GUPTA	SHREYA BHUSHAN	YASHNEIL PURKAYASTHA
Project work with screenshot	5			

COMPONENT	MARKS	SHLOK GUPTA	SHREYA BHUSHAN	YASHNEIL PURKAYASTHA
Viva	5			
TOTAL	10 MARKS			